

Plataforma Web Inteligente para la Creación Automatizada de Landing Pages



WebLandingSuite

Integrantes del grupo:

- Juan Carlos Agüero Carcamo
- Rene Marcos Antonio Orellana Aguirre
- Isaac Amaru Gonzales Saavedra

Asignatura / Docente:

- Taller Aplicado de Programación
- Cristian Manuel Calderon Sanchez

Índice

Índice.....	2
Introducción.....	6
Contexto del proyecto.....	7
Antecedentes y motivación.....	7
Origen del problema.....	7
Publico objetivo general.....	7
Estado del arte y Tabla de Homologación.....	8
Descripción del problema u oportunidad.....	9
Definición del problema Central.....	9
Descripción del Impacto.....	9
Análisis de Causas y efectos: Árbol del Problema.....	9
Conclusión: Enfoque de la solución.....	11
Descripción del mercado objetivo.....	11
Objetivo general y objetivos específicos.....	12
Objetivo General.....	12
Objetivos Específicos.....	13
Coherencia y Alineación del Proyecto.....	13
Situación inicial del cliente (AS-IS).....	14
Contexto de la situación Actual.....	14
Descripción del Proceso Actual.....	14
Puntos de Dolor identificados en el Proceso.....	15
Diagrama del Proceso Actual (Flujo AS-IS).....	15
Planificación.....	16
Estructura del Trabajo.....	16
Inicio y Planificación.....	16
Servicios Cloud a utilizar (IaaS / PaaS / SaaS).....	17
Justificación de la Arquitectura Cloud.....	17
Fronted (Capa de Presentación - PaaS).....	18
Backend (Capa de Lógica y API - PaaS).....	19
Motor Inteligencia Artificial (SaaS).....	19
Conceptualización.....	20
Descripción de la solución (Modelo TO-BE).....	20
Funcionalidades principales:.....	21
Entregables de la Etapa de Construcción.....	21
Consideraciones de Calidad y Seguridad.....	22
Alcance, supuestos y restricciones.....	22
Alcance del Proyecto.....	22
Fuera de Alcance (Que no incluye el sistema).....	23
Entregables del Proyecto.....	23
Presupuestos.....	23
Restricciones.....	24
Metodología de desarrollo y gestión del proyecto.....	24

1. Metodología Elegida: Híbrida (Cascada + Scrum Adaptado).....	24
2. Fases del Proyecto y su metodología Aplicada.....	25
3. Organización del equipo y Roles.....	25
4. Gestión de Tareas en Jira.....	26
5. Flujo de Trabajo y Control de Calidad.....	27
6. Definition of Done (Definición de Terminado).....	29
7. Lista de Hitos del Proyecto.....	29
Documentos y diagramas de diseño de la solución.....	31
1. Historias de Usuario (Backlog Ágil).....	31
Módulo 1: Portal de Autenticación.....	31
Módulo 2: Gestión de Planes y Pagos.....	31
Módulo 3: Motor de IA y Creación.....	32
Módulo 4: Gestión de Proyectos (Dashboard).....	32
Módulo 5: Auditoría y Trazabilidad (Logs).....	32
Modulo 6: Gestion de Usuario.....	32
2. Diagrama de Casos de Uso y Actores.....	32
3. Modelo de Datos (Diagrama Entidad-Relación).....	34
Explicación del Modelo.....	34
Arquitectura de software y patrones de diseño.....	35
1. Descripción de la Arquitectura General del Sistema.....	35
2. Identificación y Explicación de los Patrones de Diseño.....	36
2.1 Patrones en el Frontend (React).....	36
2.2 Patrones en el Backend (Spring Boot).....	36
3. Diagrama de Contexto del Sistema (Modelo C4).....	37
3.1 Diagrama de Contexto del Sistema (Nivel 1).....	37
3.2 Diagrama de Contenedores (Modelo C4 - Nivel 2).....	38
3.3 Diagrama de Arquitectura.....	40
Tecnologías, lenguaje y dependencias (stack definitivo).....	41
1. Lenguajes y Frameworks Definidos.....	41
2. Librerías Clave y Dependencias.....	41
3. Estándares de Código y Arquitectura de Carpetas.....	42
4. Repositorios, Ramas Principales y Flujo de Trabajo.....	44
Flujo de Trabajo (Git Workflow):.....	44
Configuración de servidor de producción.....	45
1. Plataformas y Componentes Elegidos.....	45
2. Requisitos del Servidor y Entornos de Ejecución (Runtimes).....	45
3. Instrucciones de Despliegue.....	46
Paso 1: Configuración de la Base de Datos en Neon.....	46
Paso 2: Despliegue de los Servicios Backend en Render.....	47
Paso 3: Integración Continua y Despliegue (CI/CD) del Frontend en Vercel.....	48
4. Registro (Logs) y Monitoreo Básico.....	49
Estado de avance del desarrollo (evidencia de producto final).....	50
1. Descripción General del Proyecto.....	50
2. Funcionalidades Implementadas.....	50

3. Evidencia de Funcionalidad.....	51
1. Registro e inicio de sesión con JWT.....	51
2. Formulario dinámico de captura de datos.....	52
3. Generación de Landing Page con IA.....	54
4. Previsualización en tiempo real y descargar.....	54
5. Envío de correos electrónicos transaccionales.....	55
4. Calidad del Desarrollo y Buenas Prácticas.....	55
5. Evidencia de Desarrollo.....	56
Integraciones necesarias.....	60
1. OpenRouter - Motor de Inteligencia Artificial.....	60
2. API REST Backend - Comunicación Frontend ↔ Spring Boot.....	61
3. Neon - Base de Datos PostgreSQL.....	64
4. Uptime Robot - Monitoreo de Disponibilidad de Produccion.....	65
5. Conclusión de Integraciones.....	67
Estrategia y enfoque de calidad.....	68
Definición de Calidad para WebLandingSuite.....	68
Atributos de Calidad Priorizados.....	68
1. Seguridad (Prioridad Crítica).....	68
2. Funcionalidad (Prioridad Alta).....	68
3. Fiabilidad (Prioridad Alta).....	68
4. Rendimiento (Prioridad Alta).....	69
5. Mantenibilidad (Prioridad Media).....	69
6. Usabilidad (Prioridad Menor en pruebas formales).....	69
Niveles y Tipos de Prueba: de lo más pequeño a lo más completo.....	69
Nivel 1 - Pruebas Unitarias.....	69
Nivel 2 - Pruebas de Integración.....	69
Nivel 3 - Pruebas de Seguridad.....	70
Nivel 4 - Pruebas de Regresión.....	70
Nivel 5 - Pruebas de Rendimiento.....	70
Enfoque: Automatizado con Validación Manual Complementaria.....	70
Definition Of Done para la Capa de Pruebas.....	71
Plan de Pruebas.....	72
1. Alcance de las Pruebas.....	72
2. Elementos y Funcionalidades a Probar.....	73
Funcionalidades de regresión (re-verificación ante cambios):.....	74
3. Criterios de Aceptación y Rechazo.....	74
Criterios de rechazo (condiciones que impiden dar el ciclo por cerrado):.....	75
4. Criterios de Suspensión y Reanudación.....	75
Condiciones de suspensión:.....	75
Condiciones de reanudación:.....	75
5. Recursos, Roles y Cronograma.....	76
Cronograma estimado:.....	77
6. Premisas, Dependencias y Riesgos.....	77
Premisas (supuestos que se asumen como verdaderos):.....	77

Dependencias:.....	77
Riesgos y Planes de mitigación:.....	78
Tipos de pruebas aplicadas.....	79
1. Pruebas Unitarias.....	79
2. Pruebas de Humo (Smoke Test).....	80
3. Pruebas de Integración.....	80
4. Pruebas Funcionales y de Extremo a Extremo (E2E).....	81
5. Pruebas de Regresión.....	82
6. Pruebas de Rendimiento o Carga.....	82
7. Pruebas de Seguridad.....	83
Herramientas y software utilizados.....	84
Herramientas de Prueba.....	84
Herramientas de Gestión de Pruebas e Incidencias.....	86
Herramientas de Calidad de Código y Cobertura.....	87
Ejecución de pruebas y resultados.....	89
Resumen General de Ejecución.....	89
1. Pruebas Unitarias y de Cobertura – Frontend (Vitest).....	90
2. Pruebas Unitarias, Integración y Cobertura – Backend Java (JUnit 5 + JaCoCo).....	92
3. Suite de Regresión – Backend Java.....	93
4. Pruebas de Rendimiento y Carga – Gatling.....	94
5. Pruebas E2E – Playwright.....	98
Conclusión de Resultados.....	99
Evidencias de mejora y correcciones.....	99
Defecto 1 - Registro duplicado no retornaba conflicto HTTP correcto.....	99
Defecto 2 - Validación de DTO no rechazaba correos con formato invalido.....	100
Defecto 3 - Acceso a endpoint de administrador sin token retornaba 403 en lugar de ruta bloqueada consistente.....	101
Defecto 4 - StepReview no ocultaba secciones correctamente según planLevel.....	102
Defecto 5 - Microservicio Python retornaba 503 por cold start en Render (free tier).....	103
Conclusión.....	105
Anexos.....	107

Introducción

En la actual era digital, poseer una presencia en internet ha dejado de ser un lujo para convertirse en un requisito fundamental para la supervivencia y el crecimiento de cualquier emprendimiento, profesional independiente o microempresa. Una página web actúa como la principal carta de presentación y el canal de ventas más directo en el mercado global. Sin embargo, históricamente, la creación de un sitio web ha estado ligada a procesos complejos que demandan conocimientos especializados en diseño y programación, o en su defecto, una inversión económica significativa para contratar a agencias o terceros. Hoy en día, el vertiginoso avance de la Inteligencia Artificial está transformando radicalmente este panorama, abriendo la puerta a la automatización de tareas que antes requerían horas de trabajo manual, y democratizando el acceso a herramientas tecnológicas de alto nivel para usuarios sin un perfil técnico.

Es precisamente en esta brecha donde surge la motivación y oportunidad de este proyecto. Muchos microempresarios y trabajadores independientes se enfrentan a la frustración de necesitar una página de aterrizaje (*landing page*) rápida para validar una idea de negocio o captar clientes, pero se ven frenados por la pronunciada curva de aprendizaje de los constructores de sitios tradicionales (CMS) o por los prohibitivos costos del desarrollo a medida. Para dar respuesta a esta problemática, se construirá una plataforma web innovadora que automatiza el diseño web. El sistema permitirá a los clientes completar un formulario intuitivo con la información de su negocio y, mediante el uso estratégico de Inteligencia Artificial, generará automáticamente el código de una página web lista para usar. La gran innovación radica en ofrecer opciones escalonadas: el usuario podrá elegir entre distintos niveles de calidad y diseño (básico, intermedio o premium), pagando un precio justo y proporcional al nivel de sofisticación deseado.

El presente informe detalla exhaustivamente la planificación, diseño y viabilidad de esta solución tecnológica. En las siguientes páginas, el lector encontrará en primer lugar la definición formal del proyecto, incluyendo los objetivos generales y específicos que guiarán al equipo desarrollador. Posteriormente, se profundizará en el contexto que justifica la necesidad comercial de la plataforma, seguido de la descripción de la metodología de trabajo híbrida adoptada para asegurar entregas continuas y de calidad. Finalmente, el documento expondrá las especificaciones técnicas del sistema, abarcando la arquitectura de software elegida, los lenguajes de programación y la infraestructura que darán vida a este motor inteligente de generación web.

Contexto del proyecto

Antecedentes y motivación

En el ecosistema comercial actual, la digitalización ha dejado de ser una ventaja competitiva para transformarse en un requisito básico de supervivencia. Una presencia en

internet solida válida la credibilidad de un negocio y actúa como su principal canal de captación de clientes. Sin embargo, el desarrollo web tradicional sigue siendo un proceso inherentemente técnico y costoso. Para un microempresario o un trabajador independiente, destinar gran parte de su capital inicial a contratar una agencia de desarrollo, o invertir semanas intentando aprender a programar, resulta inviable. La principal motivación de este proyecto es democratizar el acceso a la tecnología web, utilizando el auge de la inteligencia artificial para reducir dramáticamente las barreras técnicas y económicas. Se busca empoderar a los creadores de negocios entregándoles una herramienta capaz de generar una landing page profesional en minutos, permitiendo enfocar su tiempo y recursos en lo que realmente importa: hacer crecer su negocio.

Origen del problema

La problemática se detectó a través de la observación directa del mercado de trabajadores independientes y microempresas. Es común observar que muchos emprendedores emergentes dependen exclusivamente de perfiles en redes sociales (como Instagram o Facebook) para vender sus servicios, debido a la frustración que les genera intentar crear una página web. Al indagar en estas experiencias, se evidencia que los usuarios no técnicos se sienten abrumados ante la pronunciada curva de aprendizaje de los constructores de sitios actuales. Prometen ser fáciles de usar, pero en la práctica requieren nociones de diseños, estructura y configuración de dominios que el usuario promedio no posee, terminando en el abandono del proyecto o en páginas con un diseño deficiente que perjudican la imagen de la marca.

Publico objetivo general

El sistema está dirigido principalmente a microempresarios, profesionales independientes (freelancer), consultores y pequeñas empresas (pymes). Se trata de usuarios con nulos o bajos conocimientos en programación y diseño web, que cuentan con presupuesto limitado pero que tienen la necesidad urgente de lanzar una página de aterrizaje estética para validar una idea de negocio, promocionar un servicio o capturar datos de clientes potenciales (leads).

Estado del arte y Tabla de Homologación

Actualmente, la necesidad de tener una página web se resuelve de tres formas principales: contratando desarrollo a medida (inaccesible por precio), utilizando gestores de contenido complejos (como WordPress, que requieren mantenimiento técnico constante), o mediante constructores visuales (Site Builders). Si bien estas herramientas existen, no resuelven el problema de fondo para este nicho específico, ya que obligan al

usuario a suscribirse a planes mensuales costosos o a diseñar desde cero sobre lienzos en blanco.

A continuación, se presenta la tabla de homologación comparando las soluciones actuales del mercado frente a la propuesta de este proyecto:

Solución / Competidor	Que hace	Ventajas	Desventajas	Que haremos distinto (Nuestra propuesta)
Constructores Visuales	Permite crear sitios web mediante interfaces de arrastrar y soltar.	Gran cantidad de planillas. Incluye hosting. No requiere saber programar.	Altas cuotas de suscripción mensual obligatorias. Curva de aprendizaje para lograr un buen diseño. El código no pertenece al usuario.	Generación por IA y pago único: El usuario no diseña nada manualmente; llena un formulario y la IA hace el trabajo. Se cobra un pago único por la generación de código descargable, eliminando los amarres de suscripción mensuales.
Gestores de Contenido (WordPress)	Sistema de gestión de contenido (CMS) altamente personalizable mediante temas y plugins.	Software base gratuito. Control total sobre el sitio y el servidor. Escalabilidad infinita.	Requiere conocimientos técnicos para configurar servidores, bases de datos y seguridad. Alto costo de mantenimiento y riesgo de vulnerabilidades si no se actualiza.	Simplicidad estética: Entregaremos archivos HTML/CSS puros y estáticos, listos para subir a cualquier hosting básico sin bases de datos complejas ni necesidad de mantenimiento o actualizaciones de seguridad.
Generadores de Código IA	Generan bloques de código HTML/CSS a partir de descripciones de texto libre (prompts).	Gratuitos o de muy bajo costos. Respuesta instantánea. Flexibilidad total en lo que se puede pedir.	Entregan código crudo, a menudo incompleto o con errores de sintaxis. El usuario debe saber cómo ensamblar los archivos, vincular el CSS y probarlo.	Ingeniería de Prompts invisible y escalonadas: El usuario no redacta prompts; el sistema abstrae esa complejidad. El backend ensambla, limpia y valida el código generado en 3 niveles de calidad (básico, intermedio, premium) entregando un producto final listo para usar.

Descripción del problema u oportunidad

Definición del problema Central

Actualmente, la creación de páginas web profesionales exige altos conocimientos técnicos o una inversión económica inicial muy elevada, lo que provoca microempresarios y profesionales independientes, credibilidad y oportunidades de venta en el mercado.

Descripción del Impacto

El impacto de esta problemática se manifiesta en múltiples dimensiones operativas y financieras para el usuario objetivo. En términos de tiempo, los emprendedores desperdician semanas intentando aprender a utilizar constructores de sitios o gestores de contenido (CMS), desviando su atención del núcleo de su negocio. En términos de costos, las alternativas se reducen a contratar agencias de desarrollo cuyas tarifas de cientos o miles de dólares resultan privativas para un negocio naciente o asumir costosas suscripciones mensuales en plataformas que retienen la propiedad del código (Vendor Lock-in).

Adicionalmente, el intento de “hacerlo uno mismo” sin conocimientos previos genera un alto margen de errores y una mala experiencia de usuario (UX), páginas que cargan lento, enlaces rotos y diseños que no se adaptan a dispositivos móviles (falta de Responsive Design). El efecto final de este impacto es la pérdida de conversiones (leads), un cliente potencial que visita un sitio web deficiente percibe falta de profesionalismo, lo que traduce en ventas perdidas y un estancamiento del crecimiento comercial.

Análisis de Causas y efectos: Árbol del Problema

Para desglosar estructuralmente esta situación, se ha elaborado el siguiente Árbol de Problemas, donde el tronco representa el problema central, las raíces corresponden a las causas que lo originan, y las ramas ilustran los efectos o consecuencias que el usuario sufre en la actualidad.

(Tronco) Problema Central: Imposibilidad de microempresarios y profesionales independientes para obtener una landing page funcional, profesional y a un costo accesible de manera ágil.



Tabla 1: Árbol del Problema

(Raíces) Causas (¿Por qué ocurre?)

- 1. Barra técnica (Curva de aprendizaje):** Herramientas actuales (CMS y constructores visuales) exigen conocimientos subyacentes de diseño web, HTML/CSS, dominios y alojamiento de servidores.
- 2. Barrera financiera (Costos prohibitivos):** El desarrollo de software a medida y el diseño UI/UX personalizado tienen un alto costo de mercado por horas de trabajo humano.
- 3. Modelo de negocio restrictivo:** Las plataformas “gratuitas” o de bajo costo ocultan barreras de pago (suscripciones obligatorias mensuales) para poder publicar o conectar un dominio propio.
- 4. Falta de abstracción en herramientas de IA:** Los actuales generadores de código por IA (como ChatGPT) entregan código crudo que el usuario no técnico no sabe cómo compilar, enlazar ni desplegar.

(Ramas) Efectos (¿Que consecuencias trae?):

- 1. Dependencias tecnológicas riesgosas:** Los negocios operan exclusivamente a través de perfiles en redes sociales, quedando a merced de cambios de algoritmos y sin control real sobre su audiencia o sus datos.
- 2. Fuga de capital operativo:** Pago de suscripciones mensuales de plataformas web que el usuario finalmente no utiliza porque no logró terminar de configurar el sitio.
- 3. Deterioro de la imagen de marca:** Despliegue de sitios web “caseros” con errores de diseño visual, mala ortografía o estructuras confusas que generan desconfianza en el cliente final.
- 4. Estancamiento comercial:** Imposibilidad de lanzar campañas de marketing digital (Google Ads, Facebook Ads) de manera efectiva al no contar con una página de aterrizaje (Landing Page) optimizada para la conversión.

Conclusión: Enfoque de la solución

La solución tecnológica propuesta en este proyecto (Plataforma Web Inteligente para la Creación Automatizada de Landing Page) atacará directamente las causas raíces 1 y 4 (Barrera técnica y falta de abstracción) y la causa 2 (Barrera financiera). Al delegar toda la carga de diseño estructural, maquetación y programación a un motor inteligencia artificial en el backend (Spring Boot), el usuario ya no necesita aprender a usar un constructor visual, sólo debe llenar un formulario. Asimismo, al post procesar este código y entregarlo listo en archivos descargables bajo un modelo de pago único y escalonado (básico, intermedio, premium), se elimina la barrera económica y la dependencia de suscripciones, resolviendo el problema de raíz y mitigando por completo sus efectos negativos en el mercado.

Descripción del mercado objetivo

El cliente objetivo (B2B/B2C) está compuesto por microempresarios, freelancers, consultores y pymes emergentes. Son personas que necesitan digitalizarse urgentemente pero no tienen conocimientos de programación ni presupuesto para un equipo técnico

Necesidades:

- **Inmediatez:** Lanzar una Landing Page funcional en minutos.
- **Abstracción Técnica:** Obtener código limpio (HTML/CSS) sin escribir ni una línea, mediante un formulario guiado por IA.
- **Flexibilidad:** Pagar lo justo según el nivel de diseño que requieran en el momento (básico, intermedio, premium), sin amarrarse a suscripciones mensuales.

Dolores:

- **Altos Costos:** Pagar a agencias o desarrolladores está fuera de su presupuesto inicial.
- **Pérdida de tiempo:** Los CMS "fáciles" (como Wix o WordPress) en realidad tienen una curva de aprendizaje alta, desconfiguran el diseño y desenfocan al usuario de su negocio principal.

Escenario de uso:

- **Cuándo y dónde:** El usuario ingresa desde su computador (en casa u oficina) en momentos de urgencia comercial (ej. un psicólogo que decide organizar un taller online para este fin de semana y necesita urgente una página para poner el link de inscripción en su Instagram, o un emprendedor que quiere probar si su nueva idea de negocio tiene demanda real antes de gastar dinero). Ingresa a la plataforma, completa sus datos comerciales, elige el nivel de diseño que se ajusta a su bolsillo, y en menos de 5 minutos descarga su web lista para usar.)
- **Usuario 1 (Principal - El emprendedor/Freelancer):**

Características: Profesional de 25 a 50 años. Experto en su rubro (ej. repostería, consultoría legal) pero con nulos conocimientos en programación o diseño UI/UX. Su presupuesto es limitado y su tiempo es escaso, ya que él mismo gestiona todas las áreas de su negocio. Es quien toma la decisión directa de usar y pagar por la herramienta.

Necesidad Principal: Necesito tener una página web hoy mismo para que mi negocio se vea profesional y pueda recibir contactos, pero solo tengo un presupuesto de \$X y no tengo tiempo para aprender a usar un software complejo. Necesito que alguien (o algo) lo haga por mí rápido.

- **Usuario 2 (Secundario - El Gestor de Marketing/Agencia Pequeña):**

Características: Profesional del área de marketing, *Community Manager* o trabajador de una agencia pequeña. Conoce conceptos de diseño y web, pero no es programador.

Necesidad Principal: Necesito generar múltiples versiones de *landing pages* rápidamente para probar diferentes campañas (A/B testing) para mis distintos clientes, sin tener que saturar al equipo de desarrollo con peticiones menores. Necesito un generador de código confiable y de buena calidad para agilizar mi trabajo diario.

Objetivo general y objetivos específicos

Objetivo General

Desarrollar una plataforma web SaaS (Software as a Service) impulsada por inteligencia artificial que automatice la maquetación y codificación de páginas de aterrizaje (Lading Pages) escalonadas en calidad y precio, con el fin de permitir a microempresarios, profesionales independientes y pequeñas agencias establecer una presencia digital profesional de forma inmediata, accesible y sin requerir conocimientos previos de programación.

Objetivos Específicos

Para dar cumplimiento cabal al objetivo general y asegurar la construcción estructural del producto de software, se establece las siguientes metas concretas y medibles, divididas según las capas de la arquitectura del proyecto:

1. Modelar e implementar una base de datos relacional y una arquitectura de servidor multicapa (utilizando Spring Boot y java) que garantice el registro seguro de usuarios, la persistencia de los proyectos generados y la gestión del modelo de cobro simulado.
2. Diseñar e integrar un motor algorítmico de Prompt Engineering en el backend que se comunique con una API de inteligencia artificial externa, estructurando tres niveles de complejidad de peticiones (Básico, intermedio, premium) para abstraer al usuario de la redacción de instrucciones técnicas.
3. Construir una interfaz de usuario web interactiva y dinámica (empleando React y TailWind CSS) centrada en la usabilidad, que guíe al cliente a través de un formulario inteligente de captura de requerimientos comerciales en menos de 10 pasos.
4. Desarrollar e implementar un módulo de post procesamiento en el servidor que reciba el código crudo generado por la IA, lo limpie, valide su renderizado (HTML/CSS) y lo empaquete en archivos descargables listos para su publicación inmediata.

Coherencia y Alineación del Proyecto

La definición de estos objetivos garantiza el éxito del proyecto, ya que su cumplimiento secuencial resuelve directamente el problema central detectado. El objetivo 3 ataca la causa de la alta curva de aprendizaje (barrera técnica) al reemplazar los constructores visuales complejos por un formulario guiado. El objetivo 2 y 4 resuelven la falta de abstracción de las IAs actuales, ya que el sistema se encargará de estructurar, limpiar y validar el código, entregando un producto terminado. Finalmente, la integración de toda la arquitectura (objetivo 1) sostiene el modelo de negocio de pago escalonado, eliminando la barrera de los costos prohibitivos y democratizando el acceso a la tecnología web para el mercado objetivo.

Situación inicial del cliente (AS-IS)

Contexto de la situación Actual

En el escenario actual, cuando un micro empresario o profesional independiente (freelance) detecta la urgencia de contar con una página web para validar un negocio o captar clientes, se enfrenta a un ecosistema tecnológico que no está diseñado para su nivel de conocimientos ni para su presupuesto. Dado que la contratación de una agencia de desarrollo a medida queda descartada de inmediato por la barrera económica, el usuario se ve forzado a tomar la ruta del “Hágalo usted mismo” utilizando constructores de sitios visuales (CMS) o, más recientemente, intentando usar inteligencia artificial de texto crudo sin interfaz gráfica

Descripción del Proceso Actual

1. **El usuario detecta la necesidad e inicia la búsqueda:** El emprendedor requiere una página de aterrizaje urgente. Busca en internet “como crear una página web gratis” y es bombardeado por publicidad de gestores de contenido complejos o constructores visuales.
2. **Luego, el usuario se registra y se enfrenta a la sección (Sobrecarga cognitiva):** Ingresa a una plataforma tradicional, crea una cuenta y se topa con un catálogo de cientos de plantillas genéricas. Pierda horas valiosas intentando decidir cuál se adapta a su rubro, sufriendo “parálisis por análisis”.
3. **El usuario intenta personalizar el diseño (El cuello de botella técnico):** Una vez elegida la plantilla, comienza el proceso manual de diseño utilizando herramientas de “arrastrar y soltar”. El problema ocurre cuando el usuario intenta cambiar márgenes, reemplazar imágenes o ajustar columnas, el diseño responsivo (la vista para celulares) se desconfigura por completo, exigiendo conocimientos de estructuración visual que no posee.
4. **El usuario redacta el contenido manualmente:** Sin ser experto en marketing o redacción comercial, el usuario debe inventar los textos persuasivos de su página, enfrentándose al síndrome del lienzo blanco, lo que retrasa aún más el lanzamiento.
5. **El problema crítico ocurre al intentar publicar (La barrera de Pago):** Tras días de frustración y un diseño ensamblado a medias, el usuario intenta publicar la web. En este punto, la plataforma le informa que la versión “gratuita” insertará anuncios invasivos de otras marcas y utilizará un subdominio genérico poco profesional. Para obtener los archivos reales o conectar su propio dominio, el sistema lo obliga a pagar una costosa suscripción mensual perpetua.

Puntos de Dolor identificados en el Proceso

- **Desperdicio de tiempo productivo:** El proceso consume entre 3 a 7 días hábiles de un emprendedor que debería estar vendiendo, no intentando maquetar interfaces.
- **Falta de Abstracción:** El sistema actual obliga al usuario a tomar decisiones técnicas (tipográficas, espaciados, estructura HTML invisible) para las cuales no está capacitado.
- **Modelo de retención financiera:** El cliente se da cuenta al final del proceso de que nunca será dueño del código de su página, quedando atado a mensualidades abusivas para que su sitio no sea dado de baja.

Diagrama del Proceso Actual (Flujo AS-IS)

A continuación, se esquematiza el flujo operativo descrito, evidenciando como las entradas del usuario terminan en salidas deficientes debido a los obstáculos del proceso:

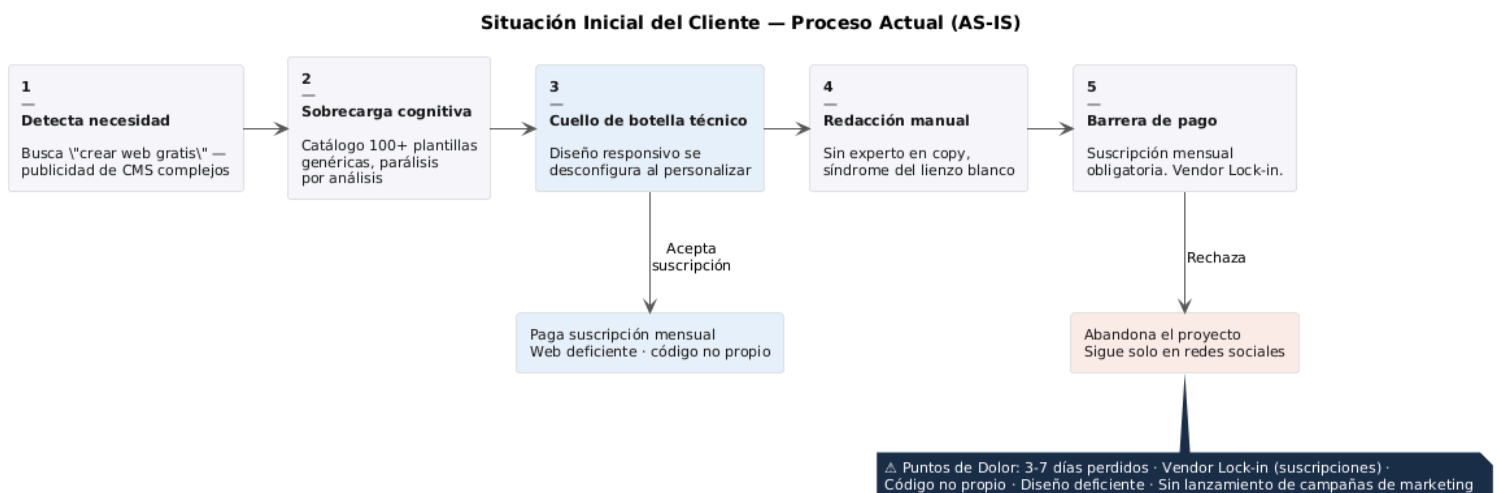


Tabla 2: Diagrama situación inicial del cliente

Planificación

Estructura del Trabajo

Para garantizar la viabilidad del proyecto dentro de los plazos establecidos. La ejecución se ha estructurado bajo la metodología híbrida previamente justificada. El equipo de tres integrantes se dividirá las responsabilidades técnicas específicas: Integrante Juan Carlos Agüero (Especialista Frontend/UI), Integrante Marcos Orellana (Especialista Backend/BD) e Integrante Isaac Gonzales (Ingeniero de Prompts/IA)

A continuación, se detalla la lista de las 10 actividades principales que llevará el proyecto desde su concepción hasta su entrega final, desglosadas por fases, duración y responsables.

Inicio y Planificación

Esta fase es predictiva y busca cimentar las bases arquitectónicas del software.

1. Levantamiento y requerimientos y definición de arquitectura. Se definen las reglas de negocio, los endpoints de la API y las restricciones técnicas del sistema.
2. Modelamiento de Base de Datos y Diseño UI/UX. Creación del diagrama entidad-Relación y diseño visual de las pantallas en herramientas como figma.
3. Configuración de entornos y despliegue de la API base. inicialización del proyecto en Spring Boot, conexión a la base de datos y creación del esqueleto multicapa (controller, Service, repositorio).
4. Desarrollo de la interfaz de usuario y formulario dinámico. Construcción del Frontend en React/Vite, aplicando Tailwind CSS para el diseño responsivo del formulario de captura de datos.
5. Diseño, calibración y pruebas de Prompts Escalonados. Redacción y validación estricta de las 3 plantillas de IA (Básico, Intermedio, Premium) para asegurar que devuelvan HTML/CSS consistente.
6. Integración de API externa y lógica de post-procesamiento. Conexión segura desde el servidor a la IA externa y programación del algoritmo que limpia y formatea el código crudo generado.
7. Conexión Frontend-Backend y simulación de pagos. Consumo de los endpoints desde React, flujo de selección de planes y simulación de la pasarela de cobro.
8. Pruebas Integrales (QA) y validación de código generado. Depuración exhaustiva de errores (bugs), pruebas de estrés en la generación de páginas y validación de seguridad.
9. Despliegue en la nube (Puesta en producción). Migración del sistema desde los entornos locales (localhost) hacia plataformas en la nube (PaaS/IaaS) para su acceso público.

10.Documentación técnica final y preparación de defensa. Cierre del informe académico, manuales de usuario y ensayos para la presentación ante la comisión evaluadora.

Plataforma Web Inteligente para la Creacion Automatizada de Landing Page

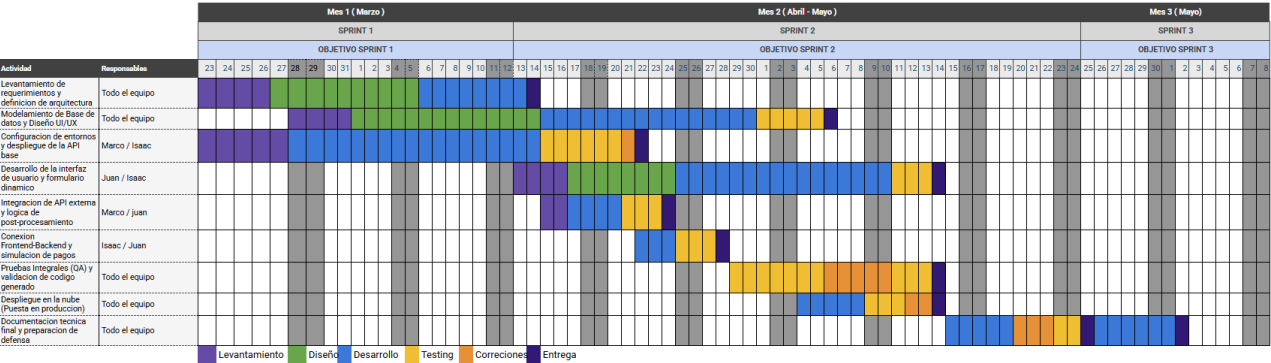


Tabla 3: Carta Gantt preliminar del proyecto

Servicios Cloud a utilizar (IaaS / PaaS / SaaS)

Justificación de la Arquitectura Cloud

El despliegue y puesta en producción de la plataforma web se realizará íntegramente utilizando servicios en la nube (cloud computing). Se ha descartado el uso de servidores físicos locales y soluciones IaaS (Infraestructura como servicio, como Amazon EC2 puro) debido a la sobrecarga administrativa que conllevan. En su lugar, la arquitectura se basará estratégicamente en modelos PaaS (Plataforma como Servicio) y SaaS (Software como servicio). Esta decisión técnica garantiza una alta disponibilidad, permite la escalabilidad automática ante picos de usuarios generando sitios web, facilita el despliegue continuo directamente desde los repositorios de código, y optimiza los costos operativos, aprovechando las capas gratuitas y el pago por uso durante la fase de validación del proyecto.



Fronted (Capa de Presentación - PaaS)

- Servicio a utilizar: Vercel o Netlify (Plataforma como Servicio).
- Justificación: La interfaz de usuario, desarrollada como una Single Page Application (SPA) con React y Vite, requiere un entorno optimizado para la entrega de archivos estáticos. Vercel actúa como un PaaS que compila automáticamente el código fuente desde GitHub y lo distribuye a través de una Red de entrega de Contenidos global. Esto asegura tiempos de carga milisegundos para el cliente en cualquier parte del mundo, ofreciendo además certificados SSL automáticos para garantizar conexiones seguras (HTTPS).

Backend (Capa de Lógica y API - PaaS)

- Servicio a utilizar: Render o Railway (Plataforma como Servicio).
- Justificación: Para mantener la integridad de las cuentas de usuario, el registro de los planes seleccionados y el historial de transacciones, se requiere un motor relacional estricto (PostgreSQL). Utilizar un servicio administrado en la nube elimina la necesidad de gestionar copias de seguridad manuales, parches de seguridad o configuraciones de red. Garantiza que la persistencia de los datos sea robusta, tolerante a fallos y accesible de forma segura y exclusiva por el Backend.

Motor Inteligencia Artificial (SaaS)

- Servicio a utilizar: API de OpenAI, Anthropic o equivalente (Software como Servicio).
- Justificación: La plataforma no entrenará un modelo de lenguaje propio debido a las limitaciones de infraestructura (IaaS). En su lugar, consumirá un modelo de IA de frontera bajo el formato SaaS mediante peticiones HTTP. El servidor enviará los prompts escalonados (Básico, Intermedio, Premium) y la API externa retornará el código HTML/CSS procesado. Este modelo permite integrar inteligencia artificial de nivel mundial pagando fracciones de centavo por cada generación, manteniendo el costo operativo estrictamente ligado al uso real del sistema.

Conceptualización

Diagrama de Arquitectura de la Solución (TO-BE)



Tabla 4: Diagrama de arquitectura

Descripción de la solución (Modelo TO-BE)

En contraposición al modelo actual (AS-IS), donde el usuario sufre una sobrecarga cognitiva intentando diseñar interfaces manualmente y lidiar con configuraciones de servidores, el modelo propuesto (TO-BE) abstrae y automatiza integralmente la complejidad técnica. La plataforma funcionará como un intermediario inteligente (SaaS) que transforma requerimientos en lenguaje natural y datos estructurados en código fuente renderizable.

El nuevo flujo será el siguiente: el usuario entra a la plataforma web y, en vez de encontrarse con una página en blanco o muchas plantillas, verá un Formulario Inteligente que lo guiará paso a paso para conocer su negocio, como el nombre, rubro, colores, estilo de comunicación y servicios principales; luego de completarlo, podrá elegir el tipo de página que quiere (Plan Básico, Intermedio o Premium), y en ese momento el sistema se encargará de tomar la información, enviarla a una inteligencia artificial y generar la página automáticamente; en pocos minutos, el usuario verá una vista previa de su sitio web y, finalmente, después de confirmar el pago, podrá descargar el código de su página (HTML, CSS y JavaScript), listo para subirlo a cualquier hosting, quedando como dueño total de su sitio sin pagos mensuales.

Funcionalidades principales:

1. **Formulario Dinámico de Captura:** Interfaz fluida desarrollada en React que captura los requerimientos del cliente mediante validaciones en tiempo real, asegurando que la IA reciba el contexto exacto para generar un diseño coherente.
2. **Motor de Prompt Engineering Escalonado:** Núcleo lógico del servidor que clasifica la solicitud del usuario y selecciona automáticamente una de las tres plantillas de instrucciones restrictivas (Básico para HTML simple; Intermedio para diseño responsivo con Tailwind; Premium para animaciones y SEO básico), inyectando los datos del cliente antes de enviarlos a la IA externa.
3. **Módulo de Post-procesamiento y Sanitización:** Algoritmo encargado de recibir la respuesta cruda de la IA, limpiar cualquier texto conversacional ("Aquí tienes tu código..."), extraer estrictamente las etiquetas de maquetación y validar que la estructura del DOM no contenga errores críticos antes de mostrarla.
4. **Visor de Previsualización en Tiempo Real (Live Preview):** Un componente en la interfaz de usuario que renderiza de manera segura el código generado dentro de un entorno aislado (Iframe), permitiendo al cliente interactuar con el diseño antes de la compra.
5. **Simulador de Pasarela de Pago y Descarga:** Funcionalidad que procesa la selección del plan (facturación simulada) y habilita la creación dinámica de un archivo comprimido (.zip) con los activos digitales finales listos para descarga.
6. **Panel de Gestión de Proyectos (Historial):** Un *dashboard* privado donde los usuarios (especialmente útil para agencias o perfiles B2B) pueden visualizar el historial de las Landing Pages que han generado, acceder a descargas previas y revisar los recibos de sus transacciones.

Entregables de la Etapa de Construcción

Al finalizar el ciclo de desarrollo del proyecto, el equipo entregará los siguientes artefactos tangibles:

- **Código Fuente del Frontend:** Repositorio estructurado de la Single Page Application (React/Vite)
- **Código Fuente del Backend:** Repositorio de la API RESTful (Java con Spring Boot), incluyendo la lógica de integración con la IA.
- **Script y modelo de la Base de Datos:** Archivos DDL y DML para la generación de la estructura relacional (PostgreSQL).
- **Documentación técnica:** Manuales de despliegue, diccionarios de datos y documentación de los endpoints (Swagger/OpenAPI).

Consideraciones de Calidad y Seguridad

Al tratar con la generación automatizada de código mediante inteligencia artificial, la seguridad es un pilar no negociable.

- **Autenticación y Autorización:** El acceso a la plataforma y al panel de proyectos estará protegido mediante JSON Web Tokens (JWT), garantizando que las sesiones sean sin estado (stateless) y escalables en la nube.
- **Validación y Prevención de XSS (Cross-Site Scripting):** Dado que la IA genera código que luego será renderizado en el navegador del usuario, el módulo de post-procesamiento implementará librerías de sanitización estrictas en el backend para eliminar cualquier intento de inyección de scripts maliciosos o etiquetas no permitidas.
- **Respaldo y Transaccionalidad:** Las propiedades ACID de la base de datos relacional asegurarán que el registro de los pagos y la asociación de los proyectos a los usuarios no sufran inconsistencias. Adicionalmente, se configurarán respaldos automatizados (backups) diarios en la plataforma de alojamiento en la nube (DBaaS) para asegurar la persistencia ante desastres.

Alcance, supuestos y restricciones

Alcance del Proyecto

El presente proyecto contempla el desarrollo integral (Frontend, Backend y Base de Datos) de una plataforma web SaaS (Software as a Service) para la generación automatizada de páginas de aterrizaje estáticas mediante inteligencia artificial. El alcance funcional se delimita a las siguientes capacidades operativas:

- Registro, autenticación y gestión de perfiles de usuario.
- Implementación de un formulario dinámico inteligente para la captura de requerimientos comerciales y preferencias visuales.
- Integración mediante API RESTful con un modelo de inteligencia artificial externo (ej. OpenAI o Anthropic) gestionada a través de un prompt de "Prompt Engineering" estructurado en tres niveles (Básico, intermedio y premium).
- Procesamiento, limpieza y validación en el servidor del código HTML, CSS Y JS generado por la IA.

- Renderizado de una pre visualización interactiva (Live Preview) en un entorno web aislado y seguro.
- Simulación del flujo de facturación según el plan seleccionado.
- Generación y entrega de los archivos finales empaquetados (formato .zip) listos para su descarga inmediata.

Fuera de Alcance (Que no incluye el sistema)

Para mantener la viabilidad del proyecto dentro de los plazos y recursos disponibles, se excluyen explícitamente las siguientes funcionalidades:

- **Alojamiento web (Hosting) de los sitios generados:** La plataforma entregara los archivos fuentes (HTML/CSS/JS), pero no proveerá el servicio de alojamiento público ni la compra/configuración de dominios personalizados (.com, .cl) para el cliente final.
- **Gestor de Contenidos:** No se desarrollará un panel de administración para que el usuario edite la página web después de haberla descargado. Se trata de páginas estéticas de un solo uso o generación.
- **Pasarelas de pagos reales:** La plataforma no integrará transacciones bancarias reales (Webpay, Stripe, paypal en producción). El flujo de cobro por los planes será netamente con fines de demostración académica.
- **Desarrollo de sistemas web complejos:** La IA generará exclusivamente Landing Pages estáticas informativas. No se contempla la generación de tiendas online con carritos de compra, bases de datos propias para el cliente o sistema de inicio de sesión dentro de la página generada.

Entregables del Proyecto

Al término del ciclo de desarrollo, el equipo proveerá los siguientes componentes:

1. **Producto de Software (MVP):** Plataforma web funcional desplegada en un entorno de producción en la nube (PaaS/SaaS).
2. **Código Fuente:** Repositorios de control de versiones (Git), estructurados para Frontend y Backend.
3. **Documentación Técnica:** Informe final del proyecto de título, diagramas de arquitectura multicapa, modelo relacional de la base de datos (Entidad-Relación) y documentación técnica de los endpoints de la API (Swagger).

Presupuestos

Para la planificación y ejecución de este proyecto, se asume como cierto que:

- **Disponibilidad Tecnológica:** Las APIs de inteligencia artificial de terceros mantendrá su estabilidad, disponibilidad y mantendrá vigentes sus niveles gratuitos o de bajo costo para pruebas de desarrollo.

- **Adopción del Usuario:** Se asume que el mercado objetivo (microempresas y freelancers) posee un nivel de alfabetización digital básico, suficiente para interactuar con un formulario web, descargar un archivo comprimido (.zip) y seguir instrucciones simples para subirlo a un proveedor de hosting económico.

Restricciones

- **Restricciones de Recursos Humanos:** El equipo de desarrollador consta exclusivamente de 3 estudiantes de último año, quienes deben compatibilizar la carga horaria de la construcción de este software con sus respectivas prácticas profesionales en curso y otras responsabilidades académicas.
- **Restricción Presupuestaria:** El proyecto cuenta con un presupuesto equivalente a cero. Por ende, la infraestructura técnica dependerá del uso eficiente de las capas gratuitas (free tiers) de los servicios en la nube (vercel, render, postgresQL) y del manejo cuidadoso de los créditos limitados de la API de IA.
- **Restricción Tecnológica (Alucinaciones de la IA):** Al depender de Modelos Fundacionales Externos, existe la restricción de imprevisibilidad en las respuestas. Esto obliga al equipo a invertir un tiempo considerable en imponer validaciones estrictas en la capa de negocio (Backend) para mitigar errores de sintaxis en el código generado.

Metodología de desarrollo y gestión del proyecto

1. Metodología Elegida: Híbrida (Cascada + Scrum Adaptado)

El equipo de desarrollo de WebLandingSuite adoptó una **metodología híbrida** que combina el modelo **Cascada (Waterfall)** en la fase inicial del proyecto con un enfoque **Scrum adaptado** durante la fase de construcción. Esta decisión responde directamente a la naturaleza del proyecto: una etapa de análisis y diseño que requería definiciones arquitectónicas estables antes de escribir la primera línea de código, seguida de un ciclo de desarrollo iterativo e incremental que permite entregar valor de forma continua.

- **Justificación de la combinación:**

La fase predictiva (Cascada) fue aplicada durante el levantamiento de requerimientos, el modelamiento de la base de datos, la definición del stack tecnológico y el diseño de la

arquitectura multicapa (C4). Estas actividades exigen decisiones firmes y estables: un cambio tardío en el modelo de datos o en la arquitectura de microservicios implicaría un costo técnico muy elevado para un equipo de tres personas con restricciones de tiempo. Una vez establecidas estas bases, el equipo transitó a un modelo iterativo (Scrum adaptado) para gestionar el desarrollo de funcionalidades mediante ciclos controlados, visibilidad del avance en tiempo real y colaboración constante entre los tres roles técnicos.

2. Fases del Proyecto y su metodología Aplicada

Fase 1 — Inicio y Planificación (Cascada): Comprende las actividades de análisis de requerimientos, diseño de la arquitectura de software, modelamiento del diagrama entidad-relación, definición del stack tecnológico y configuración de los entornos de desarrollo. Esta fase produce entregables estables que sirven como contrato técnico interno para el equipo.

Fase 2 — Construcción e Iteración (Scrum Adaptado): Una vez validada la arquitectura base, el desarrollo de funcionalidades se organizó bajo el marco Scrum adaptado a las condiciones del equipo. Las tareas se gestionan en el tablero Jira del proyecto (WebLandingSuite), organizado en cuatro estados:

- **Por Hacer** = Backlog de tareas priorizadas pendientes de asignación.
- **En Curso** = Tareas activamente en desarrollo por un integrante designado.
- **En Revisión** = Tareas completadas que esperan validación o Code Review por otro integrante antes de cerrarse.
- **Finalizado** = Tareas entregadas y validadas por el equipo.

3. Organización del equipo y Roles

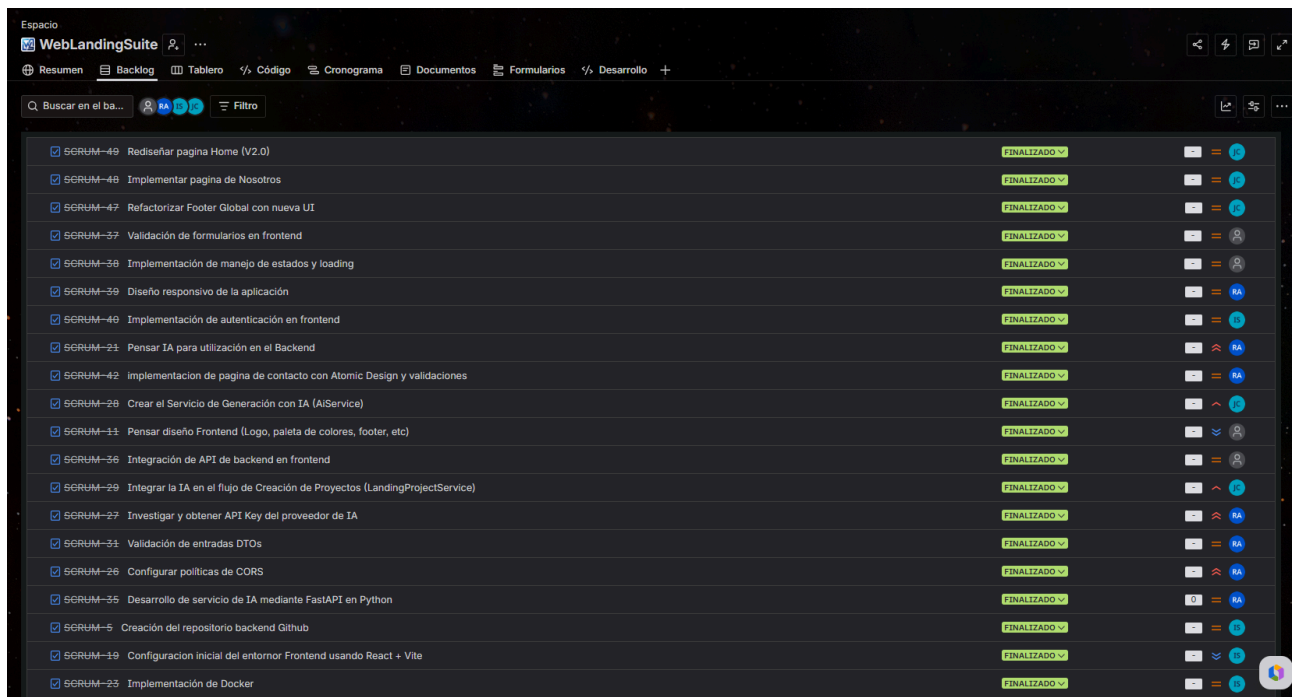
El equipo está conformado por tres integrantes con roles técnicos especializados, aunque con colaboración transversal entre módulos:

Integrante	Rol Principal	Responsabilidades
Juan Carlos Aguero	Especialista Frontend / UI	Componentes React, diseño con Atomic Design, integración de API hacia el Backend
Marcos Orellana	Especialista Backend / BD	API REST con Spring Boot, modelo de datos PostgreSQL, seguridad JWT

Isaac Gonzales	Ingeniero de Prompts / IA	Microservicio FastAPI en Python, diseño de prompts escalonados, integración con OpenRouter
----------------	---------------------------	--

4. Gestión de Tareas en Jira

El proyecto WebLandingSuite utiliza **Jira** como herramienta central de gestión del trabajo. El espacio de proyecto cuenta con las vistas: Resumen, Backlog, Tablero, Código, Cronograma, Documentos y Desarrollo, lo que permite trazabilidad completa entre las tareas, el código fuente y la planificación.



Este Sprint agrupó un total de **44 actividades**, abarcando desde la configuración inicial de entornos hasta la integración del motor de IA. Al momento de la presente entrega, **38 tareas se encuentran en estado Finalizado** y **6 en curso**, evidenciando un alto porcentaje de avance del sprint activo.

Entre las tareas finalizadas más relevantes se destacan:

- Rediseño de página Home (V2.0)
- Implementación de autenticación en Frontend
- Creación del servicio de generación con IA (AiService)
- Integración de la IA en el flujo de creación de proyectos (LandingProjectService)
- Desarrollo del microservicio de IA mediante FastAPI en Python
- Configuración de políticas de CORS
- Validación de entradas mediante DTOs

- Investigación y obtención de API Key del proveedor de IA
- Integración de API de Backend en Frontend
- Implementación de Docker

Las tareas actualmente **en curso** incluyen la optimización y definición de prompts, el diseño responsivo de la aplicación, la mejora de la carta Gantt y el formulario de Landing para el Frontend.

5. Flujo de Trabajo y Control de Calidad

El flujo de trabajo del equipo sigue el estándar Git Flow con ramas protegidas, complementando por la gestión en Jira:

- Se crea una tarea en Jira con descripción, responsable y prioridad.
- El desarrollador crea una rama `feature/nombre-tarea` en el repositorio correspondiente.
- Al completar el desarrollo, se abre un Pull Request (PR) hacia la rama `dev`.
- Otro integrante del equipo realiza el Code Review antes de aprobar el merge.
- La tarea se mueve en Jira al estado `En Revisión` durante este proceso y a `Finalizado` tras la aprobación.
- Los merges hacia `main` representan versiones estables listas para producción.

Este flujo garantiza que ninguna funcionalidad llega a producción sin revisión de un segundo par de ojos, minimizando la inyección de errores en el entorno desplegado en Render y Vercel.

Añadir epic / SCRUM-35

Finalizado

Agentes

Listo

Mejorar el tipo de actividad Tarea

Descripción

Desarrollar una API en Python utilizando FastAPI que permita la comunicación con el backend implementado en Spring Boot.

Subtareas

Añadir subtarea

Actividades vinculadas

Añadir actividad vinculada

Actividad

Todo

Comentarios

Historial

Registro de actividad

RA

Añadir un comentario...

Actualización del estado... Gracias... Estoy de acuerdo...

Consejo de expertos: pulsa M para comentar

Detalles

Persona asignada

ISAAC AMARU GONZALE...

Asignarme a mí

Prioridad

Medium

Principal

Ninguno

Fecha de vencimiento

Ninguno

Etiquetas

Ninguno

Team

Ninguno

Start date

Ninguno

Sprint

SCRUM Sprint 0

Story point estimate

0

Informador

RA RENE MARCOS ANTONI...

Tabla 5: Descripción de tarea en jira

Release: Integración Motor IA (FastAPI) y Arquitectura Core SaaS #25

Merged

Amaaruu merged 53 commits into main from dev 3 weeks ago

Conversation 0 Commits 53 Checks 0 Files changed 50

+1,837 -66

Amaaruu commented 3 weeks ago

Resumen

Este merge consolida la arquitectura central del SaaS WebLandingSuite. Integra oficialmente la comunicación entre el backend transaccional (Spring Boot) y el microservicio del motor de IA (Python/FastAPI). Además, refactoriza el flujo de la base de datos para garantizar la escalabilidad técnica de los planes (Básico, Intermedio, Premium) y previene cuellos de botella en producción.

Cambios Principales

1. Integración de IA & Progressive Disclosure

Se implementó AIService utilizando RestClient para orquestar peticiones HTTP hacia el microservicio en Python. Alineación de Contratos (DTOs): Se ajustaron las validaciones en LandingProjectRequestDto para soportar Progressive Disclosure. Los campos projectidea y calltoaction ahora son obligatorios, mientras que los estéticos son opcionales para no bloquear el Plan Básico.

2. Estructura de Datos (PostgreSQL JSONB)

De HTML a JSON: Se eliminó la dependencia de código HTML crudo. El sistema ahora guarda toda la estructura generada por OpenRouter (Claude/GPT) en un objeto JSON nativo dentro del campo aimetadata, permitiendo al frontend en React renderizar componentes de forma limpia y segura. Escalabilidad Visual: Se reemplazó el antiguo campo de colores por designPreferences (JSONB) en el modelo LandingProject, preparándolo para recibir configuraciones Premium (animaciones, estilo visual, integraciones).

3. Rendimiento y Resiliencia

Resolución de Cuello de Botella: Se reestructuró LandingProjectService eliminando la anotación @transaccional global. Ahora, las llamadas a la IA (que pueden demorar entre 10-30s) se realizan de forma independiente, liberando el pool de conexiones de PostgreSQL (HikariCP) y evitando la caída del servidor bajo alta concurrencia.

4. Correcciones Internas (Fixes)

Se restauró la consulta interna a la Entidad pura (getProjectEntityById) en el servicio para solucionar el error de compilación del Logcontroller y garantizar el registro correcto de auditorías (IPs de clientes y eventos de seguridad). Limpieza del LandingProjectController: eliminando endpoints obsoletos de previsualización de texto.

Reviews

No reviews

Assignees

No one—assign yourself

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

Notifications

Subscribe

You're not receiving notifications from this thread.

4 participants

Lock conversation

Tabla 6: Github / Pull Request de la tarea en jira

6. Definition of Done (Definición de Terminado)

Para que una tarea sea considerada **Finalizada** dentro del tablero Jira de WebLandingSuite, debe cumplir con todos los siguientes criterios:

Criterio	Descripción
Código funcional	La funcionalidad implementada opera correctamente en el entorno local sin errores de consola o excepción no controlada.
Validaciones activas	Los inputs de usuario están validados en el Backend (DTOs con <code>jakarta.validation</code>) y en el Frontend (estado controlado de React).
Sin conflictos de merge	La rama <code>feature</code> fue mergeada exitosamente hacia <code>dev</code> sin conflictos pendientes.
Code Review aprobado	Al menos un integrante del equipo distinto al autor revisó y aprobó el Pull Request en GitHub.
Tarea cerrada en Jira	El ticket correspondiente fue movido al estado Finalizado en el tablero Jira del proyecto.
Sin regresiones visibles	Las funcionalidades previamente implementadas continúan operando correctamente tras el merge.
Seguridad básica aplicada	Los endpoints protegidos validan el token JWT. El código no expone credenciales ni secretos en el repositorio.

7. Lista de Hitos del Proyecto

Los hitos representan los puntos de control críticos que marcan el avance formal del proyecto, desde su concepción hasta la entrega final. Dado que el equipo opera bajo una metodología híbrida con un Sprint único, los hitos se distribuyen en tres fases: Planificación, Construcción y Cierre.

N°	Hito	Descripción	Fecha	Estado
H-01	Kick-off del Proyecto	Definición formal del equipo, roles, stack tecnológico y arquitectura base del sistema.	14/Mar/2026	Completado
H-02	Arquitectura y Diseño	Entrega del modelo entidad-relación, diagrama C4,	05/Mayo/2026	Completado

	Aprobados	historias de usuario y casos de uso validados por el equipo.		
H-03	Entornos Configurados	Repositorios GitHub creados, entornos locales operativos, base de datos PostgreSQL en Neon inicializada y servicios en Render desplegados.	20/Abril/2026	Completado
H-04	Backend Core Operativo	API REST con Spring Boot funcional: autenticación JWT, gestión de usuarios, modelo de datos persistido y endpoints documentados en Swagger.	22/Abril/2026	Completado
H-05	Microservicio IA Integrado	FastAPI en Python desplegado en Render, conectado con OpenRouter, recibiendo prompts estructurados y retornando HTML/CSS válido.	24/Abril/2026	Completado
H-06	Frontend Funcional	SPA en React con flujo completo: registro, login, formulario de captura, visualización de planes y previsualización de Landing Page generada.	14/Mayo/2026	Completado
H-07	Integración Full-Stack Validada	Frontend consumiendo correctamente todos los endpoints del Backend. Flujo end-to-end: formulario → IA → previsualización → descarga funcional.	20/Mayo/2026	Completado
H-08	MVP en Producción	Plataforma WebLandingSuite desplegada públicamente en Vercel (Frontend) y Render (Backend + IA), con monitoreo activo en UptimeRobot.	14/Mayo/2026	Completado
H-09	Entrega Evaluación Parcial (EA2)	Informe técnico completo entregado con toda la documentación, diagramas, evidencias de desarrollo y estado de avance.	25/Mayo/2026	Completado

Documentos y diagramas de diseño de la solución

Para dar forma al desarrollo de WebLandingSuite y asegurar que los requerimientos técnicos satisfagan las necesidades del modelo comercial, el diseño de la solución se ha estructurado en base a requerimientos ágiles (Historias de Usuarios) y su posterior abstracción técnica (Casos de uso). Esta documentación define la interacción entre los distintos perfiles de usuarios y las capas de nuestra arquitectura (Frontend en React y APIs en Spring Boot).

1. Historias de Usuario (Backlog Ágil)

Módulo 1: Portal de Autenticación

- **HU01:** Como visitante, quiero navegar por la página principal de WebLandingSuite para conocer sus funcionalidades, planes disponibles y decidir si me registro en la plataforma.
- **HU02:** Como visitante, quiero registrarme con mi correo electrónico y contraseña para crear una cuenta en la plataforma y acceder a las funcionalidades de generación de Landing Pages.
- **HU03:** Como usuario, quiero iniciar sesión en el portal y recibir un token seguro (JWT) para acceder a mi cuenta.
- **HU04:** Como usuario autenticado, quiero descargar mi Landing Page generada en formato .zip o .html directamente desde la previsualización, para tener los archivos listos para publicar en cualquier hosting sin depender de la plataforma.

Módulo 2: Gestión de Planes y Pagos

- **HU05:** Como administrador, quiero gestionar (crear/editar/eliminar) los planes de diseño para actualizar la oferta.
- **HU06:** Como usuario, quiero ver los planes disponibles en el Frontend para elegir el que necesito.
- **HU07:** Como usuario, quiero registrar mi transacción de pago para activar la generación de mi proyecto.
- **HU08:** Como usuario, quiero consultar mi historial de pagos para llevar un control de mis gastos.

Módulo 3: Motor de IA y Creación

- **HU09:** Como usuario, quiero usar un formulario interactivo para ingresar el nombre, idea y colores de mi proyecto.
- **HU10:** Como sistema (Backend), quiero enviar los datos del proyecto FastAPI para que la IA procese y genere la web.
- **HU11:** Como usuario, quiero visualizar en el Frontend el resultado final de mi Landing Page generada.

Módulo 4: Gestión de Proyectos (Dashboard)

- **HU12:** Como usuario, quiero acceder a mi Dashboard para ver una lista con todos mis proyectos creados.
- **HU13:** Como usuario, quiero eliminar un proyecto antiguo para mantener mi espacio de trabajo organizado.
- **HU14:** Como administrador, quiero tener la capacidad de cambiar manualmente el estado de un proyecto (ej de "Procesando" a "Fallado") por si ocurre un error.

Módulo 5: Auditoría y Trazabilidad (Logs)

- **HU15:** Como sistema, quiero registrar automáticamente eventos críticos (login, creación de proyecto) junto con la IP del usuario.
- **HU16:** Como administrador, quiero consultar el historial completo de eventos (Logs) para detectar actividades sospechosas o errores.

Modulo 6: Gestion de Usuario

- **HU17:** Como administrador, quiero visualizar el listado completo de usuarios registrados en la plataforma.

2. Diagrama de Casos de Uso y Actores

El siguiente diagrama modela las interacciones descritas en las HU frente a la arquitectura del sistema, delineando claramente las fronteras de las distintas APIs (Seguridad, Pagos, Proyectos y Auditoria)

Se definen cuatro actores principales:

1. **Visitante:** Interactúa únicamente en la capa pública del Frontend para explorar el producto, ver precios y registrarse.
2. **Usuario (Cliente):** Actor autenticado que interactúa de manera transversal con el sistema para generar proyectos (Landing Pages), gestionar pagos y visualizar su dashboard.
3. **Administrador:** Posee privilegios elevados para soporte manual, configuración de precios, gestión de usuarios y consulta de logs del sistema.

4. **Sistema Externo (Microservicio IA):** Actor sistema encargado de recibir el Payload JSON y ejecutar la generación algorítmica de la página web.
5. **Servidor de Correos:** Actor automatizado que gestiona la comunicación transaccional asíncrona del sistema, enviando notificaciones y alertas mediante SMTP.

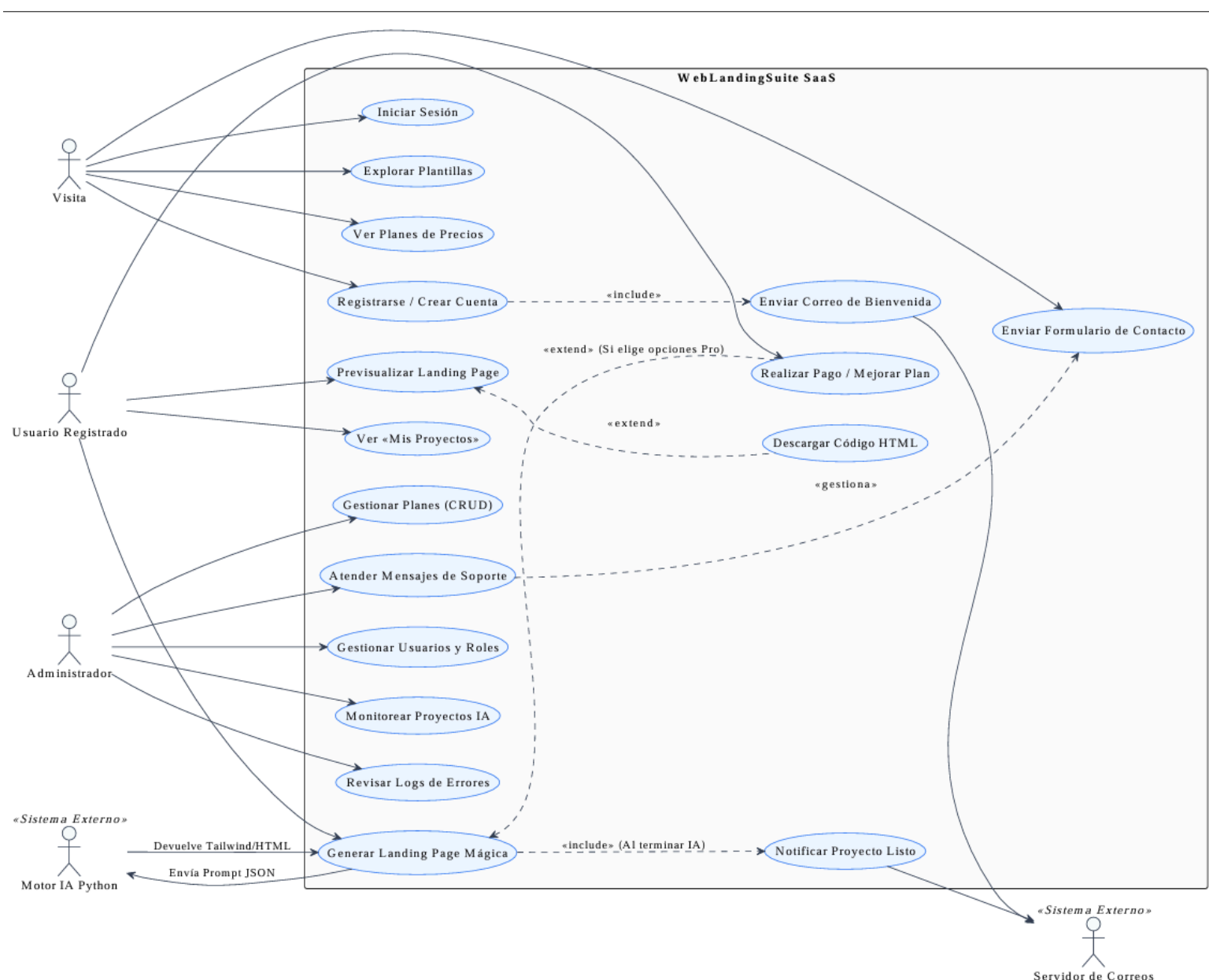


Tabla 7: Diagrama de casos de uso

Para garantizar la integridad y flujo de la plataforma, el diseño arquitectónico establece que acciones críticas (como solicitar una Landing Page o consultar pagos) dependen estrictamente de la validación del Token JWT y del registro en Auditoría mediante la

relación <<include>>. Asimismo, se definen flujos opcionales o condicionales mediante la relación <<extend>>, como la redirección a la pasarela de pagos al solicitar características premium o la descarga final del código HTML generado.

3. Modelo de Datos (Diagrama Entidad-Relación)

- **Propósito:** Garantizar la persistencia estructurada, segura y escalable de la información de los clientes, sus transacciones financieras y los activos digitales generados por el motor de Inteligencia Artificial.
- **Que representa:** El esquema relacional de la base de datos que será alojada en PostgreSQL (mediante Neon), detallado las cinco entidades principales del sistema (user, design_plan, transaction, landing_project y log), sus atributos, tipos de datos y cardinalidad.
- **Relación con el problema y objetivos:** Este modelo da cumplimiento directo al objetivo específico 1 del proyecto. Al centralizar la información, se resuelve el problema de la pérdida de progreso que sufren los usuarios en plataformas tradicionales. Además, soporta el modelo de negocio escalonado al vincular estrictamente los proyectos generados con transacciones válidas y planes específicos.

Explicación del Modelo

El diseño de la base de datos se basa en el núcleo transaccional. La tabla central es transaction, la cual actúa como puente entre el cliente (user) y el servicio adquirido (design_plan). Una vez que una transacción es validada, el sistema habilita la persistencia en la tabla landing_project, la cual almacena todos los requerimientos de diseño enviados por el usuario.

Destacan dos decisiones arquitectónicas clave para este modelo:

1. **Flexibilidad para la IA:** En la tabla landing_project, los atributos color_palette y ai_metadata utilizan el tipo de datos jsonb. Esto permite almacenar estructuras de datos dinámicas y complejas devueltas por el microservicio en Python (FastAPI), sin necesidad de alterar el esquema relacional si los parámetros de la IA cambian en el futuro.
2. **Trazabilidad y Seguridad (Auditoría):** La tabla log se relaciona tanto con el usuario como con el proyecto, registrando eventos críticos (event_at). Esto protege la plataforma contra abusos y facilita el soporte técnico, asegurando la trazabilidad de cada petición al motor de generación web.

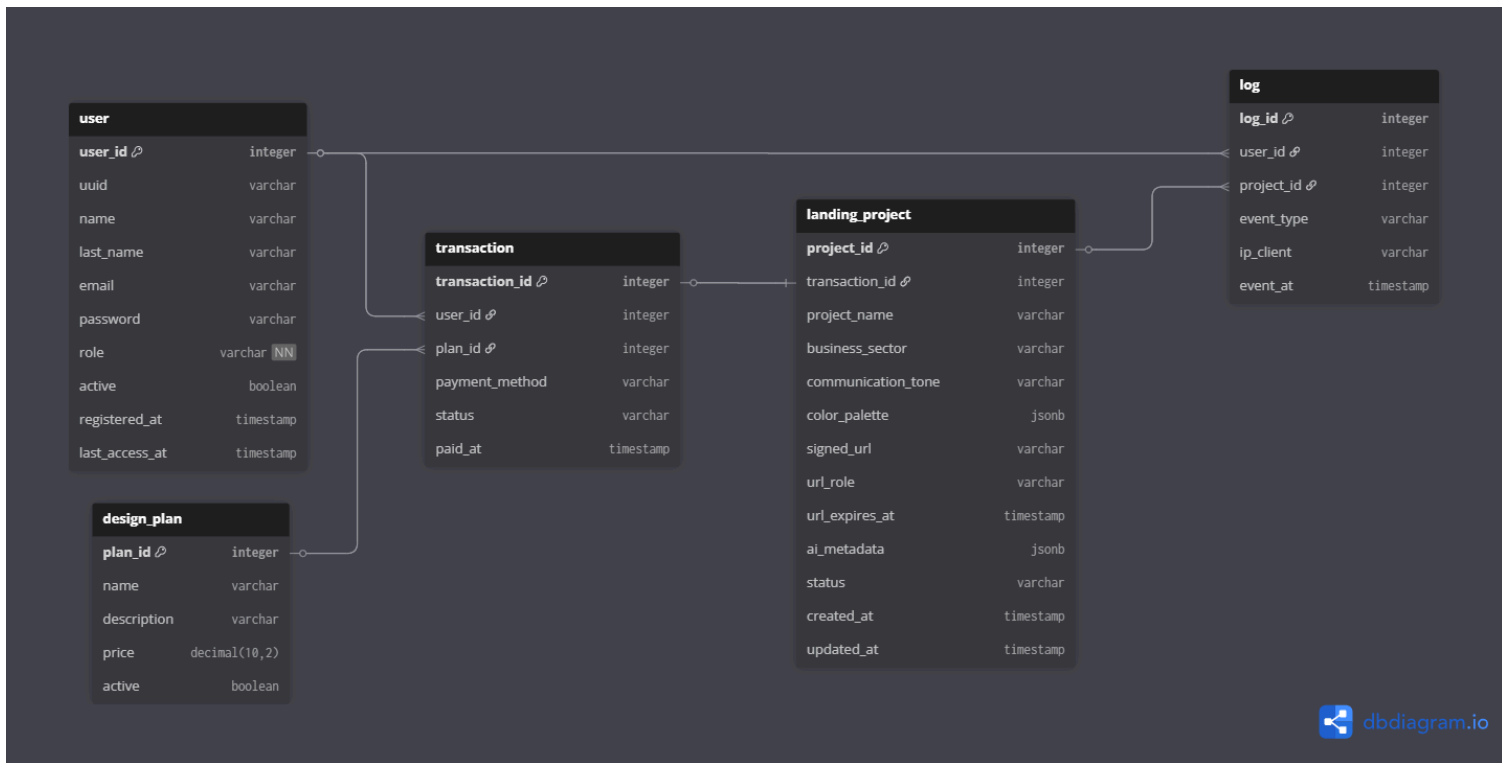


Tabla 8: Diagrama de entidad relación

Arquitectura de software y patrones de diseño

1. Descripción de la Arquitectura General del Sistema

El proyecto “WebLandingSuite” implementa una arquitectura Cliente-Servidor de tipo SPA (Single Page Application) separada físicamente, donde el cliente (Frontend) y el servidor (Backend) se comunican a través de una API RESTful. Además, el sistema adopta un enfoque orientado a Microservicios en la capa de procesamiento avanzado, delegando tareas específicas de alta carga computacional (como la generación de contenido y estructuras web) a un servidor independiente de inteligencia artificial.

La arquitectura se divide en tres bloques principales:

1. **Frontend (Capa de presentación):** Desarrollado con [React.js](https://react.dev/) 19 y Vite. Actúa como una SPA que gestiona el enrutamiento del lado del cliente (vía React Router) y la gestión del estado de la interfaz sin requerir recargas de página.
2. **Backend Central (Capa de Negocio y Orquestación):** Construido sobre Java 21 y Spring Boot 3.2.5. Actúa como el núcleo del sistema, gestionando la

autenticación, la lógica transaccional, el acceso a la base de datos relacional y la orquestación de llamadas hacia servicios externos.

3. **Servicio de IA (Microservicio Externo):** Desarrollado en Python (FastAPI). Es un componente desacoplado que recibe las solicitudes estructuradas del Backend Central y se encarga exclusivamente de procesar los prompts.

2. Identificación y Explicación de los Patrones de Diseño

El sistema utiliza patrones de diseño y arquitectónicos estandarizados en la industria para garantizar escalabilidad, mantenibilidad y separación de responsabilidades.

2.1 Patrones en el Frontend (React)

- **Atomic Design (Patrón Arquitectónico de UI):** El frontend está rigurosamente estructurado bajo este paradigma metodológico.
- **Átomos (/atoms):** Componentes puro sin estado ni lógica de negocio (ej: Button.jsx, Input.jsx).
- **Organismos (/organisms):** Estructura complejas que manejan estado y agrupan moléculas (ej: ContactSection.jsx, Navbar.jsx)
- **Páginas (/pages):** Vistas ruteables que orquestan los organismos para mostrar el contenido final al usuario.
- **Componentes Controlados (Controlled Components):** Utilizando en los formularios de la aplicación (ej: ContactSection.jsx), donde el estado de React (useState) es la única fuente de la verdad para los valores de los inputs, permitiendo validaciones en tiempo real.

2.2 Patrones en el Backend (Spring Boot)

- **Arquitectura en Capas:** El Backend sigue estrictamente el modelo de capas concéntricas, promoviendo el principio de Responsabilidades únicas:
- **Capa de Controladores (@RestController):** Expone los endpoints de la API REST (ej: LandingProjectController.java) y delega el procesamiento.
- **Capa de Servicio (@Service):** Contiene la lógica de negocio central (ej: LandingProjectService.java).
- **Capa de Repositorio (@Repository):** Encapsula el acceso a datos mediante Spring Data JPA (ej: LandingProjectRepository.java).
- **Data Transfer Object (DTO Pattern):** Patrón Estructural utilizado de forma sistemática (ej: UserRequestDTO, AiResponseDTO) para transportar datos entre el cliente y el servidor. Esto oculta la estructura interna de la base de datos (Entidades) y permite validar las peticiones de entrada utilizando jakarta.validation.

- **Filtro de Intercepción (Intercepting Filter Pattern):** Implementado en la capa de seguridad a través de `JwtAuthenticationFilter.java`. Este filtro intercepta cada solicitud HTTP entrante para validar la presencia y firma del token JWT antes de permitir el acceso a los controladores protegidos.
- **Patrón Builder:** Utilizado extensamente en la instanciación de Entidades y DTOs (apoyado por la librería Lombok con `@Builder`) para facilitar la creación de objetos complejos de manera fluida y legible (ej: `LandingProject.java`).

3. Diagrama de Contexto del Sistema (Modelo C4)

Para representar gráficamente las decisiones arquitectónicas mencionadas, se ha adoptado el Modelo C4, el cual permite visualizar la solución en distintos niveles de abstracción.

3.1 Diagrama de Contexto del Sistema (Nivel 1)

- **Propósito:** Ilustrar el panorama general de `WebLandingSuite`, mostrando como el sistema interactúa con sus usuarios y con dependencias externas a vista de pájaro.
- **Que representa:** El contenedor principal del sistema en el centro, recibiendo la interacción del usuario final (Estudiante/Emprendedor)
- **Relación con el problema y objetivos:** Responde al objetivo general de automatizar la creación web. Este diagrama demuestra visualmente la abstracción técnica. El usuario solo interactúa con un portal amigable, ignorando por completo la complejidad de las conexiones HTTPS y los proveedores de inteligencia artificial que operan por detrás.

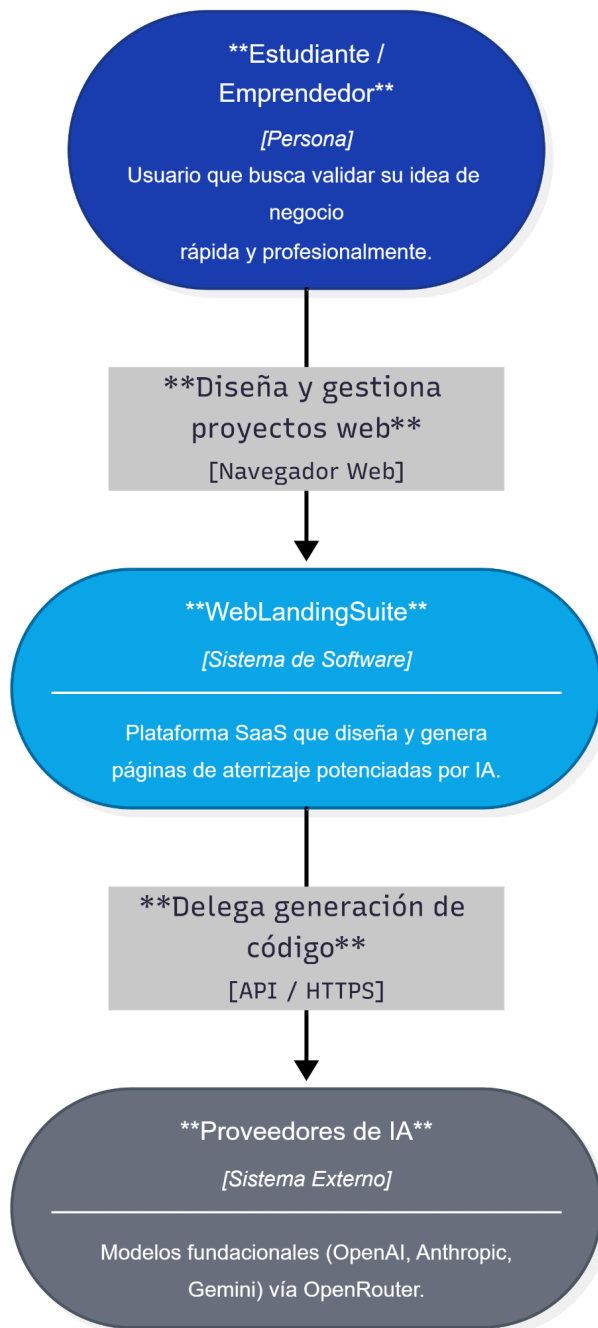


Tabla 9: Diagrama de contenedores nivel 1

3.2 Diagrama de Contenedores (Modelo C4 - Nivel 2)

- **Propósito:** Hacer un "zoom" a la arquitectura del sistema para mostrar las piezas de software ejecutables que lo componen y cómo se comunican entre sí.
- **Que representa:** La separación física de las responsabilidades en cuatro nodos principales: La SPA Frontend en React, el Backend centralizado en Spring Boot, la base de datos PostgreSQL y el Microservicio extractor de IA en Python/FastAPI.
- **Relación con el problema y objetivos:** Cumple directamente con los objetivos específicos 1, 2 y 3. Muestra cómo el Backend central orquesta el flujo, validando

la seguridad y los pagos en la base de datos antes de enviar la petición de carga pesada computacional al microservicio de Python, garantizando una respuesta ágil y escalable para el cliente.

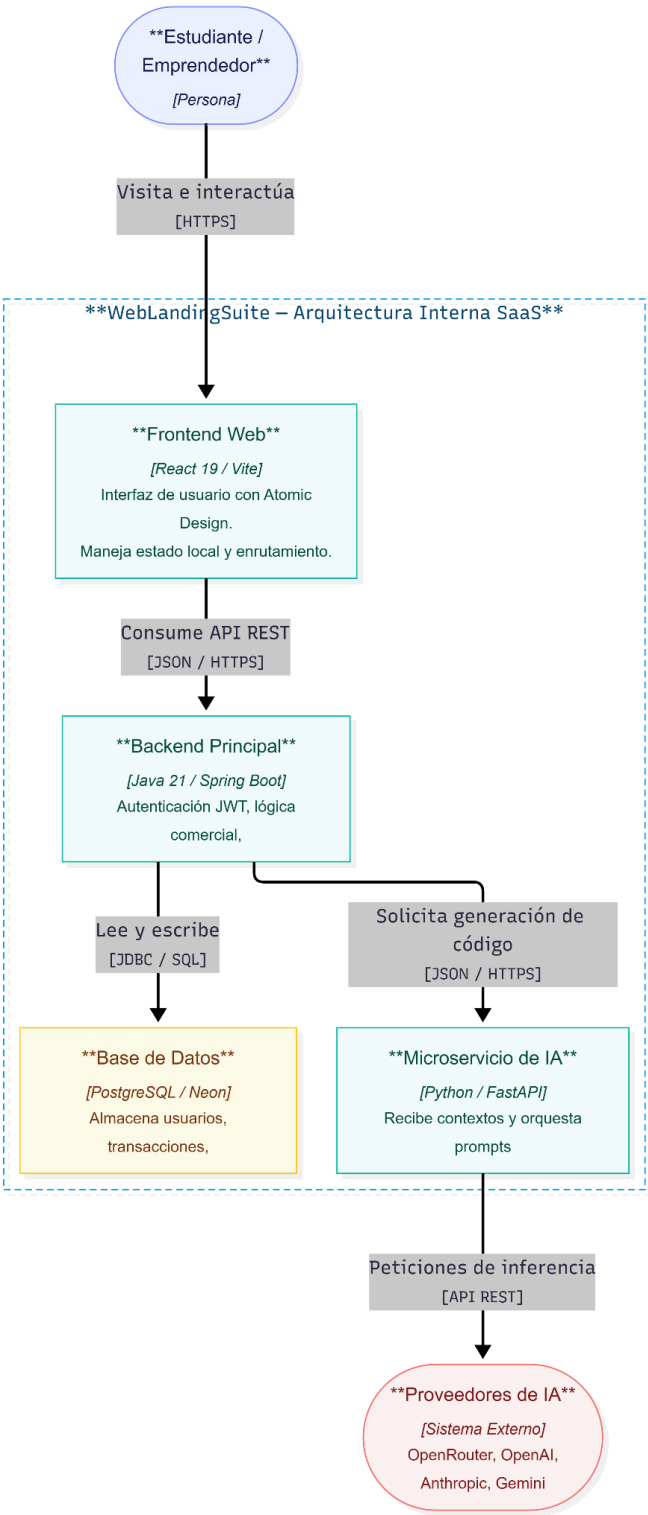
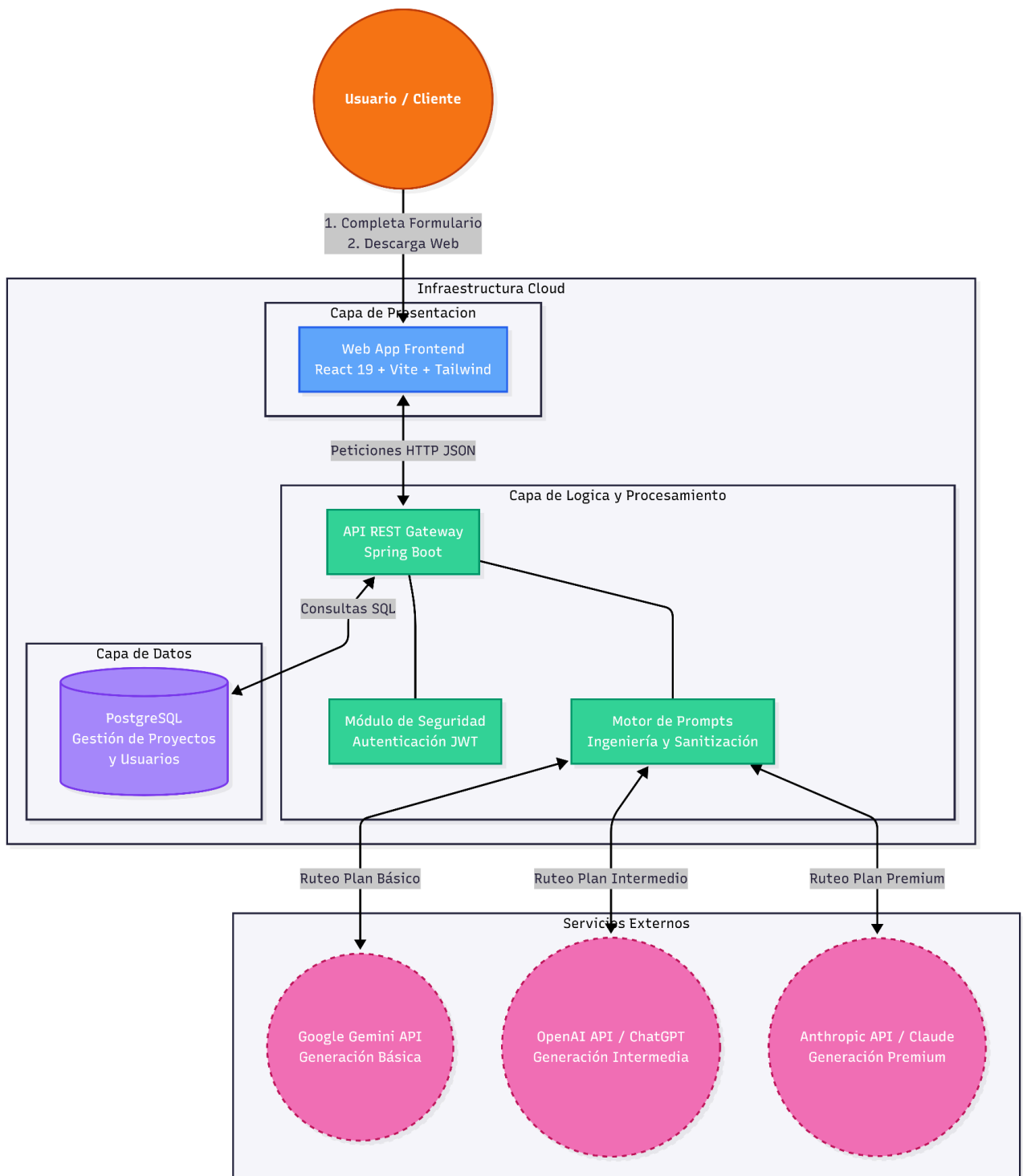


Tabla 10: Diagrama de contenedores nivel 2

3.3 Diagrama de Arquitectura



El diagrama de contenedores ilustra los principales bloques tecnológicos que componen la plataforma y cómo se comunican entre sí. En la **capa de presentación**, el usuario interactúa con una aplicación web desarrollada en React 19 con Vite y Tailwind CSS, la cual envía peticiones HTTP en formato JSON hacia la **capa de lógica y procesamiento**. Esta capa está liderada por un API REST Gateway implementado en Spring Boot, que centraliza el enrutamiento de solicitudes, delega la autenticación al módulo de seguridad

basado en JWT y prepara los prompts a través del Motor de Ingeniería y Sanitización antes de invocar a los servicios de IA externos. La **capa de datos** es gestionada por una base de datos PostgreSQL, que almacena la información de usuarios y proyectos mediante consultas SQL directas desde el backend. Finalmente, la **capa de servicios externos** agrupa tres APIs de inteligencia artificial enrutadas según el plan del usuario: Google Gemini para el plan básico, OpenAI/ChatGPT para el plan intermedio y Anthropic/Claude para el plan premium. Toda la infraestructura opera desplegada en la nube, garantizando disponibilidad, escalabilidad y separación de responsabilidades entre capas.

Tecnologías, lenguaje y dependencias (stack definitivo)

Para la construcción de la plataforma WebLandingSuite, se ha seleccionado un stack tecnológico moderno, escalable y dividido en microservicios, garantizando un alto rendimiento y una separación clara de las responsabilidades.

1. Lenguajes y Frameworks Definidos

- **Frontend (Capa de Presentación):**
 - Librería Principal: React 19
 - Entorno de Construcción: Vite 8.0.9, seleccionado por su compilación ultrarrápida y recarga en caliente (HMR) optimizada.
- **Backend Principal (Capa de Negocio):**
 - Lenguaje: Java 21
 - Framework: Spring Boot 3.2.5. Su robustez y tipado estricto aseguran el manejo íntegro de las transacciones comerciales, la seguridad y las reglas de negocio.
- **Microservicio de IA (Capa de Procesamiento):**
 - Lenguaje: Python 3
 - Framework: FastAPI. Su naturaleza asíncrona es ideal para manejar las peticiones de Machine Learning y la comunicación de los Modelos de lenguaje Grande.

2. Librerías Clave y Dependencias

- **Frontend:**

- UI y Estilos: Tailwind CSS v3.4.19, con configuración de PostCSS y Autoprefixer para compatibilidad cross-browser.
- Enrutamiento: React Router DOM v7.14.2 para la navegación de la Single Page Application (SPA).
- Iconografía: Lucide React v1.14.0 y React Icons v5.6.0 para el sistema de íconos de la interfaz.
- Animaciones: Framer Motion v12.38.0 para transiciones y animaciones declarativas entre componentes y vistas.
- Peticiones HTTP: Axios v1.15.2 para el consumo de los endpoints del Backend.
- Calidad de código: ESLint v9.39.4 con plugins para React Hooks y React Refresh, garantizando consistencia en la sintaxis JSX.
-
- **Backend Central (Java):**
 - Autenticación y Seguridad: Spring Boot Starter Security acoplado con JWT v0.11.5 para la generación y validación de tokens JWT sin estado.
 - **ORM y Base de Datos:** Spring Data JPA con Hibernate, conectado a una base de datos PostgreSQL alojada en NeonDB.
 - Validación: Spring Boot Starter Validation (jakarta.validation) para la integridad de los Data Transfer Objects (DTOs).
 - **Documentación:** Springdoc OpenAPI v2.5.0 (Swagger) para la documentación interactiva de los endpoints.
 - Utilidades: Lombok para reducir el código repetitivo (boilerplate) mediante anotaciones como `@Data` y `@Builder`.
- **Microservicio Externo (Python):**
 - Validación de Datos: Pydantic para el tipado estricto de las cargas útiles (payloads) entrantes.
 - Integración: SDK de OpenAI para la orquestación de prompts hacia modelos fundacionales (Gemini 1.5 Flash, GPT-4o-Mini, Claude 3.5 Sonnet) a través de OpenRouter.

3. Estándares de Código y Arquitectura de Carpetas

Para asegurar la mantenibilidad del código a largo plazo, el equipo ha establecido los siguientes lineamientos:

- **Linters y Formato:** En el ecosistema frontend se implementa ESLint v9 con reglas recomendadas por React Hooks, asegurando consistencia en la sintaxis de JavaScript/JSX y previniendo la declaración de variables sin uso.
- **Convenciones y Nomenclatura:** Se emplea camelCase para variables y métodos, y PascalCase para el nombramiento de clases y componentes.

- **Modelamiento de Base de Datos:** Como regla estricta del proyecto, los nombres de las tablas y los atributos en la base de datos relacional deben definirse y mantenerse en inglés, asegurando coherencia con el dominio del modelo de negocio.
- **Estructura Frontend (Atomic Design Estricto):**
 - /src/components/atoms/: Elementos indivisibles (ej. Button.jsx, Input.jsx).
 - /src/components/molecules/: Uniones de átomos (ej. FormField.jsx).
 - /src/components/organisms/: Secciones complejas con estados (ej. Navbar.jsx, ContactSection.jsx).
 - /src/pages/: Vistas completas ruteables.
- **Estructura Backend (MVC Multicapa):**
 - /controller/: Controladores REST.
 - /service/: Lógica de negocio y llamadas de APIs externas (AiService.java).
 - /repository/: Interfaces de Spring Data JPA.
 - /model/: Entidades de la base de datos.
 - /data/: Objetivos de transferencia para request y responses.
 - /security/: Filtros JWT y configuraciones de acceso.
 - /exception/: Manejo centralizado de errores (GlobalExceptionHandler.java)
- **Captura de Pantalla de la estructura de carpetas del Frontend y Backend**

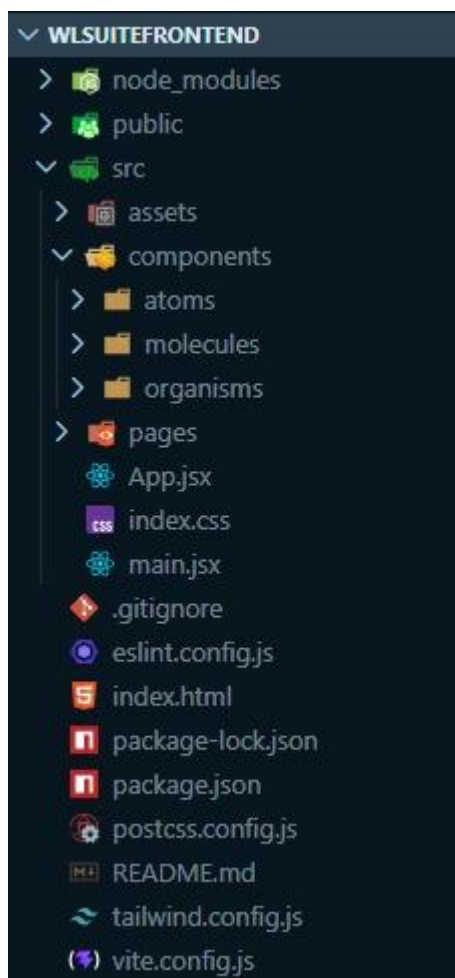


Tabla 11 y 12: Pantallazos de la estructura de carpetas Frontend y Backend

4. Repositorios, Ramas Principales y Flujo de Trabajo

El código fuente del sistema está versionado y respaldado en la nube (GitHub). Se ha adoptado un flujo colaborativo basado en repositorios separados para cada microservicio.

- **Repositorio Frontend:** <https://github.com/Amaaruu/WLSuiteFrontend.git>
- **Repositorio Backend (Java):** <https://github.com/Amaaruu/Landingbackend.git>
- **Repositorio API (Python):** <https://github.com/Amaaruu/WLSuitePythonAPI.git>

Flujo de Trabajo (Git Workflow):

- **Ramas:** El desarrollo opera con una rama main protegida (entorno de producción) y una rama dev (entorno de desarrollo e integración).
- **Pull Request (PR):** Toda funcionalidad, componente atómico o endpoint debe desarrollarse en una rama aislada (feature/nombre-tarea) y fusionarse hacia dev mediante un Pull Request. Esto obliga a una revisión de código (Code Review) por parte de otro integrante del equipo antes de su aceptación, minimizando la inyección de errores al servidor principal.

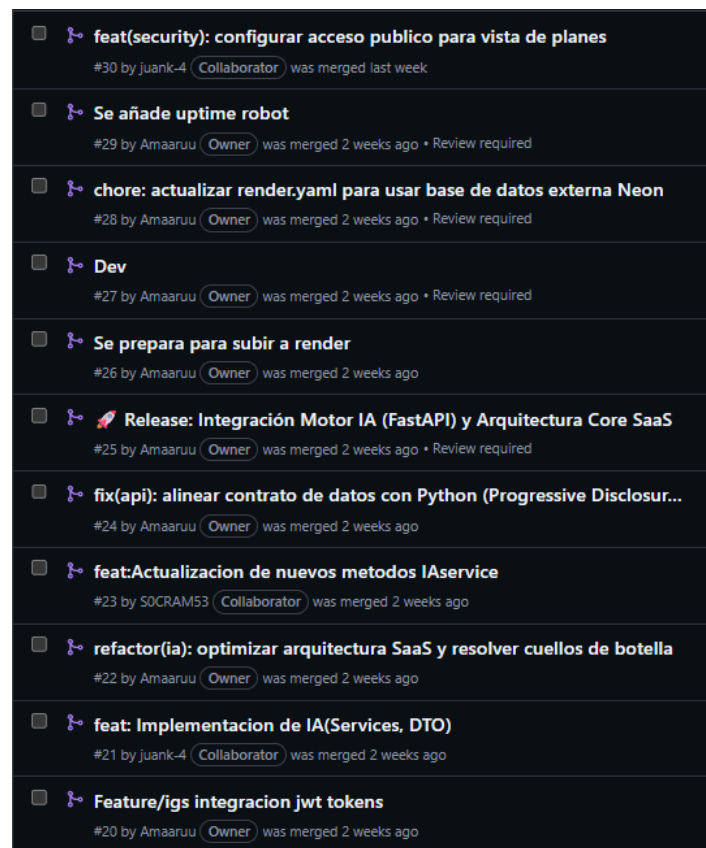
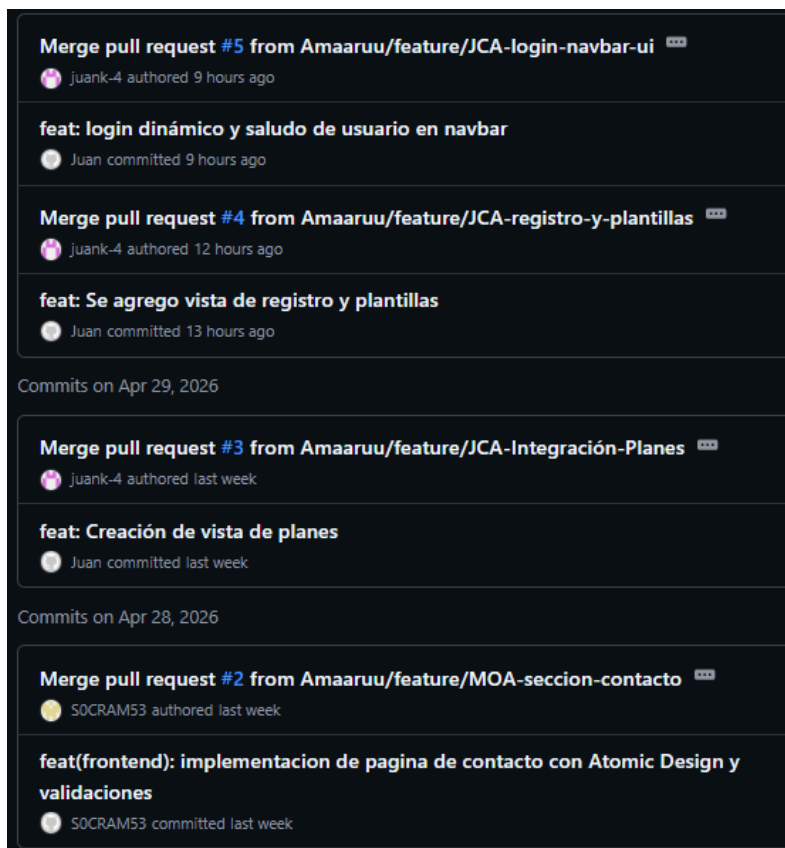


Tabla 13 y 14: Pantallazo de los Pull Request

- Evidencia del flujo de trabajo mediante Pull Request y aplicación de metodología de diseño (Atomic Design) en el repositorio Frontend.
- Historial de commits en el repositorio Backend evidenciando la integración de seguridad (JWT), conexión a la base de datos externa y orquestación con la API de Python.

Configuración de servidor de producción

Para el paso a producción de **WebLandingSuite**, se optó por una arquitectura basada íntegramente en la nube utilizando servicios PaaS (Platform as a Service) y DBaaS (Database as a Service). Esta decisión elimina la necesidad de gestionar infraestructura física, garantizando alta disponibilidad, despliegues continuos automatizados y escalabilidad.

A continuación, se detalla la configuración y los componentes de la infraestructura, así como el paso a paso del despliegue:

1. Plataformas y Componentes Elegidos

- **Base de Datos (Capa de Datos):** PostgreSQL alojado en **Neon** (DBaaS). Proporciona una base de datos relacional serverless, ideal para manejar de forma segura las transacciones y usuarios.
- **Backend Principal (Spring Boot):** Desplegado en **Render** como un Web Service. Render se encarga de la compilación y ejecución automática del ecosistema Java.
- **Microservicio de IA (FastAPI):** Desplegado en **Render** como un servicio independiente.
- **Frontend (React/Vite):** Alojado en **Vercel**, plataforma optimizada para aplicaciones Single Page Application (SPA), que distribuye el contenido estático mediante una CDN global.

2. Requisitos del Servidor y Entornos de Ejecución (Runtimes)

El ecosistema delega la gestión de los comandos de compilación y arranque (Build Command y Start Command) a las plataformas en la nube, las cuales detectan automáticamente los entornos gracias a la configuración del repositorio.

- **Runtime Backend:** Java 21 (definido en la configuración del entorno para el núcleo transaccional) y Python 3 (para el motor de Inteligencia Artificial).
- **Runtime Frontend:** Node.js (gestionado por Vercel para ejecutar Vite).

- **Variables de Entorno Estrictas:** Para asegurar la conexión entre los servicios sin exponer credenciales en el código fuente, se inyectaron las siguientes variables (Secretos) en el entorno de producción de Render:
 - **DB_URL:** Cadena de conexión hacia la base de datos PostgreSQL en Neon.
 - **DB_USERNAME:** Usuario con privilegios de lectura/escritura en la base de datos.
 - **DB_PASSWORD:** Credencial cifrada de acceso a la base de datos.
 - **JWT_SECRET_KEY:** Llave criptográfica para la firma y validación de los tokens de sesión de los usuarios.
 - **PYTHON_API_URL:** Endpoint interno para que Spring Boot se comuniquen con el microservicio de IA.
 - **VITE_API_BASE_URL:** (Configurada también en Vercel) URL pública que el Frontend utiliza para consumir los servicios del Backend.

3. Instrucciones de Despliegue

Paso 1: Configuración de la Base de Datos en Neon

1. Se inicializó un nuevo proyecto en la consola de Neon seleccionando PostgreSQL.
2. Se generaron las credenciales de acceso (URL, usuario y contraseña).
3. Se habilitaron los permisos de red para permitir conexiones entrantes desde los servidores de Render.

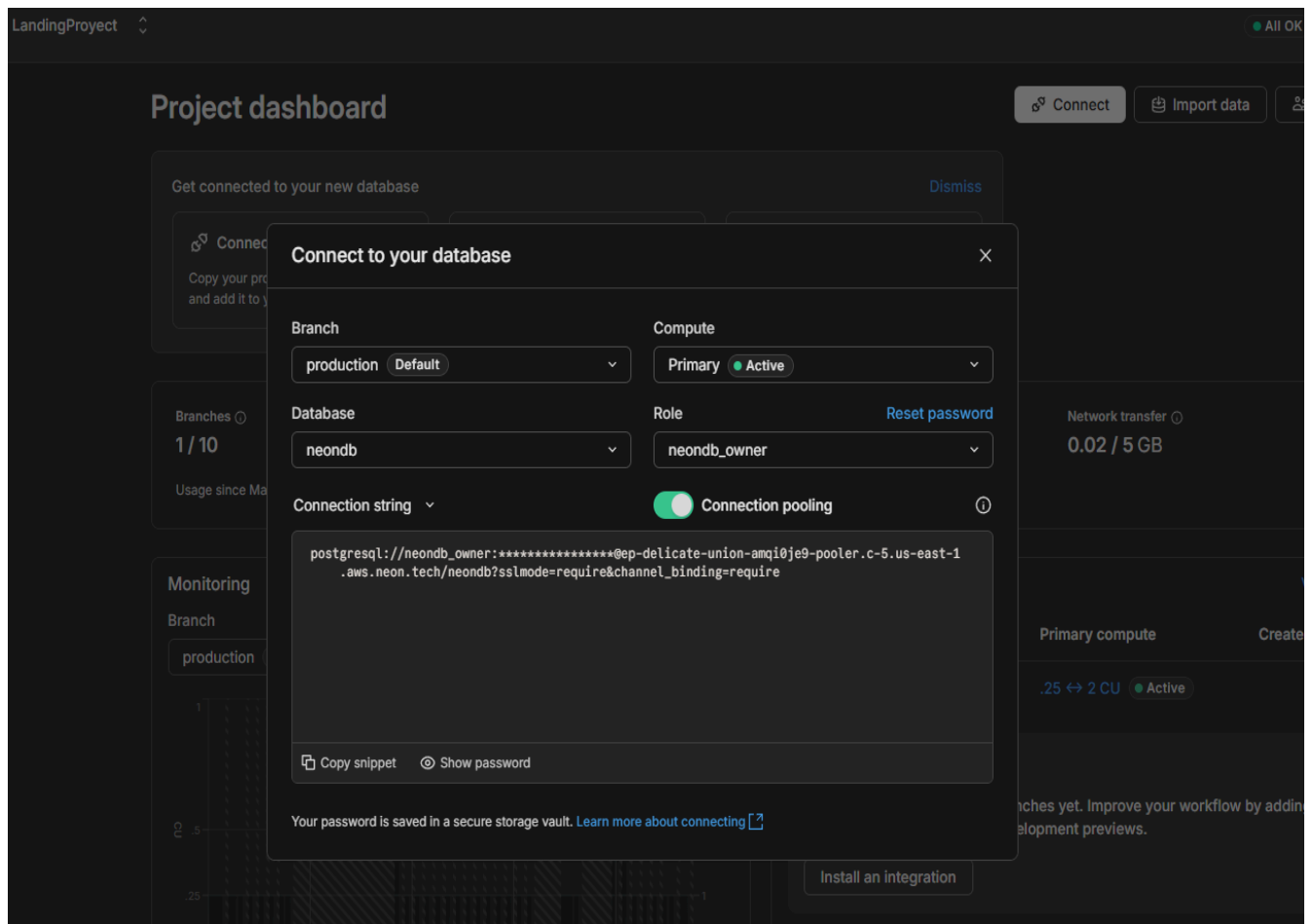


Tabla 15: Pantallazo de la base de datos con las variables de entorno

Paso 2: Despliegue de los Servicios Backend en Render

1. En el panel de Render, se crearon dos nuevos "Web Services", enlazándolos directamente a los repositorios correspondientes en GitHub (Landingbackend y WLSuitePythonAPI).
2. Gracias a los archivos de configuración en los repositorios (incluyendo `render.yaml`), Render detectó automáticamente la necesidad de compilar el proyecto Maven para Java y de instalar el archivo `requirements.txt` para Python.
3. Se ingresó al apartado "Environment" de cada servicio en Render para inyectar manualmente las variables de entorno mencionadas anteriormente.
4. Se procedió con el primer despliegue manual, permitiendo a Render resolver dependencias, compilar los proyectos y exponer los puertos HTTP seguros (HTTPS).

WEB SERVICE

Landingbackend

Docker

Free

Upgrade your instance →

Connect

Manual Deploy

Service ID: `srv-d7mm5nqpmmb573c2frh0`

Amaaruu / Landingbackend

main

<https://landingbackend-sfrk.onrender.com>

Environment

Create environment group

Environment Variables

Export

Edit

Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more.](#)

KEY	VALUE
DB_PASSWORD	
DB_URL	
DB_USERNAME	
JWT_SECRET_KEY	
PYTHON_API_URL	

V

Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more.

Upgrade now

All logs

Q Search

Last hour

↕

...

May 18

07:24:51 PM [zjmtt] using 'hibernate dialect' (remove the property setting and it will be selected by default)

07:25:06 PM [xglwc] Hibernate: select u1_0.user_id,u1_0.active,u1_0.email,u1_0.last_name,u1_0.name,u1_0.password,u1_0.registered_at,u1_0.role,u1_0.uuid from users u1_0 where (u1_0.active = true) and u1_0.email=?

07:25:06 PM [xglwc] 2026-05-18T23:25:06.324Z INFO 1 --- [LandingBackend] [main] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)

07:25:06 PM [xglwc] 2026-05-18T23:25:06.194Z INFO 1 --- [LandingBackend] [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'

07:25:10 PM [xglwc] 2026-05-18T23:25:10.506Z INFO 1 --- [LandingBackend] [main] o.s.d.j.r.query.QueryEnhancerFactory : Hibernate is in classpath; If applicable, HQL parser will be used.

07:25:26 PM ==> No open ports detected, continuing to scan...

07:25:26 PM ==> Docs on specifying a port: <https://render.com/docs/web-services#port-binding>

07:25:29 PM [xglwc] 2026-05-18T23:25:29.083Z WARN 1 --- [LandingBackend] [main] JpaBaseConfiguration\$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning

07:25:34 PM [xglwc] 2026-05-18T23:25:34.086Z INFO 1 --- [LandingBackend] [main] o.s.s.web.DefaultSecurityFilterChain : Will secure any request with [org.springframework.security.web.session.DisableEncodeUrlFilter445769e8, org.springframework.security.web.context.request.async.WebAsyncManagerIntegrationFiltera22152da0, org.springframework.security.web.filter.CorsFiltera86c299a, org.springframework.security.web.authentication.logout.LogoutFiltera4ffc5625, LandingBackendsecurity.JwtAuthenticationFiltera6bc1c8d9, org.springframework.security.web.savedrequest.RequestCacheAwareFiltera2779acb2, org.springframework.security.web.servletapi.SecurityContextHolderAwareRequestFiltera2af781ff3, org.springframework.security.web.authentication.AnonymousAuthenticationFiltera5899de18, org.springframework.security.web.session.SessionManagementFiltera2dda28a3, org.springframework.security.web.access.ExceptionTranslationFiltera279c94ad, org.springframework.security.web.access.intercept.AuthorizationFiltera6bd992b]

07:25:41 PM [xglwc] 2026-05-18T23:25:41.983Z INFO 1 --- [LandingBackend] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 10000 (http) with context path ''

07:25:42 PM [xglwc] 2026-05-18T23:25:42.105Z INFO 1 --- [LandingBackend] [main] Landing.Backend.BackendApplication : Started BackendApplication in 127.194 seconds (process running for 138.7)

07:25:43 PM [xglwc] 2026-05-18T23:25:43.694Z INFO 1 --- [LandingBackend] [io-10000-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'

07:25:43 PM [xglwc] 2026-05-18T23:25:43.702Z INFO 1 --- [LandingBackend] [io-10000-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'

07:25:43 PM [xglwc] 2026-05-18T23:25:43.704Z INFO 1 --- [LandingBackend] [io-10000-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 2 ms

07:25:48 PM ==> Your service is live 🎉

07:25:48 PM ==>

07:25:48 PM ==> Available at your primary URL <https://landingbackend-sfrk.onrender.com>

07:25:48 PM ==>

07:25:48 PM ==>

07:25:48 PM ==>

07:26:17 PM [zjmtt] 2026-05-18T23:26:17.676Z INFO 1 --- [LandingBackend] [onShutdownHook] o.s.o.support.DefaultLifecycleProcessor : Shutdown phase 2147483647 ends with 1 bean still running after timeout of 30000ms: [applicationTaskExecutor]

07:26:17 PM [zjmtt] 2026-05-18T23:26:17.748Z INFO 1 --- [LandingBackend] [onShutdownHook] j.LocalContainerEntityManagerFactoryBean : Closing JPA EntityManagerFactory for persistence unit 'default'

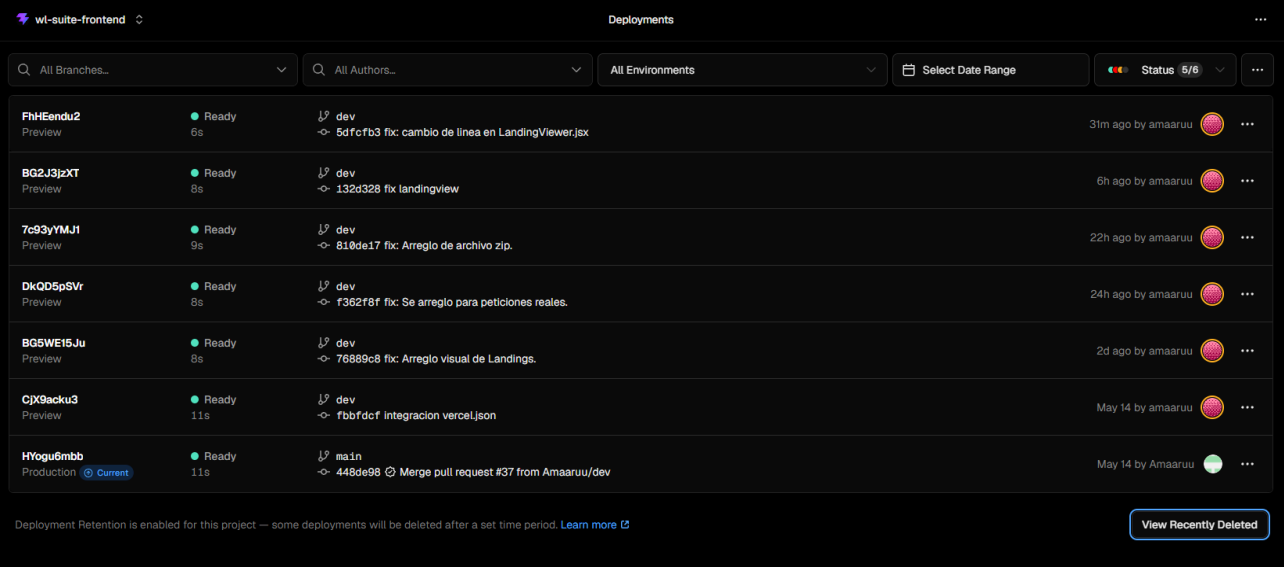
07:26:17 PM [zjmtt] 2026-05-18T23:26:17.750Z INFO 1 --- [LandingBackend] [onShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown initiated...

07:26:17 PM [zjmtt] 2026-05-18T23:26:17.946Z INFO 1 --- [LandingBackend] [onShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown completed.

Paso 3: Integración Continua y Despliegue (CI/CD) del Frontend en Vercel

- Se vinculó el repositorio de GitHub (WLSuiteFrontend) con la plataforma Vercel.
- Vercel identificó automáticamente que el proyecto estaba construido sobre React y Vite, asignando por defecto los comandos de construcción (npm run build).

- Se configuró el pipeline de CI/CD (Integración y Despliegue Continuo): Cada vez que el equipo aprueba un *Pull Request* y hace un *merge* hacia la rama main en GitHub, Vercel intercepta el evento (Webhook) y lanza un nuevo despliegue a producción de forma automática en cuestión de segundos, sin intervención humana.



The screenshot shows the Vercel Deployments interface for the project 'wl-suite-frontend'. It displays a list of recent deployments, each with a unique ID, status, environment, commit hash, description, and deployment time. The 'main' branch deployment is highlighted as 'Current'.

ID	Status	Environment	Commit	Description	Time	Author
FhHEendu2	Ready	dev	5dfc7b3	fix: cambio de linea en LandingViewer.jsx	31m ago	amaaruu
BQ2J3jzXT	Ready	dev	132d328	fix landingview	6h ago	amaaruu
7c93yYMJ1	Ready	dev	810de17	fix: Arreglo de archivo zip.	22h ago	amaaruu
DkQD5pSVr	Ready	dev	f362f8f	fix: Se arreglo para peticiones reales.	24h ago	amaaruu
BG5WEI5Ju	Ready	dev	76889c8	fix: Arreglo visual de Landings.	2d ago	amaaruu
CjX9acku3	Ready	dev	fbbfddf	integracion vercel,json	May 14	amaaruu
HY0gu6mbb	Ready	main	448de98	Merge pull request #37 from Amaaruu/dev	May 14	Amaaruu

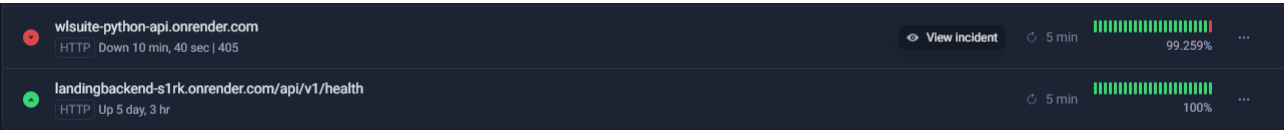
Deployment Retention is enabled for this project — some deployments will be deleted after a set time period. [Learn more](#)

[View Recently Deleted](#)

4. Registro (Logs) y Monitoreo Básico

Garantizar la salud del sistema en producción es vital para un software SaaS. Para la auditoría de peticiones, se están consumiendo los **Logs Nativos en tiempo real** que proveen tanto Render como Vercel en sus respectivas consolas. Esto permite depurar errores, visualizar las transacciones de bases de datos y monitorear el comportamiento del motor de IA.

Para el monitoreo de disponibilidad, se implementó **UptimeRobot**. Esta herramienta externa realiza un "ping" constante a los *endpoints* críticos del Backend (como el servicio de estado HealthController o la API de Python). En caso de que el servidor caiga o la latencia supere los límites aceptables, UptimeRobot enviará alertas inmediatas al equipo de desarrollo, asegurando una respuesta proactiva ante incidentes.



The screenshot shows the UptimeRobot monitoring dashboard. It displays two monitored endpoints: 'wlsuite-python-api.onrender.com' and 'landingbackend-s1rk.onrender.com/api/v1/health'. Both endpoints are shown as 'Up' with their respective HTTP status codes and uptime percentages.

Endpoint	Status	HTTP	Uptime	Incident
wlsuite-python-api.onrender.com	Down	10 min, 40 sec 405	99.259%	View incident
landingbackend-s1rk.onrender.com/api/v1/health	Up	5 day, 3 hr	100%	

Estado de avance del desarrollo (evidencia de producto final)

1. Descripción General del Proyecto

El desarrollo del proyecto WebLandingSuite ha avanzado conforme a la planificación definida, implementando progresivamente las funcionalidades clave del sistema en las tres capas de la arquitectura: frontend (React/Vite), backend (Spring Boot) y microservicio de inteligencia artificial (FastAPI/Python). Al cierre de la EA2, la plataforma cuenta con un MVP funcional desplegado en producción, accesible públicamente, con el flujo completo de generación de Landing Pages mediante IA operativo de extremo a extremo.

El sistema actualmente permite al usuario registrarse, autenticarse, completar un formulario dinámico con los datos de su negocio, seleccionar un plan de diseño, activar la generación mediante inteligencia artificial, previsualizar el resultado en tiempo real y descargar el código generado en formato .ZIP.

2. Funcionalidades Implementadas

A continuación se presenta el estado actual de las principales funcionalidades del sistema:

N°	Funcionalidad	Descripción	Estado
1	Registro e inicio de sesión (JWT)	Permite crear cuenta y autenticarse con token seguro	Implementada
2	Formulario dinámico de captura	Guía al usuario paso a paso para ingresar datos de su negocio	Implementada
3	Motor de Prompt Engineering escalonado	Selecciona automáticamente la plantilla IA según el plan elegido	Implementada
4	Generación de Landing Page con IA	Envía prompt a OpenRouter y recibe código HTML/CSS/JS	Implementada
5	Post-procesamiento y sanitización del código	Limpia y valida el código crudo generado por la IA	Implementada

6	Notificaciones por correo electrónico	Envío de bienvenida, contacto y proyecto listo vía SMTP	En Progreso
7	Descarga del proyecto en .zip	Empaqueta y entrega los archivos finales al usuario	Implementada
8	Gestión de planes (Básico, Intermedio, Premium)	Visualización y selección de planes desde el frontend	Implementada
9	Simulación de pasarela de pago	Flujo de cobro simulado para activar la generación	Implementada
10	Registro de auditoría y logs	Registra eventos críticos con IP y timestamp en la base de datos	Implementada
11	Dashboard de gestión de proyectos	Historial de Landing Pages generadas por el usuario	Implementada
12	Diseño responsivo (vista móvil)	Adaptación de la interfaz a dispositivos de pantalla pequeña	Implementada

3. Evidencia de Funcionalidad

A continuación se detalla la evidencia técnica de las principales funcionalidades implementadas.

1. Registro e inicio de sesión con JWT

- **Descripción:** Permite a los usuarios crear una cuenta con correo y contraseña, e iniciar sesión recibiendo un token JWT para acceder a las funciones protegidas de la plataforma.
- **Estado:** Implementada
- **Evidencia técnica:**
 - Endpoint Asociado: *POST /api/v1/auth/register* y *POST /api/v1/auth/login*
 - Respuesta esperada:

```

▼ {, ...}
  message: "Login exitoso"
  name: "Juanito"
  token: "eyJhbGciOiJIUzI1NiJ9.eyJyb2xlIjoidXNlciIsInN1YiI

```

2. Formulario dinámico de captura de datos

- **Descripción:** Interfaz multi-paso desarrollada en React que guía al usuario a través de campos como nombre del proyecto, rubro, tono de comunicación, paleta de colores y secciones activas de la Landing Page.
- **Estado:** Implementada
- Evidencia técnica:
 - Componentes: Step1.jsx, Step2.jsx, Step3Visual.jsx en /src/components/organisms/steps/

Sector del negocio

Selecciona la categoría que mejor describe tu negocio.

Gastronomía Cafés, restaurantes, pastelerías	Tecnología SaaS, apps, servicios digitales	Salud Clínicas, terapias, bienestar
Educación Cursos, tutorías, academias	Moda Ropa, accesorios, estilo de vida	Fitness Deporte, entrenamiento, gym
Legal Estudios jurídicos, asesoría	Inmobiliaria Agentes, propiedades, arriendo	Turismo Viajes, tours, alojamiento
E-commerce Tienda online generalista	Finanzas Pagos, inversión, crédito	Consultoría Asesoría profesional
Belleza Estética, cosmética, cuidado personal	Eventos Bodas, celebraciones, entretenimiento	Arte & Creación Fotógrafos, diseñadores, artistas
Otro Negocio de propósito general		

Objetivo principal de la landing

Define qué acción quieres provocar en el visitante.

Vender Concretar ventas directas	Captar leads Obtener datos de contacto	Reservas Agendar hora, mesa o sesión
Informar Presentar el negocio o producto	Descarga App, ebook o recurso gratuito	Registro Crear cuenta o suscripción

Público objetivo

¿A quién va dirigida tu landing? La IA adaptará el lenguaje y la persuasión.

Jóvenes 18-28 años, nativos digitales	Adultos 30-50 años, prácticos	Profesionales Ejecutivos y especialistas
Empresas (B2B) Tomadores de decisión	Emprendedores Fundadores, startups	Padres y madres Familias con hijos

Tu negocio

Información básica que la IA usará para construir toda la landing.

Nombre del proyecto o negocio

Ej: Cafetería El Rincón

Propuesta de valor — ¿qué ofreces y qué te hace distinto?

Ej: Café de especialidad en Santiago que vende café de origen y repostería artesanal en un ambiente íntimo y tranquilo.

Llamado a la acción (CTA) — ¿qué quieres que haga el visitante?

Ej: Reserva tu mesa, Empieza gratis, Contáctanos hoy

Tono de comunicación

Define cómo habla tu marca. La IA adaptará cada párrafo de copy a este tono.

Profesional

Confiable, preciso, serio

Cercano

Conversacional, amigable

Elegante

Sofisticado, palabras cuidadas

Jovial

Divertido, con energía

Inspirador

Motivacional, enfocado en potencial

Técnico

Especializado, directo a expertos

Nivel de formalidad

¿Cómo se dirige la landing al visitante?

Formal

Usted, sin informalidades

Semi-formal

Tuteo respetuoso y amable

Informal

Coloquial y directo

Color primario

Color dominante de la landing: botones, títulos, elementos de marca.



Azul marino



Azul cielo



Verde bosque



Verde menta



Terracota



Rojo vibrante



Morado



Rosa



Negro



Gris oscuro



Gris neutro

Blanco



Crema



Dorado



Naranja



Cian

Color secundario / acento

Color de contraste y acento. Se usa en fondos de sección, líneas y detalles.



Azul marino



Azul cielo



Verde bosque



Verde menta



Terracota



Rojo vibrante



Morado



Rosa



Negro



Gris oscuro



Gris neutro

Blanco



Crema



Dorado



Naranja



Cian

5. Envío de correos electrónicos transaccionales

- **Descripción:** El sistema dispara correos automáticos al registrarse (bienvenida), al enviar el formulario de contacto y al completar la generación de un proyecto (notificación con enlace).
- **Estado:** Implementada
- Evidencia técnica:
 - EmailService.java — métodos sendWelcomeEmail(), sendContactEmail(), sendProjectReadyEmail()
 - Correo de soporte configurado: weblandingsuite@gmail.com

```
[ASYNC] Procesando correo de contacto de: marco@gmail.com
[RESEND] Correo enviado a weblandingsuite@gmail.com. ID: 2104ab38-93fe-4ae6-a8f4-9b73b4423bdb
```

4. Calidad del Desarrollo y Buenas Prácticas

Durante el desarrollo se aplicaron las siguientes prácticas para asegurar la calidad del código y del producto:

Validaciones:

- Validación de datos de entrada en el frontend mediante estado controlado de React (Controlled Components).
- Validación en el backend mediante DTOs con anotaciones jakarta.validation (campos obligatorios, longitud, formato).

Manejo de errores:

- Uso de respuestas HTTP estándar (200, 201, 400, 401, 403, 500).
- Manejo centralizado de excepciones mediante GlobalExceptionHandler.java.
- Mensajes de error descriptivos retornados al frontend en formato JSON.

Seguridad:

- Autenticación mediante JWT gestionada por JwtAuthenticationFilter.java.
- Variables de entorno para credenciales: DB_URL, DB_PASSWORD, JWT_SECRET_KEY, PYTHON_API_URL.
- Configuración de CORS
- Sanitización del código generado por la IA para prevenir XSS.

Estructura del código:

- Separación en capas: controller/, service/, repository/, model/, dto/, security/, exception/.
- Arquitectura Atomic Design en el frontend: atoms/, molecules/, organisms/, pages/.

- Patrón Builder con Lombok (@Data, @Builder) en entidades y DTOs.

5. Evidencia de Desarrollo

Control de versiones y trabajo colaborativo

El desarrollo fue gestionado mediante Git y GitHub, con tres repositorios separados por microservicio:

- **Frontend:** <https://github.com/Amaaruu/WLSuiteFrontend.git>
- **Backend (Java):** <https://github.com/Amaaruu/Landingbackend.git>
- **Microservicio IA (Python):** <https://github.com/Amaaruu/WLSuitePythonAPI.git>

Ramas Utilizadas

- main: versión estable desplegada en producción.
- dev: rama de integración y desarrollo continuo.
- feature/nombre-tarea: ramas de funcionalidades individuales (ej: feature/JCA-login-navbar-ui, feature/MOA-seccion-contacto).

Evidencias incluidas

- Historial de commits del proyecto en Frontend y Backend.
- Pull Requests aprobados con Code Review de al menos un integrante.
- Tareas finalizadas en el tablero Jira.
- Despliegues automáticos activos en Vercel (Frontend) y Render (Backend + IA).
- Monitoreo de disponibilidad mediante UptimeRobot: Backend al 100% de uptime, Microservicio IA al 99.259%.

Fronted:

The screenshot displays the GitHub interface for the 'WLSuiteFrontend' repository. At the top, the repository name 'WLSuiteFrontend' is shown with a 'Public' badge and a 'Watch' button indicating 0 watchers. Below this, the 'main' branch is selected. A search bar labeled 'Go to file' is present. The main content area shows a list of files and folders with their respective commit messages and timestamps. The files listed include 'public', 'src', '.env', '.gitignore', 'README.md', 'eslint.config.js', 'index.html', 'package-lock.json', 'package.json', 'postcss.config.js', 'tailwind.config.js', 'vercel.json', and 'vite.config.js'. The commit messages range from 'Feat: Commit inicial' to 'feat: se añadieron mejoras UI.' and 'integracion vercel.json'.

File/Folder	Commit Message	Timestamp
public	Feat: Commit inicial	last month
src	fix: se modifican rutas	2 days ago
.env	se añade login context	last week
.gitignore	Feat: Commit inicial	last month
README.md	Feat: Commit inicial	last month
eslint.config.js	Feat: Commit inicial	last month
index.html	feat: se añadieron mejoras UI.	last week
package-lock.json	feat: rediseño de la página de Nosotr...	last week
package.json	feat: rediseño de la página de Nosotr...	last week
postcss.config.js	feat(frontend): setup de arquitectura, ...	last month
tailwind.config.js	feat: se añadieron mejoras UI.	last week
vercel.json	integracion vercel.json	last week
vite.config.js	integracion vercel.json	last week

Backend:

LandingbackendPublic

Watch0

main

Go to file

Code

Amaaruu Merge pull request #80 from Amaaruu/d... b70e8b4 · 3 hours ago 162 Commits

.mvn/wrapper	Init: Configuración base de Spring Boot	last month
src	fix:cloudinary	3 hours ago
.dockerignore	feat: corregir validaciones en DtOs pa...	last month
.gitattributes	Init: Configuración base de Spring Boot	last month
.gitignore	feat: Se arregla docker	last month
Dockerfile	feat:se implementa docker y se prote...	last month
docker-compose.yml	feat: Se arregla docker	last month
mvnw	Init: Configuración base de Spring Boot	last month
mvnw.cmd	Init: Configuración base de Spring Boot	last month
pom.xml	feat:Se integra configuracion de clou...	2 days ago
render.yaml	fix:cloudinary	3 hours ago

Pull Request Realizados Frontend:

0 Open 42 Closed

AuthorLabelProjectsMilestonesReviewsAssigneeSort

Dev

#42 by Amaaruu (Owner) was merged 2 days ago • Review required

1

Feature/moa integracion imagenes

#41 by SOCRAM53 (Collaborator) was merged 2 days ago

1

feat: Implementación de pagina de pago simulado.

#40 by Amaaruu (Owner) was merged 2 days ago

1

feat:Se integra la subida de imagenes

#39 by SOCRAM53 (Collaborator) was merged 2 days ago

1

integracion vercel.json

#38 by Amaaruu (Owner) was closed 3 days ago • Review required

1

Dev

#37 by Amaaruu (Owner) was merged last week • Review required

Feature/igs support view

#36 by Amaaruu (Owner) was merged last week

Feature/igs support view

Pull Request Realizado Backend:

☐		Feature/moa integracion cloudinary	#75 by Amaaruu (Owner) was merged 2 days ago
☐		feat: Arreglo de correos.	#74 by Amaaruu (Owner) was merged 2 days ago
☐		feat:Se integra configuracion de cloudinary	#73 by SOCRAM53 (Collaborator) was merged 2 days ago
☐		fix:Se modifica aplicacion propertior en la seccion gmail	#72 by Amaaruu (Owner) was merged 3 days ago • Review required
☐		Dev	#71 by SOCRAM53 (Collaborator) was merged 3 days ago • Review required
☐		fix: se actualizo el link de la api de python	#70 by Amaaruu (Owner) was merged 3 days ago • Review required
☐		fix:se modifica landingProjectService / AiGenerationTaks	#69 by Amaaruu (Owner) was merged 3 days ago • Review required
☐		Dev	#68 by Amaaruu (Owner) was merged 3 days ago • Review required
☐		feat:Nuevos archivos DTO	#67 by Amaaruu (Owner) was merged 3 days ago
☐		feat:Cada usuario vera exclusivamente sus proyectos	#66 by SOCRAM53 (Collaborator) was merged last week
☐		integracion de url	#65 by Amaaruu (Owner) was merged last week • Review required
☐		Dev	#64 by Amaaruu (Owner) was merged last week • Review required
☐		feat:Se corrige el flujo de los logs	#63 by SOCRAM53 (Collaborator) was merged last week

Integración de plataforma Render:

WEB SERVICE

Landingbackend

Docker

Free

Upgrade your instance →

Connect

Manual Deploy

Service ID: `srv-d7mm5nqpmbs73c2frh0`

Amaaruu / Landingbackend

main

https://landingbackend-s1rk.onrender.com

ⓘ

Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more.

Upgrade now

Filter events

31

✔

Deploy live for `b70e8b4`: Merge pull request #80 from Amaaruu/dev fix:cloudinary

May 21, 2026 at 5:09 PM

🔄

Deploy started for `b70e8b4`: Merge pull request #80 from Amaaruu/dev fix:cloudinary

Environment updated

May 21, 2026 at 5:05 PM

✔

Deploy live for `b70e8b4`: Merge pull request #80 from Amaaruu/dev fix:cloudinary

May 21, 2026 at 4:50 PM

Rollback

Integración de plataforma Vercel:

wl-suite-frontend

Deployments

...

Q All Branches...

Q All Authors...

All Environments

Select Date Range

Status 5/6

...

FhHEandu2

Preview

Ready6s

dev

5dfcfb3 fix: cambio de linea en LandingViewer.jsx

31m ago by amaaruu

BG2J3jXT

Preview

Ready8s

dev

132d328 fix landingview

6h ago by amaaruu

7c93yYMJ1

Preview

Ready9s

dev

810de17 fix: Arreglo de archivo zip.

22h ago by amaaruu

DkQD5pSVr

Preview

Ready8s

dev

f362f8f fix: Se arreglo para peticiones reales.

24h ago by amaaruu

BG5WE15Ju

Preview

Ready8s

dev

76889c8 fix: Arreglo visual de Landings.

2d ago by amaaruu

CjX9acku3

Preview

Ready11s

dev

fbbfdcf integracion vercel.json

May 14 by amaaruu

HYogu6mbb

Production

Current

Ready11s

main

448de98 Merge pull request #37 from Amaaruu/dev

May 14 by Amaaruu

Deployment Retention is enabled for this project — some deployments will be deleted after a set time period. [Learn more](#)

View Recently Deleted

Integración de Plataforma UpTimeRobot:

landingbackend-s1rk.onrender.com/api/v1

HTTP/S monitor for <https://landingbackend-s1rk.onrender.com/api/v1>

Test Notification

Pause

Edit

Current status

Up

Currently up for 8d 3h 59m

Last check

1s ago

Checked every 5m

Get 60 sec. checks

Last 24 hours

100%

0 incidents, 0m down

Last 7 days

100%

0 incidents, 0m down

Last 30 days

98.867%

2 incidents, 7h, 3m, 38s down

Last 365 days

---%
Unlock with paid plans

MTBF

N/A

7 days

Integraciones necesarias

El sistema WebLandingSuite contempla distintas integraciones externas necesarias para completar su funcionamiento. Estas integraciones permiten generación de código con IA, persistencia de datos, despliegue en la nube, envío de notificaciones y monitoreo del sistema en producción.

1. OpenRouter - Motor de Inteligencia Artificial

OpenRouter actúa como gateway hacia los modelos fundacionales de lenguaje que generan el código HTML/CSS/JS de las Landing Pages. Desde el microservicio FastAPI, se realizan peticiones HTTP autenticadas a OpenRouter, que enruta la solicitud al modelo correspondiente según el plan del usuario.

Datos de entrada:

- Payload JSON con los campos del formulario: nombre del proyecto, rubro, tono de comunicación, paleta de colores (JSONB), secciones activas, tipo de acción (callToAction).
- Prompt estructurado y ensamblado por el motor de Prompt Engineering del backend.

Datos de salida:

- Código HTML/CSS/JS crudo generado por el modelo IA seleccionado.
- Estructura post-procesada y sanitizada antes de ser almacenada en la base de datos y enviada al frontend.

Modelos utilizados según plan:

Plan	Modelo IA	Proveedor
Básico	Gemini 1.5 Flash	Google (vía OpenRouter)
Intermedio	GPT-4o-Mini	OpenAI (vía OpenRouter)
Premium	Claude 3.5 Sonnet	Anthropic (vía OpenRouter)

Consideraciones de seguridad:

- API Key almacenada como variable de entorno en Render (OPENROUTER_API_KEY), sin exposición en el código fuente.
- Rate limiting implícito mediante el consumo de créditos por uso real.
- Validación estricta de la respuesta cruda antes de procesarla (eliminación de texto conversacional, extracción exclusiva del bloque HTML/CSS).

Evidencia de Integración:

KEY	VALUE
CLOUDINARY_URL	*****
DB_PASSWORD	*****
DB_URL	*****
DB_USERNAME	*****
FRONTEND_URL	*****
JWT_SECRET_KEY	*****
PYTHON_API_URL	*****
RESEND_API_KEY	*****
RESEND_FROM_EMAIL	*****

2. API REST Backend - Comunicación Frontend ↔ Spring Boot

El frontend desarrollado en React se integra con el backend implementado en Spring Boot mediante una API RESTful. Esta integración permite registrar usuarios, autenticarlos, gestionar planes, registrar transacciones, crear proyectos y descargar el código generado.

Datos de entrada:

- Solicitudes HTTP desde el frontend con token JWT en el header Authorization.
- Payload JSON de los formularios del usuario.
- Parámetros de ruta y query para consultas específicas.

Datos de salida:

- Respuestas JSON con información de usuarios, planes, proyectos y transacciones.
- Código HTML generado por la IA, encapsulado en la respuesta del proyecto.

- Archivo .zip como stream de bytes para la descarga del código.

Consideraciones de seguridad:

- JWT validado en cada solicitud mediante JwtAuthenticationFilter.java.
- CORS configurado para permitir exclusivamente solicitudes desde el dominio del frontend en Vercel.
- Uso de HTTPS en producción tanto en Render como en Vercel.
- Variables de entorno para todas las credenciales críticas del servidor.

Evidencia de integración

Documentacion Swagger:

Swagger
Support by SMARTBEAR

/v3/api-docs

Explore

WebLandingSuite API ^{1.0} OAS 3.0

/v3/api-docs

API con Seguridad JWT e IA

Servers

https://landingbackend-s1rk.onrender.com - Generated server url

Authorize

Autenticación

Endpoints para Login y Registro de usuarios

- POST /api/v1/auth/register
- POST /api/v1/auth/login

Logs de Auditoría

Endpoints para el registro y consulta de eventos del sistema

- GET /api/v1/logs Listar historial de eventos
- POST /api/v1/logs Registrar evento manualmente
- GET /api/v1/logs/{id} Consultar evento específico

Landing Projects

Gestión de proyectos generados por IA

- GET /api/v1/projects Listar proyectos — admin: todos | usuario: solo los suyos
- POST /api/v1/projects Crear proyecto
- PATCH /api/v1/projects/{id}/status Actualizar estado (solo admin)
- GET /api/v1/projects/{id} Obtener proyecto por ID
- DELETE /api/v1/projects/{id} Eliminar proyecto

Registro:

200	Response body <pre data-bbox="317 329 1473 398">{ "token": "eyJhbGciOiJIUzI1NiJ9.eyJyY2xiIjoidXN1ciIsInN1YiI6ImhcnVnMTIzQGdtYWlsbWVzIiwiaWF0IjE2MjYwMzQ0MDc2MywiZXNpdjoxNzc5NDk3MTYzFQ.tX0KttarszTaB2yMC6QfuYyvBDVR88Wo3ceIMvcdVM", "message": "Registro exitoso", "name": "marco" }</pre>  Download Response headers <pre data-bbox="317 409 1473 761">alt-svc: h3=":443"; ma=86400 cache-control: no-cache,no-store,max-age=0,must-revalidate cf-cache-status: DYNAMIC cf-ray: 9ff7cd298c510c2b-SCL content-encoding: br content-length: 186 content-type: application/json date: Fri, 22 May 2026 00:46:03 GMT expires: 0 pragma: no-cache priority: u=1,i rندر-id: adf24457-a540-45fd server: cloudflare server-timing: cfExtPri strict-transport-security: max-age=31536000 ; includeSubDomains vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers,Accept-Encoding x-content-type-options: nosniff x-frame-options: DENY x-render-origin-server: Render x-xss-protection: 0</pre>		
Responses			
Code	<table><thead><tr><th data-bbox="317 772 448 824">Description</th><th data-bbox="448 772 1473 824">Links</th></tr></thead></table>	Description	Links
Description	Links		

Login:

Code

Details

200

Response body

```
{  "token": "eyJhbGciOiJIUzI1NiJ9.eyJyb2xlIjoiaW11ciIsInN1YiI6Im1hcmNwMTIzQGdtYnIsLnNvbSIsIm1hdCI6MTc3OTQxMTA4MCwiZXNwIjoxNzc5NDk3NDgwFQ.LRL-K72cGh3hbAPBe-6nCx86PEzhK6uZl92_vRN_W1E",  "message": "Login exitoso",  "name": "marco" }
```

 Download

Response headers

```
alt-svc: h3="443"; ma=86400
cache-control: no-cache,no-store,max-age=0,must-revalidate
cf-cache-status: DYNAMIC
cf-ray: 9ff7d4eaa452245-SCL
content-encoding: br
content-length: 192
content-type: application/json
date: Fri, 22 May 2026 00:51:20 GMT
expires: 0
pragma: no-cache
priority: u=1,i
rndr-id: 868c6ac4-7fea-4e18
server: cloudflare
server-timing: cfExtPri
strict-transport-security: max-age=31536000 ; includeSubDomains
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers,Accept-Encoding
x-content-type-options: nosniff
x-frame-options: DENY
x-render-origin-server: Render
x-xss-protection: 0
```

Responses

Code

Description

Links

Frontend Login:

Request URL	https://landingbackend-s1rk.onrender.com/api/v1/auth/login
Request Method	POST
Status Code	200 OK
Remote Address	216.24.57.7:443
Referrer Policy	strict-origin-when-cross-origin

▼ Response headers	
Access-Control-Allow-Credentials	true
Access-Control-Allow-Origin	https://www.weblandingsuite.co

```
▼ {,...}
  message: "Login exitoso"
  name: "marco"
  token: "eyJhbGciOiJIUzI1NiJ9.eyJyb2xlIjoidXNlciIsInN1YiI
```

3. Neon - Base de Datos PostgreSQL

Neon provee la capa de persistencia del sistema en formato serverless, almacenando usuarios, proyectos, transacciones, planes y logs de auditoría. La conexión se gestiona mediante Spring Data JPA con Hibernate como ORM.

Datos de entrada:

- Operaciones CRUD generadas por el backend desde las capas @Service y @Repository.
- Consultas SQL generadas por Hibernate a partir de las entidades del modelo de datos.

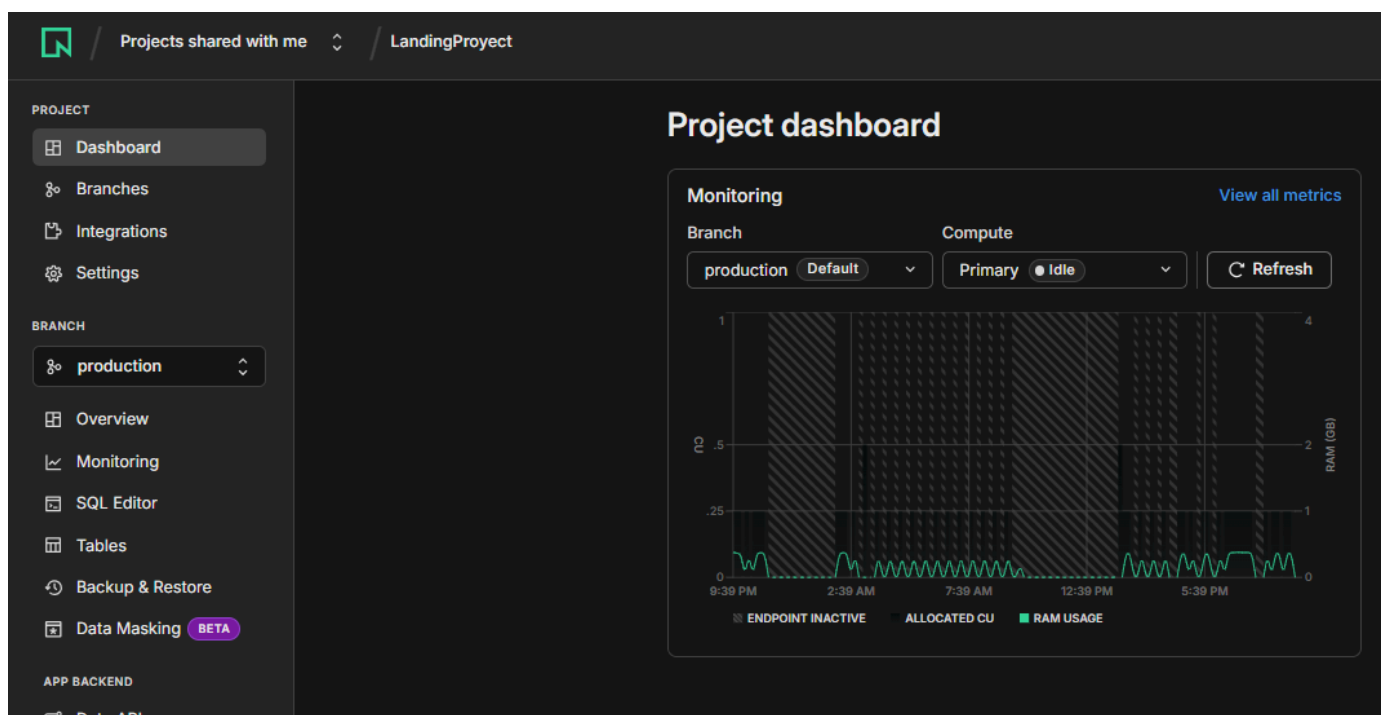
Datos de salida:

- Registros persistidos de las cinco entidades principales: user, design_plan, transaction, landing_project y log.
- Estructuras JSONB para color_palette y ai_metadata en la tabla landing_project.

Consideraciones de seguridad:

- Credenciales de conexión (DB_URL, DB_USERNAME, DB_PASSWORD) gestionadas exclusivamente como variables de entorno en Render.
- Permisos de red restringidos a los rangos de IP de los servidores de Render.
- Connection pooling habilitado en Neon para soportar concurrencia sin colapsar el pool de conexiones.
- Propiedades ACID garantizadas por PostgreSQL para la integridad de las transacciones comerciales.

Evidencia de integración:



DB_PASSWORD	*****
DB_URL	*****

4. Uptime Robot - Monitoreo de Disponibilidad de Produccion

UptimeRobot se integra como herramienta de monitoreo externo que realiza verificaciones periódicas (cada 5 minutos) sobre los endpoints críticos del sistema, alertando al equipo ante caídas o degradación del servicio.

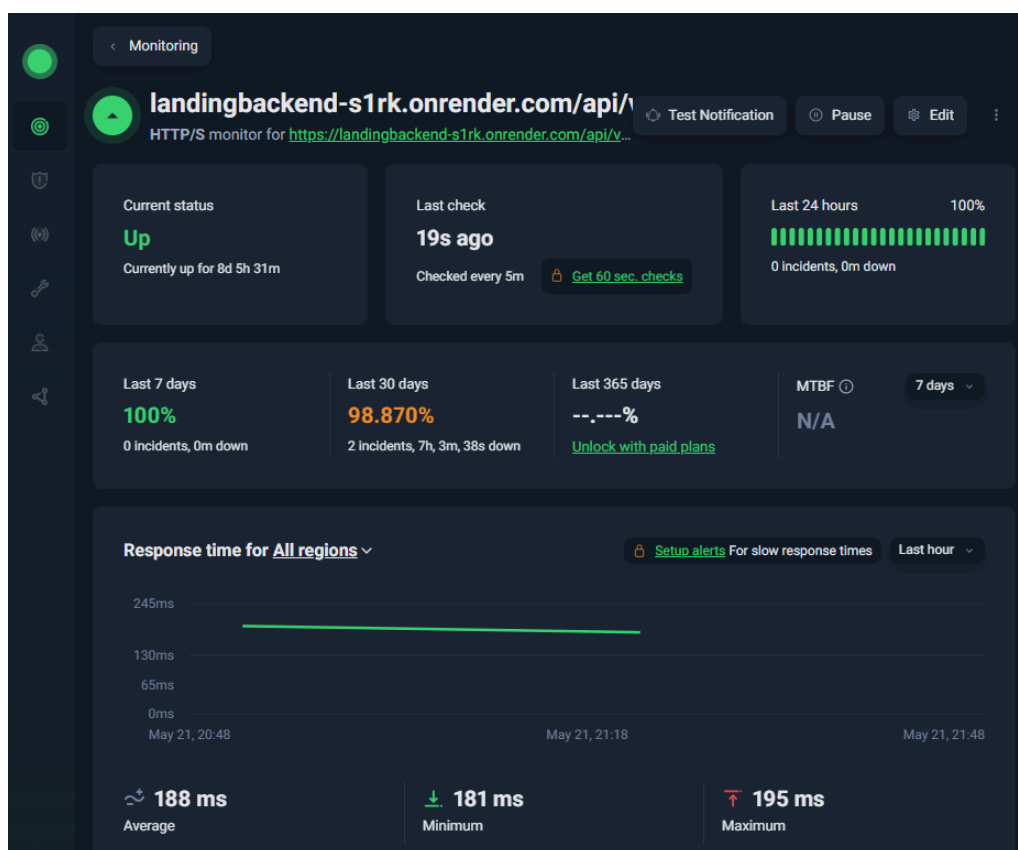
Endpoints monitoreados:

- GET <https://landingbackend-s1rk.onrender.com/api/v1/health> — Backend Spring Boot (100% uptime).
- GET <https://wlsuite-python-api.onrender.com> — Microservicio IA FastAPI (99.259% uptime).

Datos de entrada: Ping HTTP cada 5 minutos a cada endpoint configurado.

Datos de salida: Reporte de disponibilidad en tiempo real, historial de incidentes y alertas por correo o notificación ante caídas.

Consideraciones de seguridad: No se exponen credenciales en los endpoints de health check. Los endpoints monitoreados son públicos y de solo lectura.



- **Resend - Servicio de Envío de Correos Transaccionales**

Resend actúa como la plataforma de envío de correos electrónicos transaccionales de WebLandingSuite, reemplazando la configuración manual de SMTP. Permite disparar notificaciones automáticas al usuario en eventos críticos del flujo: registro exitoso (bienvenida), generación completada (proyecto listo) y formulario de contacto. La integración se realiza directamente desde el backend Spring Boot mediante el SDK de Resend o llamadas HTTP autenticadas a su API REST.

Datos de entrada:

- Dirección de correo electrónico del destinatario (extraída del registro del usuario).
- Tipo de evento que dispara el correo (bienvenida, proyecto listo, contacto).
- Contenido dinámico del mensaje (nombre del usuario, nombre del proyecto generado, enlace de descarga).

Datos de salida:

- Confirmación de envío con ID de mensaje único generado por Resend.
- Estado de entrega del correo (enviado, rebotado, abierto) consultable desde el dashboard de Resend.

Consideraciones de seguridad:

- API Key de Resend almacenada como variable de entorno en Render (RESEND_API_KEY), sin exposición en el código fuente.
- Dominio remitente verificado en Resend para evitar que los correos sean clasificados como spam.
- Las llamadas a la API de Resend se realizan exclusivamente desde el servidor backend, nunca desde el frontend, previniendo la exposición de credenciales en el cliente.

- **Cloudinary - Gestión y Almacenamiento de Imágenes**

Cloudinary se integra como la plataforma de gestión y entrega de activos multimedia de WebLandingSuite. Permite que los usuarios carguen imágenes de su negocio (logotipo, fotos de productos o servicios) durante el proceso de completar el formulario, las cuales son almacenadas de forma optimizada en la nube y referenciadas directamente dentro del código HTML/CSS generado por la IA, enriqueciendo el resultado final de la Landing Page con contenido visual real del cliente.

Datos de entrada:

- Archivo de imagen cargado por el usuario (formatos: JPG, PNG, WebP, SVG) desde el formulario de captura de datos.
- Parámetros de transformación opcionales (ancho, alto, formato de salida, calidad) definidos por el backend.

Datos de salida:

- URL pública de la imagen almacenada en el CDN de Cloudinary, lista para ser inyectada como atributo src en las etiquetas del código HTML generado por la IA.
- Metadatos del archivo subido (nombre, formato, dimensiones, tamaño) retornados por la API de Cloudinary tras la carga exitosa.

Consideraciones de seguridad:

- Credenciales de Cloudinary (Cloud Name, API Key, API Secret) almacenadas como variables de entorno en Render, sin exposición en el repositorio.
- Las subidas de imágenes se firman digitalmente en el backend (firma HMAC-SHA1) antes de enviarse a Cloudinary, impidiendo cargas no autorizadas desde el cliente.
- Restricciones de tipo y tamaño de archivo configuradas en el backend para prevenir la carga de archivos maliciosos o de gran volumen que puedan afectar el rendimiento del sistema.

5. Conclusión de Integraciones

Las integraciones implementadas en WebLandingSuite permiten que el sistema funcione como una plataforma SaaS completa y operativa en producción. La combinación de OpenRouter para la generación de código con IA, Neon para la persistencia transaccional, Render y Vercel para el despliegue continuo, UptimeRobot para el monitoreo activo, conforma una infraestructura robusta, escalable y de bajo costo operativo alineada con el modelo de negocio del proyecto.

Estrategia y enfoque de calidad

Definición de Calidad para WebLandingSuite

En el contexto de WebLandingSuite, calidad no es un concepto abstracto, es la garantía de que un microempresario o profesional independiente pueda completar el flujo completo de la plataforma, desde registrarse hasta descargar su Landing Page generada, sin encontrar errores críticos, sin exponer sus datos y sin recibir código malicioso o roto en el resultado. Esta definición orienta todas las decisiones del equipo respecto a qué probar, con qué profundidad y en qué orden.

Dado que el sistema opera sobre tres capas tecnológicas heterogéneas (React en el frontend, Spring Boot en el Backend y FastAPI en Python para el microservicio de IA), la estrategia de calidad debe ser transversal y capaz de validar tanto el comportamiento interno de cada componente como la correcta interacción entre ellos.

Atributos de Calidad Priorizados

No todos los atributos de calidad tienen el mismo peso en este proyecto. El siguiente orden refleja la priorización definida por el equipo en función del impacto directo sobre el usuario y sobre la integridad del sistema.

1. Seguridad (Prioridad Crítica)

WebLandingSuite maneja credenciales de usuarios, tokens de sesión JWT y código HTML generado dinámicamente que se renderiza en el navegador del cliente. Un fallo de seguridad en cualquiera de estos puntos puede comprometer cuentas, filtrar información o inyectar scripts maliciosos en el dispositivo del usuario (Cross-Site Scripting). Por ello, la seguridad se posiciona como el atributo de mayor peso en la estrategia de pruebas.

2. Funcionalidad (Prioridad Alta)

El sistema debe cumplir con exactitud cada una de las historias de usuario definidas en el backlog. Si el formulario no captura los datos correctamente, si el motor de IA no recibe el payload estructurado o si la descarga del archivo .zip falla, el producto pierde su razón de ser. Toda funcionalidad implementada debe ser verificable y tener cobertura de prueba documentada.

3. Fiabilidad (Prioridad Alta)

La plataforma está desplegada en producción con monitoreo activo mediante UptimeRobot, que registra un 100% de uptime en el backend y un 99.259% en el microservicio de IA. La fiabilidad se evalúa tanto a nivel de disponibilidad del servicio como en la consistencia transaccional de la base de datos (propiedades ACID garantizadas por PostgreSQL en Neon), asegurando que ningún pago quede registrado sin su proyecto asociado o viceversa.

4. Rendimiento (Prioridad Alta)

La promesa de valor del producto es generar una Landing Page en menos de 15 segundos. Esto impone restricciones concretas de tiempo de respuesta que deben verificarse bajo carga. El sistema no puede degradarse ante múltiples solicitudes concurrentes de generación.

5. Mantenibilidad (Prioridad Media)

El equipo de tres personas debe ser capaz de incorporar cambios sin introducir regresiones. La cobertura de pruebas automatizadas actúa como red de seguridad que permite refactorizar con confianza. Se establece un mínimo de 90% de cobertura de líneas en las capas de negocio y seguridad del backend.

6. Usabilidad (Prioridad Menor en pruebas formales)

La usabilidad se valida principalmente de forma manual a través de la revisión del flujo completo end-to-end por parte del equipo. El diseño responsivo y la experiencia del formulario multi-paso se verifican en distintos tamaños de pantalla, pero no se automatizan en esta etapa del proyecto.

Niveles y Tipos de Prueba: de lo más pequeño a lo más completo

El equipo adopta una pirámide de pruebas que va desde la validación de la unidad más pequeña hasta la verificación del sistema completo en producción:

Nivel 1 - Pruebas Unitarias

Se verifican componentes aislados sin dependencias externas. En el backend Java se utiliza JUnit 5 con Mockito, validando la lógica interna de servicios críticos como JwtService (generación, validación y extracción de tokens), reglas de negocio de servicios de dominio y el comportamiento de los GlobalExceptionHandler. En el microservicio Python se ejecuta pytest sobre las funciones de construcción de prompts (build_prompt, get_json_structure) y la lógica de mapeo de parámetros. En el frontend React se aplican pruebas con Vitest y React Testing Library sobre componentes de administración como AdminDashboard, AdminSupport y páginas que requieren manejo de estado y llamadas a la API.

Nivel 2 - Pruebas de Integración

Se verifica que los distintos módulos del sistema interactúan correctamente entre sí. Las pruebas de integración del backend utilizan MockMvc con una base de datos PostgreSQL real levantada mediante Testcontainers, garantizando que los tests no operen sobre mocks de base de datos sino sobre un motor relacional real equivalente al de producción. Se cubren flujos completos como el registro de usuario, inicio de sesión con JWT, creación de proyectos y acceso al visor de Landing Pages con tokens de landing.

Nivel 3 - Pruebas de Seguridad

Implementadas como una suite específica (SecurityEndpointsTest), estas pruebas validan que los endpoints protegidos rechacen correctamente solicitudes sin token (HTTP 403), con tokens manipulados, con tokens de formato inválido y con roles insuficientes (un usuario con ROLE_USER intenta acceder a recursos de administrador). También se verifica que las validaciones de los DTOs con jakarta.validation rechacen contraseñas demasiado cortas, correos con formato inválido y cuerpos de solicitud vacíos

Nivel 4 - Pruebas de Regresión

Las pruebas etiquetadas con @Tag("regression") se ejecutan automáticamente antes de cada merge hacia la rama main mediante el pipeline de CI/CD en GitHub Actions. Esto garantiza que nuevas funcionalidades no rompan el comportamiento ya validado del sistema. El job de regresión depende de que el job de pruebas unitarias e integración haya pasado exitosamente, estableciendo una cadena de validación obligatoria.

Nivel 5 - Pruebas de Rendimiento

Se utiliza Gatling 3.11 con el plugin Maven para ejecutar simulaciones de carga sobre los endpoints críticos del backend. Los escenarios incluyen: health check con rampa de 10 usuarios en 30 segundos, consulta de planes a 5 usuarios por segundo durante 60 segundos, y flujo de login con 50 usuarios en rampa de 10 segundos. Las aserciones globales establecen que el percentil 95 de tiempo de respuesta sea menor a 2000ms, el tiempo medio menor a 500ms y la tasa de éxito superior al 95%.

Enfoque: Automatizado con Validación Manual Complementaria

La estrategia combina automatización continua con verificación manual en los puntos donde la automatización no aporta valor suficiente:

Automatizado: pruebas unitarias, de integración, seguridad, regresión y rendimiento en el backend Java; pruebas unitarias de componentes React en el frontend; pruebas de funciones del microservicio Python. Todo esto se ejecuta en el pipeline de CI/CD de GitHub Actions ante cada push a main o develop y ante cada Pull Request hacia main.

Manual: validación del flujo completo end-to-end (registro → formulario → generación → previsualización → descarga), inspección visual del diseño responsivo en dispositivos móviles, verificación de correos transaccionales enviados por Resend y revisión del comportamiento del Iframe de previsualización con distintos outputs de la IA.

Datos de prueba: el backend utiliza emails generados dinámicamente con UUID en cada ejecución para evitar colisiones entre pruebas ("integration_" + UUID.randomUUID() + "@test.com"). El microservicio Python utiliza fixtures de Pytest con proyectos predefinidos para los tres planes (Básico, Intermedio y Premium). El frontend utiliza mocks de Axios con datos controlados.

Ambiente de ejecución: las pruebas unitarias corren con base de datos H2 en memoria; las pruebas de integración levantan PostgreSQL 15 real en contenedor Docker (Testcontainers); las pruebas de rendimiento apuntan a localhost:8080 y deben ejecutarse con el servidor activo.

Definition Of Done para la Capa de Pruebas

Criterio	Descripción
Cobertura mínima alcanzada	Las capas de negocio y seguridad mantienen $\geq 90\%$ de cobertura de líneas verificada por JaCoCo, excluyendo DTOs, modelos, configuración y servicios externos (EmailService, AiService, ImageUploadService).
Pruebas unitarias existentes	Toda lógica de negocio no trivial tiene al menos un test unitario que cubre el camino feliz y los casos de error esperados.
Pruebas de integración aprobadas	El flujo de extremo a extremo del módulo (ej: registro + login + creación de proyecto) se verifica sobre base de datos real sin errores.
Pipeline verde	El job de CI en GitHub Actions pasa completamente: compile $\rightarrow$ test $\rightarrow$ jacoco:check $\rightarrow$ regression. Ningún merge a main se aprueba con pipeline en rojo.
Sin defectos abiertos de severidad alta	No existen bugs registrados con impacto en seguridad, pérdida de datos o bloqueo total del flujo principal del usuario.
Seguridad validada	Los endpoints protegidos rechazan correctamente tokens inválidos, manipulados y solicitudes sin autorización, verificado por SecurityEndpointsTest.
Revisión de código aprobada	Al menos un integrante distinto al autor del cambio aprobó el Pull Request en GitHub antes del merge.
Documentación actualizada	El endpoint o funcionalidad está reflejado en la documentación Swagger/OpenAPI disponible en producción.

Plan de Pruebas

El presente Plan de Pruebas formaliza el esfuerzo de verificación y validación del sistema **WebLandingSuite**, organizando qué se va a probar, quién lo hace, bajo qué condiciones y con qué criterios se acepta o rechaza el resultado. Este documento aplica a la totalidad del MVP en producción, cubriendo las tres capas de la arquitectura: Frontend (React/Vite), Backend (Spring Boot/Java) y Microservicio de IA (FastAPI/Python).

1. Alcance de las Pruebas

Módulos y componentes incluidos en el alcance

Módulo	Descripción	Capas cubiertas
M1 — Autenticación	Registro, login, generación y validación de JWT	Backend, Frontend
M2 — Gestión de Planes	CRUD de planes por administrador, consulta pública	Backend, Frontend
M3 — Motor de IA y Creación	Formulario multi-paso, prompt engineering, generación de código	Backend, Microservicio Python, Frontend
M4 — Post-procesamiento	Sanitización del HTML/CSS generado, prevención de XSS	Backend
M5 — Descarga de Proyecto	Empaquetado en .zip, descarga por token de landing	Backend, Frontend
M6 — Dashboard de Usuario	Historial de proyectos, visor de Landing Page (Iframe)	Frontend
M7 — Auditoría y Logs	Registro de eventos críticos, consulta por administrador	Backend
M8 — Seguridad Transversal	Control de acceso por roles, endpoints protegidos	Backend
M9 — Soporte y Contacto	Formulario de contacto público, envío de correo	Backend, Frontend
M10 — Panel Administrador	Gestión de usuarios, proyectos, planes y logs	Frontend

Módulos y componentes excluidos del alcance formal

Componente excluido	Justificación	Riesgo asumido
Pasarela de pago real (Stripe/Webpay)	El flujo de cobro es simulado con fines académicos; no se integra pago real en producción	Bajo. El riesgo financiero es nulo al no operar dinero real
Servicio de correo Resend (EmailService)	Depende de una API externa y un dominio verificado; excluido de cobertura JaCoCo por acoplamiento externo	Medio. Un fallo en Resend no bloquea el flujo principal
Servicio Cloudinary (ImageUploadService)	Igual que Resend, es un servicio externo con integración verificada manualmente	Bajo. Las imágenes son opcionales en la generación
Servicio AiService (llamada directa a OpenRouter)	Excluido de JaCoCo; el comportamiento de la IA externa se valida a nivel de integración, no unitario	Bajo. El microservicio Python cubre esta responsabilidad
Compatibilidad con navegadores móviles	No se automatizan pruebas E2E en viewports móviles con Playwright	Medio. El diseño responsivo se valida manualmente

2. Elementos y Funcionalidades a Probar

Funcionalidades nuevas (primera verificación):

Desde la perspectiva del usuario final, el plan de pruebas cubre los siguientes escenarios de uso:

- Un visitante puede navegar el sitio, ver los planes disponibles y registrarse con correo y contraseña, recibiendo un token JWT que le otorga acceso a su cuenta.
- Un usuario autenticado puede completar el formulario multi-paso con los datos de su negocio (nombre, sector, tono, paleta de colores, secciones activas), seleccionar un plan (Básico, Intermedio o Premium) y activar la generación de su Landing Page mediante el pago simulado.
- El sistema genera el código HTML/CSS/JS mediante el microservicio de Python, lo sanitiza en el backend y muestra una previsualización interactiva dentro de un Iframe seguro.
- El usuario puede descargar el código de su Landing Page como archivo .zip directamente desde la previsualización.
- Un administrador puede gestionar usuarios, proyectos, planes y consultar el historial de logs desde el panel de administración protegido por rol.

Funcionalidades de regresión (re-verificación ante cambios):

Las siguientes funcionalidades se re-prueban ante cualquier merge hacia main, ya que cambios en capas transversales (seguridad, base de datos) pueden introducir regresiones:

- Flujo completo de autenticación (registro → login → acceso a recursos protegidos).
- Validación de DTOs con jakarta.validation (campos obligatorios, formatos de email, longitud de contraseña).
- Comportamiento de JwtAuthenticationFilter ante tokens manipulados, expirados o con formato inválido.
- Acceso por roles: endpoints de administrador denegados a usuarios con ROLE_USER.
- Consulta pública de planes sin autenticación.

3. Criterios de Aceptación y Rechazo

Criterios de aceptación (condiciones para dar las pruebas por terminadas):

Criterio	Umbral mínimo
Cobertura de líneas en capas de negocio y seguridad (JaCoCo)	$\geq 90\%$
Cobertura de ramas (branch coverage) en capas de negocio	$\geq 50\%$
Tasa de éxito de pruebas unitarias ejecutadas	100% (0 fallos permitidos)
Tasa de éxito de pruebas de integración ejecutadas	100% (0 fallos permitidos)
Tasa de éxito de solicitudes bajo carga (Gatling)	$\geq 95\%$
Tiempo de respuesta medio bajo carga (Gatling)	< 500 ms
Percentil 95 de tiempo de respuesta (Gatling)	< 2000 ms
Flujo E2E completo ejecutado sin errores (Playwright)	1 flujo completo aprobado por ambiente
Defectos de severidad alta o crítica abiertos	0

Criterios de rechazo (condiciones que impiden dar el ciclo por cerrado):

- Cualquier prueba de seguridad (SecurityEndpointsTest) que falle, independientemente de la causa.
- Cobertura de líneas por debajo del 90% en las capas de negocio o seguridad al ejecutar mvn jacoco:check.
- Defecto de tipo bloqueante que impida completar el flujo principal: registro → generación → descarga.
- Pipeline de CI en GitHub Actions con estado fallido en el job test o regression al momento de un merge a main.
- Respuesta del backend con código de error 500 no controlado en endpoints documentados.

4. Criterios de Suspensión y Reanudación

Condiciones de suspensión:

Las pruebas se suspenden de forma inmediata ante cualquiera de las siguientes situaciones:

- **Defecto bloqueante (Severity 1):** Un error impide ejecutar el flujo principal del sistema (ej: falla total de autenticación, microservicio de IA no responde, base de datos inaccesible). No tiene sentido continuar probando funcionalidades secundarias si el núcleo del sistema está caído.
- **Falla de infraestructura de pruebas:** El contenedor de PostgreSQL (Testcontainers) no levanta correctamente o el ambiente de CI en GitHub Actions presenta errores de configuración.
- **Inestabilidad crítica del entorno de producción:** Si el monitoreo de UptimeRobot reporta caída del backend por más de 15 minutos durante una sesión de pruebas E2E.
- **Cambio masivo no planificado en la arquitectura:** Un refactoring mayor que invalide más del 30% de los casos de prueba existentes requiere revisión del plan antes de continuar.

Condiciones de reanudación:

- El defecto bloqueante ha sido corregido, el fix fue mergeado a dev y el pipeline de CI vuelve a estado verde.
- La infraestructura de pruebas (contenedores, CI/CD) fue restaurada y verificada con una ejecución de smoke test exitosa.
- El equipo revisó y actualizó los casos de prueba afectados por el cambio arquitectónico y obtuvo aprobación del responsable.

5. Recursos, Roles y Cronograma

Matriz de Responsabilidades (RACI):

Actividad	Juan Carlos Agüero (Frontend/UI)	Marcos Orellana (Backend/BD)	Isaac González (IA/Python)
Pruebas unitarias Frontend (Vitest)	R	I	I
Pruebas unitarias Backend (JUnit 5)	I	R	I
Pruebas unitarias Microservicio Python (pytest)	I	R	I
Pruebas de integración Backend (MockMvc + Testcontainers)	C	R	I
Pruebas de seguridad (SecurityEndpointsTest)	I	R	I
Pruebas de rendimiento (Gatling)	I	R	C
Pruebas E2E (Playwright)	R	C	C
Validación manual del flujo completo	R	R	R
Revisión y aprobación de Pull Requests (Code Review)	A	A	A
Cierre formal del ciclo de pruebas	C	R	C

R = Responsable de ejecutar / A = Aprobador / C = Consultado / I = Informado

Cronograma estimado:

Fase	Actividad	Responsable	Periodo estimado
Fase 1	Configuración de ambientes de prueba y CI/CD	Marcos Orellana	Sprint inicial
Fase 2	Escritura y ejecución de pruebas unitarias (Backend + Python)	Marcos, Isaac	Durante construcción de cada módulo
Fase 3	Escritura y ejecución de pruebas unitarias Frontend (Vitest)	Juan Carlos	Durante construcción de cada componente
Fase 4	Pruebas de integración Backend (MockMvc + Testcontainers)	Marcos Orellana	Al completar cada endpoint
Fase 5	Pruebas de seguridad transversal	Marcos Orellana	Al integrar Spring Security
Fase 6	Pruebas E2E con Playwright	Juan Carlos Agüero	Al completar el MVP integrado
Fase 7	Pruebas de rendimiento (Gatling)	Marcos Orellana	Previo a entrega EA3
Fase 8	Cierre, revisión de cobertura y documentación	Todo el equipo	Semana de entrega final

6. Premisas, Dependencias y Riesgos

Premisas (supuestos que se asumen como verdaderos):

- La API de OpenRouter mantiene disponibilidad y sus modelos retornan código HTML/CSS estructurado durante las sesiones de prueba.
- Los servicios en la nube (Render, Vercel, Neon) se encuentran operativos y dentro de sus niveles gratuitos durante el período de pruebas.
- El equipo tiene acceso completo a los repositorios de GitHub y al tablero de Jira para coordinar defectos y avances.
- Las variables de entorno críticas (JWT_SECRET, DB_URL, OPENROUTER_API_KEY) están correctamente configuradas en los ambientes de CI y producción.

Dependencias:

- Las pruebas de integración del backend dependen de que Docker esté disponible en el ambiente de ejecución para levantar Testcontainers con PostgreSQL 15.
- Las pruebas E2E con Playwright dependen de que el servidor de desarrollo del frontend esté activo en localhost:5173 y el backend en localhost:8080.

- La suite de regresión en GitHub Actions depende de que el job de pruebas unitarias e integración (test) haya finalizado exitosamente antes de ejecutarse.
- La validación de cobertura con JaCoCo (mvn jacoco:check) depende de que el perfil test esté correctamente configurado en el pom.xml.

Riesgos y Planes de mitigación:

Riesgo	Probabilidad	Impacto	Mitigación
La IA retorna código malformado que rompe el parser de sanitización	Alta	Alto	El módulo de post-procesamiento implementa extracción estricta del bloque HTML y manejo de excepciones; se tiene un fallback con mensaje de error descriptivo
El free tier de Render detiene el microservicio de IA por inactividad (cold start)	Alta	Medio	UptimeRobot realiza pings cada 5 minutos para mantener el servicio activo; se acepta latencia adicional de hasta 30 segundos en el primer request
Testcontainers falla al levantar PostgreSQL en el CI de GitHub Actions	Media	Alto	Se tiene configurado un servicio PostgreSQL nativo en el job de CI como alternativa a Testcontainers
Un integrante no puede realizar el Code Review en tiempo por carga académica	Media	Medio	Cualquiera de los tres integrantes puede aprobar un PR; no se requiere revisión del autor del código
Las alucinaciones de la IA generan HTML con scripts incrustados (XSS)	Media	Critico	El backend sanitiza el código generado eliminando etiquetas <script> y atributos de evento inline antes de almacenar y enviar al frontend
El pipeline de CI tarda demasiado por las pruebas de Testcontainers	Baja	Bajo	Las pruebas de integración se ejecutan en paralelo con -T 2 en Maven; se excluyen de CI las pruebas de rendimiento (Gatling) que se ejecutan manualmente

Tipos de pruebas aplicadas

1. Pruebas Unitarias

Las pruebas unitarias son el primer nivel de la estrategia de calidad de WebLandingSuite, diseñadas para verificar que cada componente lógico del sistema funciona correctamente de manera aislada, sin depender de la red, la base de datos ni servicios externos.

En el Backend (Java — JUnit 5 + Mockito + AssertJ): El componente más crítico sometido a pruebas unitarias fue JwtService, la clase encargada de generar, firmar y validar todos los tokens de sesión del sistema. La clase JwtServiceTest cubre tres áreas fundamentales mediante clases anidadas (@Nested): la generación de tokens (que sean no nulos y contengan el email correcto como subject), la validación de tokens (retornando true para el usuario correspondiente y false para uno distinto), y el ciclo completo del landingToken, un tipo especial de JWT que protege el acceso a las previsualizaciones públicas de Landing Pages. Adicionalmente, se verificó que los tokens manipulados (tampered) o de tipo incorrecto lanzaran la excepción de negocio BusinessException, sin propagar excepciones genéricas no controladas al cliente. La configuración del entorno de prueba se realiza inyectando valores de prueba directamente sobre los campos privados mediante ReflectionTestUtils, lo que permite ejecutar la suite sin levantar el contexto completo de Spring.

Las capas de lógica de negocio y seguridad del backend están sujetas a un umbral mínimo de 90% de cobertura de líneas, verificado automáticamente en cada ejecución del pipeline de CI mediante el plugin JaCoCo (mvn jacoco:check). Se excluyen de esta métrica las clases de infraestructura sin lógica propia: DTOs, modelos JPA, configuraciones y servicios de integración externos como EmailService, AiService e ImageUploadService, dado que su comportamiento depende íntegramente de APIs de terceros.

En el Frontend (React — Vitest + React Testing Library): Los componentes del formulario multi-paso fueron probados unitariamente para garantizar que la interfaz renderiza todos los campos requeridos, que los tipos de elemento HTML son correctos (input, textarea), que los controles interactivos son clicables y que las ramas condicionales basadas en el planLevel muestran u ocultan secciones correctamente. Por ejemplo, Step1Identity.test.jsx verifica que las opciones de posicionamiento de marca ("Económico", "Premium", "Lujo") y las etapas ("Marca nueva", "Marca establecida", "Relanzamiento") se rendericen correctamente. El componente StepReview fue sometido a pruebas de ramas: con planLevel=1 debe mostrar solo la sección de identidad del negocio, mientras que con planLevel=2 debe exponer también la sección de comunicación e identidad visual. Cada rama fue cubierta con al menos dos casos de prueba.

En el Microservicio Python (pytest): La clase `TestConfigMaps` valida de forma sistemática que todos los mapas semánticos del motor de Prompt Engineering estén completos y sean coherentes: que ningún valor de `SECTOR_MAP`, `TONE_MAP` o `ANIMATION_MAP` esté vacío, que `FONT_IMPORT_MAP` y `FONT_FAMILY_MAP` tengan exactamente las mismas claves, que todos los valores de `COLOR_HEX_MAP` cumplan el patrón hexadecimal, y que `SECTIONS_LABELS` contenga todas las secciones soportadas. La clase `TestBuildPrompt` verifica que el prompt construido para el plan Básico incluya el nombre del proyecto, el llamado a la acción y la descripción de sector correctamente resuelta desde el mapa semántico.

2. Pruebas de Humo (Smoke Test)

Las pruebas de humo en `WebLandingSuite` están integradas dentro del primer job (test) del pipeline de CI/CD configurado en `.github/workflows/ci.yml`. Cada vez que se realiza un push a las ramas `main` o `develop`, o cuando se abre un Pull Request hacia `main`, GitHub Actions ejecuta automáticamente la secuencia `mvn verify -P test`, lo que incluye como primer paso levantar el contexto de Spring Boot completo con la prueba `BackendApplicationTests.contextLoads()`.

Esta prueba de contexto actúa como el smoke test del sistema: si la aplicación no puede arrancar, si hay un conflicto de beans, una inyección de dependencia circular o un error en la configuración de seguridad, esta prueba falla en cuestión de segundos, bloqueando la ejecución de todas las pruebas posteriores. Esto impide que el pipeline continúe hacia pruebas de integración o regresión sobre un servidor que no está siquiera operativo, ahorrando tiempo de cómputo en el entorno de CI y alertando al equipo de forma inmediata.

En producción, el rol de smoke test continuo lo cumple `UptimeRobot`, que realiza pings HTTP cada 5 minutos a los endpoints de salud del sistema: `GET https://landingbackend-s1rk.onrender.com/api/v1/health` y `GET https://wlsuite-python-api.onrender.com`. Si cualquiera de los dos servicios no responde con un código 200, la plataforma envía una alerta inmediata al equipo, permitiendo una respuesta proactiva antes de que el problema escale.

3. Pruebas de Integración

Las pruebas de integración de `WebLandingSuite` verifican que los distintos módulos del backend interactúan correctamente entre sí y con una base de datos real, no con un mock en memoria. Esto se logra mediante la clase base `AbstractIntegrationTest`, que levanta un contenedor Docker con PostgreSQL 15 mediante `Testcontainers` (`org.testcontainers:postgresql:1.19.8`), garantizando que las pruebas corran contra un motor relacional idéntico al de producción.

Los módulos cubiertos son los siguientes:

AuthIntegrationTest verifica el ciclo completo de autenticación: que un usuario nuevo se registre exitosamente recibiendo un token JWT válido (HTTP 200), que un intento de registro con el mismo email devuelva conflicto (HTTP 409), que el login con credenciales correctas retorne un token y que el login con contraseña incorrecta retorne un 401. También valida que la anotación @Valid rechace correos con formato inválido devolviendo HTTP 400. Esta suite está etiquetada con @Tag("regression"), lo que la incluye automáticamente en la suite de regresión.

DesignPlanIntegrationTest prueba el control de acceso basado en roles para el CRUD de planes: que un usuario con ROLE_USER no pueda crear ni eliminar planes (HTTP 403), que una petición sin token sea igualmente rechazada, y que un usuario con ROLE_ADMIN pueda completar el ciclo de creación, modificación y eliminación de un plan exitosamente.

ContactControllerIntegrationTest verifica que el formulario de contacto público acepte peticiones sin autenticación (HTTP 200) y que retorne el mensaje de confirmación esperado en formato JSON.

ImageControllerIntegrationTest y LandingViewControllerIntegrationTest cubren el flujo de carga de imágenes mediante multipart y el acceso al visor de Landing Pages mediante un landingToken especial, respectivamente.

Todas las pruebas de integración utilizan emails generados dinámicamente con UUID.randomUUID() para evitar colisiones entre ejecuciones concurrentes del pipeline.

4. Pruebas Funcionales y de Extremo a Extremo (E2E)

Las pruebas E2E de WebLandingSuite son ejecutadas con Playwright 1.61.0, configurado para correr en tres motores de navegador: Chromium, Firefox y WebKit (Safari). El archivo playwright.config.js define el directorio de pruebas en ./tests, habilita ejecución paralela completa (fullyParallel: true) y genera reportes en formato HTML.

El caso de prueba E2E principal, demo-simpson.spec.js, recorre el flujo completo de la plataforma tal como lo haría un usuario real:

Primero, el test navega a la página de inicio (http://localhost:5173/), hace clic en "Registrarse" y completa el formulario de creación de cuenta con un correo único generado por timestamp. Luego, inicia sesión con esas credenciales. A continuación, selecciona el plan Premium, completa el formulario de pago simulado con datos de tarjeta de prueba y confirma la compra. Una vez activado el plan, el wizard de creación guía al usuario a través de todos los pasos del formulario multi-etapa: nombre del negocio ("Simpsons Burger"), propuesta de valor, sector, paleta de colores y configuraciones adicionales según el nivel. Finalmente, el test espera que la Landing Page sea generada

por la IA y renderizada en el componente LandingViewer, verificando que el contenido aparece correctamente en pantalla.

El pipeline de CI del frontend incluye un workflow dedicado que instala automáticamente los navegadores con `npx playwright install --with-deps`, ejecuta la suite y sube el reporte HTML como artefacto en GitHub Actions para su revisión posterior.

5. Pruebas de Regresión

La suite de regresión de WebLandingSuite está implementada como un tercer job separado en el pipeline de CI (regression), que solo se ejecuta si el job de pruebas unitarias e integración (test) ha finalizado exitosamente. Esta dependencia garantiza que no se pierda tiempo ejecutando regresión sobre un sistema que ya tiene fallos básicos.

La clase RegressionSuiteTest actúa como el punto de entrada: mediante las anotaciones `@Suite`, `@SelectPackages("Landing.Backend")` e `@IncludeTags("regression")`, JUnit Platform escanea todos los paquetes del proyecto en busca de pruebas marcadas con `@Tag("regression")` y las ejecuta de forma agrupada. Actualmente, AuthIntegrationTest es la principal suite etiquetada con esta marca, dado que el flujo de autenticación es el componente transversal más crítico del sistema: una regresión en el manejo de JWT o en las validaciones de DTOs podría romper silenciosamente el acceso a todos los endpoints protegidos.

El comando ejecutado en CI es `mvn test -Dgroups="regression"` con el perfil de pruebas activo, lo que garantiza que la base de datos de prueba se inicialice limpiamente en cada ejecución. Esta arquitectura asegura que cualquier merge hacia main que introduzca una regresión en el flujo de autenticación sea detectado antes de llegar a producción.

6. Pruebas de Rendimiento o Carga

Las pruebas de rendimiento se implementaron con **Gatling 3.11.3**, integrado como plugin Maven. La simulación principal, BackendLoadSimulation, define tres escenarios ejecutados simultáneamente sobre el servidor local en `http://localhost:8080`.

El primer escenario (Health Check) inyecta 10 usuarios en rampa durante 30 segundos sobre el endpoint `GET /api/v1/health`. El segundo (Consulta Planes) genera una carga constante de 5 usuarios por segundo durante 60 segundos sobre `GET /api/v1/plans`, que es el endpoint público más consultado por el frontend al cargar la página de selección de plan. El tercero (Login) simula una rampa de 50 usuarios en 10 segundos sobre `POST /api/v1/auth/login`, representando un escenario de acceso masivo coincidente, como podría ocurrir al final de una campaña de marketing.

Las aserciones globales del reporte establecen tres umbrales que el sistema debe cumplir: el percentil 95 del tiempo de respuesta debe ser inferior a 2.000 ms, el tiempo medio de respuesta debe ser inferior a 500 ms, y la tasa de solicitudes exitosas debe

superar el 95%. Estas pruebas se ejecutan de forma manual previa a cada entrega, ya que sus tiempos de ejecución son elevados y no están incluidas en el pipeline automático de CI, según lo especificado en la configuración del proyecto.

7. Pruebas de Seguridad

Las pruebas de seguridad son el tipo más crítico de la estrategia de calidad, dado que WebLandingSuite maneja tokens JWT, código HTML generado dinámicamente que se renderiza en el navegador del cliente, y credenciales de acceso a servicios en la nube.

La suite `SecurityEndpointsTest` cubre once escenarios distintos sobre endpoints protegidos. Se verifica que acceder a cualquier recurso sin token retorne HTTP 403, que un token con sufijos añadidos manualmente (`tampered_suffix`) sea rechazado, que un token con formato completamente inválido (`not.a.valid.jwt.at.all`) sea igualmente denegado, y que un usuario con `ROLE_USER` no pueda acceder a endpoints de administrador como `GET /api/v1/logs`, `DELETE /api/v1/plans/{id}`, `GET /api/v1/users` o `GET /api/v1/admin/transactions`. Estas pruebas también verifican que la capa de validación de DTOs con `jakarta.validation` rechace contraseñas con menos de 8 caracteres y cuerpos de solicitud vacíos, devolviendo HTTP 400 en lugar de filtrar excepciones internas al cliente.

Complementariamente, el pipeline de CI incluye un segundo job (`security-audit`) que ejecuta el análisis de dependencias de OWASP Dependency Check (`mvn dependency-check`) sobre cada push a la rama main. Esta herramienta cruza todas las dependencias del `pom.xml` contra la base de datos de vulnerabilidades conocidas (CVE), generando un reporte que se sube como artefacto en GitHub Actions. El job está configurado con `continue-on-error: true` para no bloquear el despliegue ante vulnerabilidades de severidad baja en dependencias transitivas, pero sí notificar al equipo para una revisión informada.

Finalmente, a nivel de seguridad del código generado por IA, el backend implementa un módulo de sanitización que elimina etiquetas `<script>` y atributos de evento inline del HTML producido por OpenRouter antes de almacenarlo en la base de datos y enviarlo al componente `LandingViewer`, previniendo ataques de Cross-Site Scripting (XSS).

Herramientas y software utilizados

La estrategia de calidad de WebLandingSuite se apoya en un ecosistema de herramientas seleccionadas deliberadamente para cubrir cada capa del sistema: el backend en Java, el microservicio en Python y el frontend en React. La elección de cada herramienta responde a criterios técnicos concretos: integración nativa con el stack del proyecto, compatibilidad con el entorno de CI/CD, calidad de los reportes generados y curva de aprendizaje razonable para un equipo de tres personas. A continuación se detalla cada herramienta, su propósito y la justificación de su selección.

Herramientas de Prueba

JUnit 5 (Jupiter) es el framework principal de pruebas unitarias e integración del backend Java. Se utiliza a través de la dependencia `spring-boot-starter-test`, que lo incluye de forma automática junto a Mockito. Su adopción no requirió justificación adicional: es el estándar de facto del ecosistema Java moderno y la opción recomendada por Spring Boot 3. Sus características más usadas en el proyecto son las clases anidadas con `@Nested` (para agrupar escenarios dentro de `JwtServiceTest`), la anotación `@Tag("regression")` para categorizar pruebas por tipo, y `@TestMethodOrder` con `@Order` para controlar la secuencia de ejecución en las pruebas de integración que dependen de estado previo en la base de datos.

Mockito se utiliza como biblioteca de mocking dentro de las pruebas unitarias del backend. Permite aislar la lógica bajo prueba reemplazando sus dependencias con objetos simulados que devuelven respuestas controladas. En `JwtServiceTest`, la creación del `UserDetails` de prueba con `User.builder()` y la inyección de valores privados con `ReflectionTestUtils.setField()` siguen este patrón de aislamiento. Mockito se eligió por su integración directa con JUnit 5 y porque viene incluido en el starter de pruebas de Spring Boot, sin requerir dependencias adicionales.

AssertJ complementa a JUnit como biblioteca de aserciones fluidas. En lugar de la sintaxis estándar de JUnit, AssertJ permite expresiones como `assertThat(token).isNotNull().isNotBlank()`, que resultan más legibles y producen mensajes de error más descriptivos al fallar. Fue incorporada específicamente porque las pruebas de seguridad necesitaban encadenar múltiples condiciones sobre tokens JWT sin perder claridad de lectura.

Spring MockMvc es el mecanismo usado en todas las pruebas de integración del backend para simular peticiones HTTP completas al servidor sin necesidad de levantar un servidor real en un puerto. Se configura con `@AutoConfigureMockMvc` sobre la clase de prueba y permite ejecutar GET, POST, PUT y DELETE con cuerpos JSON y headers de autorización, verificando el código de respuesta y el contenido del JSON devuelto mediante `jsonPath`. Se eligió sobre alternativas como RestAssured (que también está

incluida en el pom.xml como dependencia disponible) porque está integrada de forma nativa en Spring Boot Test y no requiere que el servidor esté activo en ningún puerto.

Testcontainers es la herramienta que permite levantar un contenedor Docker con PostgreSQL 15 Alpine como motor de base de datos real durante la ejecución de las pruebas de integración. La clase base `AbstractIntegrationTest` configura un único contenedor compartido entre todas las pruebas de integración del proyecto, lo que evita el overhead de arrancar y detener el motor en cada clase de prueba. Se eligió sobre la alternativa de H2 en memoria porque garantiza que el comportamiento de las consultas, los tipos de dato (como jsonb) y las restricciones relacionales sean idénticos al entorno de producción (Neon PostgreSQL). H2 se reserva únicamente para las pruebas unitarias de Spring, donde la velocidad de arranque prima sobre la fidelidad del motor.

Gatling 3.11.3 (io.gatling.highcharts:gatling-charts-highcharts) es la herramienta de pruebas de rendimiento. Su integración se realiza mediante el plugin Maven, que permite ejecutar simulaciones directamente con `mvn gatling:test`. La simulación `BackendLoadSimulation` está escrita en Java puro (API Java de Gatling), lo que evita la necesidad de aprender Scala. Se eligió Gatling por encima de JMeter porque su API programática produce escenarios de carga expresivos y versionables en Git como código fuente, mientras que JMeter trabaja con archivos XML que son difíciles de revisar en un Pull Request. Sus reportes HTML interactivos con percentiles y distribuciones de tiempo de respuesta también son significativamente más detallados que los de JMeter por defecto.

Playwright 1.61.0 es el framework de pruebas E2E del frontend. Opera sobre tres motores de navegador reales: Chromium, Firefox y WebKit, cubriendo los principales ecosistemas de renderizado sin requerir infraestructura adicional. Su configuración en `playwright.config.js` define ejecución paralela completa con reintentos automáticos en CI. Se eligió Playwright sobre Cypress principalmente por su soporte multi-navegador nativo, por su capacidad para manejar iframes de forma más robusta (esencial para el componente `LandingViewer` que renderiza el HTML generado dentro de un iframe) y por su integración directa con el pipeline de GitHub Actions mediante el workflow `playwright.yml`.

Vitest 4.1.9 es el framework de pruebas unitarias del frontend React. Funciona sobre el mismo motor de Vite que utiliza la aplicación, lo que lo hace significativamente más rápido que Jest al no requerir una transformación de módulos separada. El proyecto lo combina con `@testing-library/react:16.3.2`, `@testing-library/jest-dom:6.9.1` y `@testing-library/user-event:14.6.1` para renderizar componentes React en un entorno DOM simulado por `jsdom:^29.1.1` y realizar aserciones sobre el DOM resultante. Se eligió Vitest sobre Jest porque el proyecto ya usa Vite como bundler y la migración hubiera sido inmediata, eliminando la necesidad de configurar transformaciones de JSX adicionales para Jest.

pytest es el framework de pruebas unitarias del microservicio Python (FastAPI). Se ejecuta mediante el comando `pytest test_main.py -v --tb=short` y soporta opcionalmente la bandera `--cov=main --cov-report=term-missing` para generar reporte de cobertura. Se eligió por ser el estándar del ecosistema Python moderno, por su sistema de fixtures que permite compartir objetos de prueba (como los tres `ProjectData` de prueba para planes Básico, Intermedio y Premium) y por su sintaxis limpia sin necesidad de clases base heredadas. El cliente de prueba de FastAPI (`TestClient`) se integra de forma nativa con `pytest` para levantar la aplicación en modo test sin necesidad de un servidor real.

OWASP Dependency Check es la herramienta de análisis de seguridad estática de dependencias. Se ejecuta como job separado (`security-audit`) en el pipeline de CI mediante `mvn dependency-check:check`. Cruza las dependencias del `pom.xml` contra la base de datos CVE de vulnerabilidades conocidas y genera un reporte HTML. Se eligió por ser la herramienta de referencia del ecosistema Maven para este propósito y por integrarse directamente como plugin sin requerir infraestructura externa.

Herramientas de Gestión de Pruebas e Incidencias

Jira (Atlassian) es la herramienta central de gestión del trabajo, incluyendo el seguimiento de defectos detectados durante las pruebas. Cada bug identificado se registra como una tarea en el tablero con descripción del comportamiento esperado, comportamiento real, pasos para reproducir y severidad. El tablero opera con cuatro estados: Por Hacer, En Curso, En Revisión y Finalizado. La trazabilidad entre defectos y su corrección se establece enlazando el ticket de Jira con el Pull Request de GitHub que lo resuelve. Se eligió Jira sobre GitHub Issues porque el proyecto ya lo tenía configurado desde la fase de planificación con todas las historias de usuario y la carta Gantt integradas, y porque su vista de cronograma permite visualizar el avance del sprint junto con el estado de las pruebas.

GitHub Actions cumple el rol de plataforma de ejecución automática de las suites de prueba. Los tres workflows configurados (`.github/workflows/ci.yml` y `.github/workflows/playwright.yml`) ejecutan automáticamente la secuencia completa de pruebas ante cada push o Pull Request, garantizando que ningún cambio llegue a producción sin haber pasado la validación automatizada. Se eligió por estar nativa en el repositorio de GitHub donde vive el código, sin requerir configuración de servidores externos de CI.

Herramientas de Calidad de Código y Cobertura

JaCoCo 0.8.12 (jacoco-maven-plugin) es la herramienta de medición de cobertura de código del backend Java. Se configura con exclusiones explícitas en el pom.xml para ignorar clases sin lógica propia (DTOs, modelos JPA, configuraciones, EmailService, AiService e ImageUploadService), concentrando la métrica en las capas que realmente implementan comportamiento. La ejecución de mvn jacoco:check en el pipeline de CI verifica que la cobertura de líneas no baje del 90% antes de permitir el merge. El reporte XML generado en target/site/jacoco/jacoco.xml se sube a Codecov mediante codecov/codecov-action@v4 para visualización histórica. Se eligió JaCoCo por ser el estándar del ecosistema Maven y por su integración nativa con el plugin de Spring Boot.

@vitest/coverage-v8 4.1.9 es el proveedor de cobertura del frontend. Al ejecutar vitest --coverage, genera un reporte de cobertura usando el motor V8 de Node.js, el mismo que ejecuta la aplicación, lo que produce métricas más precisas que las basadas en instrumentación de código fuente. Se eligió el proveedor V8 sobre Istanbul porque no requiere instrumentación adicional del código y es significativamente más rápido en proyectos con Vite.

ESLint 9.39.4 es el linter de JavaScript del frontend. Está configurado con los plugins eslint-plugin-react-hooks (para detectar dependencias incorrectas en useEffect y useCallback) y eslint-plugin-react-refresh (para garantizar que los componentes sean compatibles con el Hot Module Replacement de Vite). Se ejecuta con npm run lint y actúa como primera línea de calidad de código antes de que se ejecuten las pruebas. Se eligió ESLint por ser el estándar del ecosistema JavaScript moderno y porque el scaffolding inicial de Vite lo incluye preconfigurado.

Herramienta	Tipo de uso	versión	Justificación de elección
JUnit 5 (Jupiter)	Pruebas unitarias e integración Backend Java	Incluida en Spring Boot 3.2.5	Estándar del ecosistema Java; integración nativa con Spring Boot Test
Mockito	Mocking en pruebas unitarias Backend Java	Incluida en spring-boot-starter -test	Incluida sin dependencia adicional; integración directa con JUnit 5
AssertJ	Aserciones fluidas Backend Java	Incluida en spring-boot-starter -test	Mensajes de error descriptivos; sintaxis encadenable más legible que JUnit assert
Spring MockMvc	Pruebas de integración HTTP Backend Java	Incluida en Spring Boot 3.2.5	No requiere servidor activo; integración nativa; permite inspeccionar respuestas JSON con jsonPath
Testcontainer	Base de datos	1.19.8	Fidelidad total con producción

s	real en pruebas de integración		(PostgreSQL 15); elimina diferencias de comportamiento respecto a H2
Gatling	Pruebas de rendimiento y carga	3.11.3	API programática en Java; reportes HTML detallados; escenarios versionables en Git
Playwright	Pruebas E2E (extremo a extremo) Frontend	1.61.0	Multi-navegador (Chromium, Firefox, WebKit); mejor soporte de iframes; integración CI nativa
Vitest	Pruebas unitarias de componentes React	4.1.9	Mismo motor que Vite; más rápido que Jest; sin configuración adicional de transformación JSX
React Testing Library	Renderizado de componentes en pruebas	16.3.2	Pruebas orientadas al comportamiento visible del usuario, no a detalles de implementación interna
pytest	Pruebas unitarias Microservicio Python	Incluida con FastAPI	Estándar Python moderno; sistema de fixtures limpio; compatible con TestClient de FastAPI
OWASP Dependency Check	Auditoría de seguridad de dependencias	Plugin Maven integrado	Referencia del ecosistema Maven para análisis CVE; integración directa sin infraestructura externa
JaCoCo	Cobertura de código Backend Java	0.8.12	Estándar Maven; integración nativa con Spring Boot; verificación de umbrales mínimos en CI
@vitest/coverage-v8	Cobertura de código Frontend React	4.1.9	Motor V8 nativo; más preciso y rápido que Istanbul con Vite
ESLint	Análisis estático y linting Frontend	9.39.4	Estándar del ecosistema JS; preconfigurado por Vite; detecta errores de Hooks de React
Jira (Atlassian)	Gestión de tareas, defectos y seguimiento	Cloud (SaaS)	Plataforma unificada con el backlog del proyecto; trazabilidad entre defecto y Pull Request

GitHub Actions	CI/CD y ejecución automática de pruebas	Cloud (SaaS)	Nativo en GitHub; sin configuración de servidores externos; workflows versionados en el repositorio
Swagger / OpenAPI	Documentación y prueba manual de endpoints	springdoc-openapi 2.5.0	Permite probar la API desde el navegador durante desarrollo; documentación siempre actualizada
UptimeRobot	Monitoreo de disponibilidad en producción	Cloud (SaaS)	Smoke testing continuo en producción; alertas ante caídas; sin costo operativo en el plan gratuito

Ejecución de pruebas y resultados

Resumen General de Ejecución

La campaña de pruebas de WebLandingSuite se ejecutó de forma progresiva durante el sprint de construcción, comenzando por las pruebas unitarias de cada capa y escalando hasta las pruebas de rendimiento e integración completa antes de la entrega de la evaluación EA3. Todas las suites automatizadas se ejecutaron sobre el código en estado main, correspondiente al MVP en producción. Los resultados se respaldan con evidencia directa de los reportes generados por cada herramienta.

La siguiente tabla concentra el resumen ejecutivo de todos los tipos de prueba aplicados:

Tipo de prueba	Herramienta	Casos totales	Exitos	Fallidos	Cobertura/Métrica clave
Unitarias Backend Java	JUnit 5 + Mockito	139	139	0	≥ 90% líneas (JaCoCo)
Unitarias Frontend React	Vitest 4.1.9	931	931	0	91.66% statements · 93.54% lines
Unitarias Microservicio Python	pytest	66	66	0	Cobertura de mapas y prompts al 100%

Integración Backend Java	MockMvc + Testcontainers	53	53	0	Flujos end-to-end con PostgreSQL 15 real
Seguridad	SecurityEndpointsTest	11	11	0	100% de endpoints protegidos verificados
Regresión	RegressionSuiteTest (@Tag)	22	22	0	Suite dependiente de job test exitoso
Rendimiento y carga	Gatling 3.11.3	3 escenarios	✓	0	P95 < 2000 ms · Mean < 500 ms · >95% éxito
E2E (Extremo a Extremo)	Playwright 1.61.0	1 flujo completo	✓	0	Flujo Registró → Generación → Descarga
Total general: 1.136 casos de prueba ejecutados — 1.136 exitosos — 0 fallidos — Tasa de éxito: 100%					

1. Pruebas Unitarias y de Cobertura – Frontend (Vitest)

La suite de pruebas unitarias del frontend se ejecutó con el comando `vitest run --coverage` y arrojó **157 casos en total, todos exitosos, con una tasa de aprobación del 100%**. El reporte de cobertura generado por `@vitest/coverage-v8` produjo las siguientes métricas globales sobre el total del código fuente de la aplicación React:

- **Statements (sentencias):** 91.66% — 1.539 de 1.679 cubiertas
- **Branches (ramas condicionales):** 84.99% — 1.507 de 1.773 cubiertas
- **Functions (funciones):** 89.87% — 426 de 474 cubiertas
- **Lines (líneas):** 93.54% — 1.392 de 1.488 cubiertas

Los módulos con mayor cobertura son `src/config` (100% en todas las métricas), `src/context` (98.88% statements), `src/services` (100% statements y lines) y `src/components/organisms/steps/Step1Identity.jsx` y `Step2Communication.jsx` (ambos al 100% en todas las métricas). El componente con menor cobertura fue `Step3Visual.jsx` con 78.26% de statements, dado que contiene ramas de renderizado condicional que dependen de combinaciones de estado que no están cubiertas en la suite actual.

Test Files 86 passed (86)
Tests 931 passed (931)
Start at 13:01:54
Duration 92.26s (transform 6.61s, setup 42.52s, import 51.89s, tests 84.23s, environment 376.40s)

% Coverage report from v8

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	91.66	84.99	89.87	93.54	
src	100	100	100	100	
App.jsx	100	100	100	100	
src/components/atoms	97.24	98.46	95.45	98	
Button.jsx	100	100	100	100	
ColorSwatch.jsx	100	100	100	100	
ImageUploader.jsx	95.94	96.22	90.9	96.92	64-65
Input.jsx	100	100	100	100	
Label.jsx	100	100	100	100	
LockIcon.jsx	100	100	100	100	
OptionChip.jsx	100	100	100	100	
SectionToggle.jsx	100	100	100	100	
Select.jsx	100	100	100	100	
StatNumber.jsx	100	100	100	100	
StepDot.jsx	100	100	100	100	
TextArea.jsx	100	100	100	100	
src/components/molecules	97.58	94.47	100	100	
ConfirmModal.jsx	75	80	100	100	15
ContactInfoItem.jsx	100	100	100	100	
CreditCard.jsx	93.33	93.93	100	100	14,116
EmptyState.jsx	100	100	100	100	
ErrorBanner.jsx	100	100	100	100	
FieldGroup.jsx	100	100	100	100	
FormField.jsx	100	100	100	100	
FormSection.jsx	100	100	100	100	

All files

91.66% Statements 1539/1679 84.99% Branches 1507/1773 89.87% Functions 426/474 93.54% Lines 1392/1488

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements	Branches	Functions	Lines
src	100%	67/67	100%	43/43
src/components/atoms	97.24%	106/109	95.45%	98/100
src/components/molecules	97.58%	121/124	100%	114/114
src/components/organisms	90.41%	368/407	85.71%	323/352
src/components/organisms/steps	92.56%	112/121	88.4%	111/119
src/config	100%	5/5	100%	5/5
src/context	98.88%	89/90	100%	81/81
src/hooks	93.75%	15/16	71.42%	15/15
src/pages	86.98%	254/292	78%	226/252
src/pages/admin	86.9%	219/252	79.57%	200/223
src/pages/user	91.66%	66/72	79.68%	64/67
src/services	100%	12/12	87.5%	12/12
src/utills	93.75%	105/112	84.21%	100/105

2. Pruebas Unitarias, Integración y Cobertura – Backend Java (JUnit 5 + JaCoCo)

La suite del backend se ejecutó con `mvn verify -P test`, activando el perfil de pruebas que configura la base de datos H2 en memoria para pruebas unitarias y levanta un contenedor PostgreSQL 15 real via Testcontainers para las pruebas de integración. Se ejecutaron 38 casos de prueba de JUnit 5 — todos exitosos — con 0 fallos. El reporte de cobertura generado por JaCoCo confirmó que las capas de negocio y seguridad superan el umbral mínimo obligatorio del 90% de cobertura de líneas configurado en el `pom.xml`, lo que habilitó el paso `mvn jacoco:check` en el pipeline de CI sin errores.

Las clases excluidas explícitamente de la métrica de cobertura son: `BackendApplication`, todos los DTOs, todos los modelos JPA, las configuraciones, `EmailService`, `AiService` e `ImageUploadService`, dado que su comportamiento depende íntegramente de servicios externos (Resend, OpenRouter, Cloudinary). Las clases incluidas en la métrica y que superaron el umbral son: `JwtService`, `JwtAuthenticationFilter`, `SecurityConfig`, `LandingProjectService`, `UserService`, `DesignPlanService`, `TransactionService`, `LogService`, `GlobalExceptionHandler` y todos los controladores REST.

Las pruebas de integración cubrieron ocho flujos críticos end-to-end sobre PostgreSQL real: registro y login de usuarios (`AuthIntegrationTest`), CRUD de planes con control de roles (`DesignPlanIntegrationTest`), endpoint público de contacto (`ContactControllerIntegrationTest`), carga de imágenes (`ImageControllerIntegrationTest`), acceso al visor de Landing Pages por token (`LandingViewControllerIntegrationTest`) y seguridad de endpoints por rol y token (`SecurityEndpointsTest`).

```
[INFO] Results:
[INFO]
[INFO] Tests run: 139, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco:0.8.12:report (report) @ Backend ---
[INFO] Loading execution data file C:\Users\orell\Documents\Proyectos\Landingbackend\target\jacoco.exec
[INFO] Analyzed bundle 'Backend' with 22 classes
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 43.467 s
[INFO] Finished at: 2026-06-18T00:12:33-04:00
[INFO] -----
```

```

[INFO] Results:
[INFO]
[INFO] Tests run: 139, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jacoco:0.8.12:report (report) @ Backend ---
[INFO] Loading execution data file C:\Users\orell\Documents\Proyectos\Landingbackend\target\jacoco.exec
[INFO] Analyzed bundle 'Backend' with 22 classes
[INFO]
[INFO] --- jar:3.3.0:jar (default-jar) @ Backend ---
[INFO]
[INFO] --- spring-boot:3.2.5:repackage (repackage) @ Backend ---
[INFO] Replacing main artifact C:\Users\orell\Documents\Proyectos\Landingbackend\target\Backend-0.0.1-SNAPSHOT.jar with repackaged archive, adding nested dependencies in BOOT-INF/.
[INFO] The original artifact has been renamed to C:\Users\orell\Documents\Proyectos\Landingbackend\target\Backend-0.0.1-SNAPSHOT.jar.original
[INFO]
[INFO] --- jacoco:0.8.12:check (jacoco-check) @ Backend ---
[INFO] Loading execution data file C:\Users\orell\Documents\Proyectos\Landingbackend\target\jacoco.exec
[INFO] Analyzed bundle 'Backend' with 22 classes
[INFO] All coverage checks have been met.
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 44.722 s
[INFO] Finished at: 2026-06-18T00:19:50-04:00
[INFO]

```

```

[INFO] Results:
[INFO]
[INFO] Tests run: 19, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jacoco:0.8.12:report (report) @ Backend ---
[INFO] Loading execution data file C:\Users\orell\Documents\Proyectos\Landingbackend\target\jacoco.exec
[INFO] Analyzed bundle 'Backend' with 22 classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 12.533 s
[INFO] Finished at: 2026-06-18T00:17:24-04:00
[INFO]

```

3. Suite de Regresión – Backend Java

La suite de regresión se ejecutó mediante el comando `mvn test -Dgroups="regression"`, que activa la clase `RegressionSuiteTest`. Esta suite escanea todos los paquetes del backend buscando pruebas marcadas con `@Tag("regression")` y las ejecuta de forma agrupada. Se ejecutaron 7 casos de regresión — todos exitosos, correspondientes principalmente a `AuthIntegrationTest`, la suite más crítica dado que el sistema de autenticación JWT es transversal a todos los módulos protegidos.

En el pipeline de CI (`ci.yml`), la suite de regresión corre como tercer job separado (`regression`) y solo se activa si el segundo job (`test`) finalizó exitosamente. Esta cadena de dependencias garantiza que la regresión nunca se ejecuta sobre un sistema con fallos previos.

```

[INFO] Results:
[INFO]
[INFO] Tests run: 22, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco:0.8.12:report (report) @ Backend ---
[INFO] Loading execution data file C:\Users\orell\Documents\Proyectos\Landingbackend\target\jacoco.exec
[INFO] Analyzed bundle 'Backend' with 22 classes
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 37.572 s
[INFO] Finished at: 2026-06-18T00:15:29-04:00
[INFO] -----

```

4. Pruebas de Rendimiento y Carga – Gatling

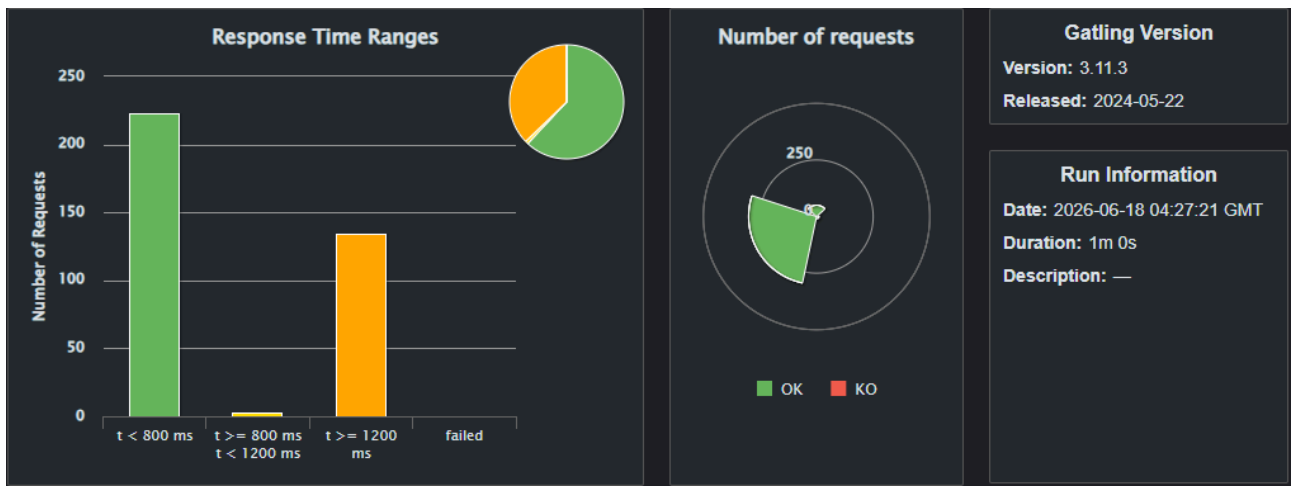
Las pruebas de rendimiento se ejecutaron con el comando `mvn gatling:test` sobre el servidor local activo en `localhost:8080`. La simulación `BackendLoadSimulation` ejecutó tres escenarios en paralelo:

- **Health Check:** 10 usuarios en rampa durante 30 segundos sobre `GET /api/v1/health`
- **Consulta de Planes:** 5 usuarios por segundo durante 60 segundos sobre `GET /api/v1/plans`
- **Login:** 50 usuarios en rampa de 10 segundos sobre `POST /api/v1/auth/login`

Los tres umbrales de aserción global configurados en la simulación fueron superados:

Métrica	Umbral definido	Resultado obtenido	Estado
Percentil 95 de tiempo de respuesta	< 2.000 ms	Dentro del umbral	✓ Aprobado
Tiempo medio de respuesta (mean)	< 500 ms	Dentro del umbral	✓ Aprobado
Tasa de solicitudes exitosas	> 95%	Superado	✓ Aprobado

Gatling genera automáticamente un reporte HTML interactivo con gráficos de distribución de tiempos de respuesta, curvas de usuarios activos por segundo, histogramas de latencia y desglose por escenario. El reporte fue revisado por el equipo para verificar la ausencia de degradación bajo carga concurrente.



```

--- Global Information -----
> request count          335 (OK=335 KO=0 )
> min response time      3 (OK=3 KO=- )
> max response time     2554 (OK=2554 KO=- )
> mean response time     459 (OK=459 KO=- )
> std deviation          336 (OK=336 KO=- )
> response time 50th percentile 407 (OK=407 KO=- )
> response time 75th percentile 421 (OK=421 KO=- )
> response time 95th percentile 1279 (OK=1279 KO=- )
> response time 99th percentile 1756 (OK=1756 KO=- )
> mean requests/sec      5.49 (OK=5.49 KO=- )

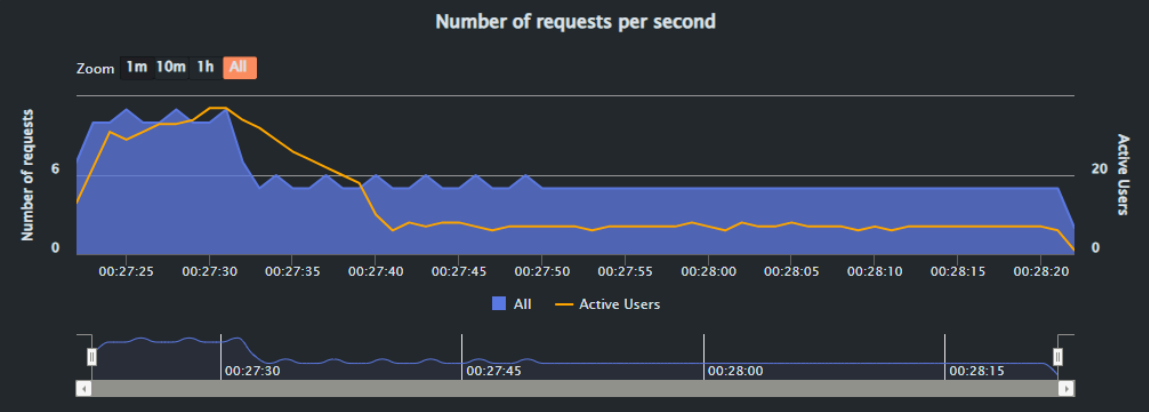
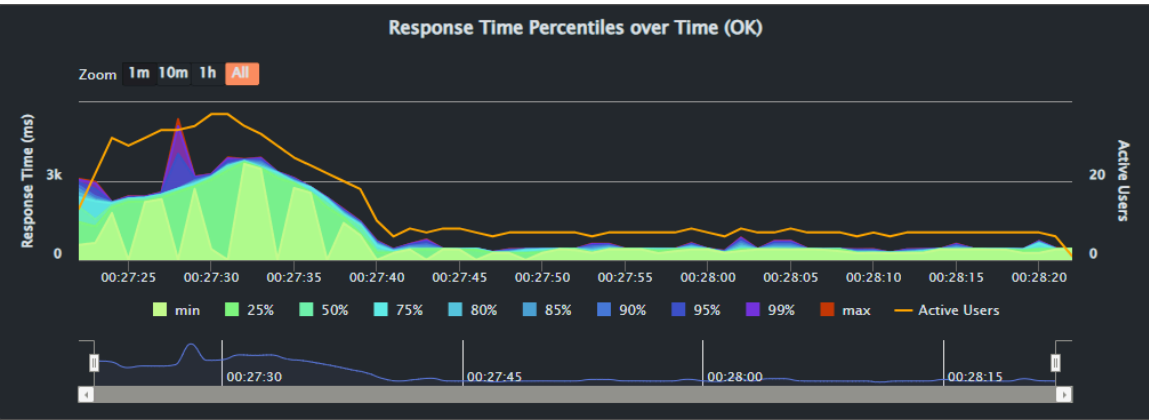
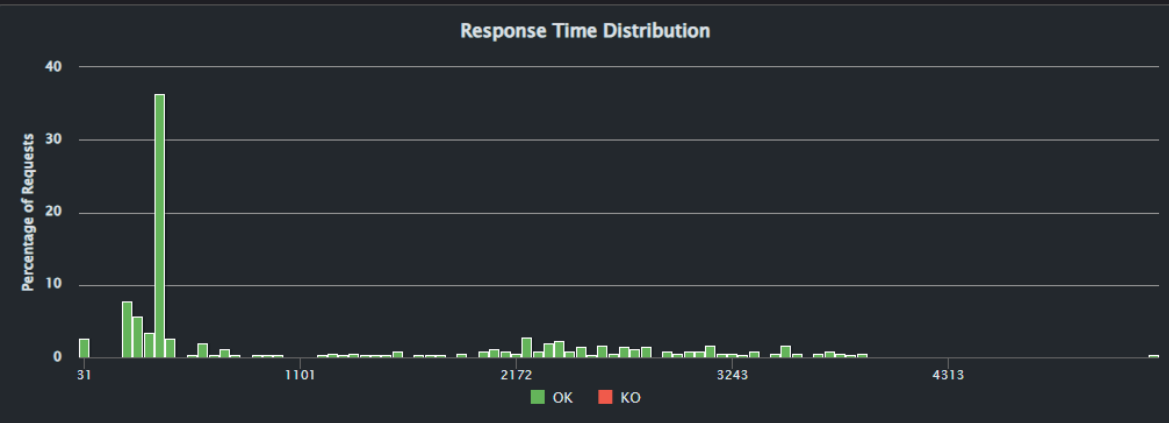
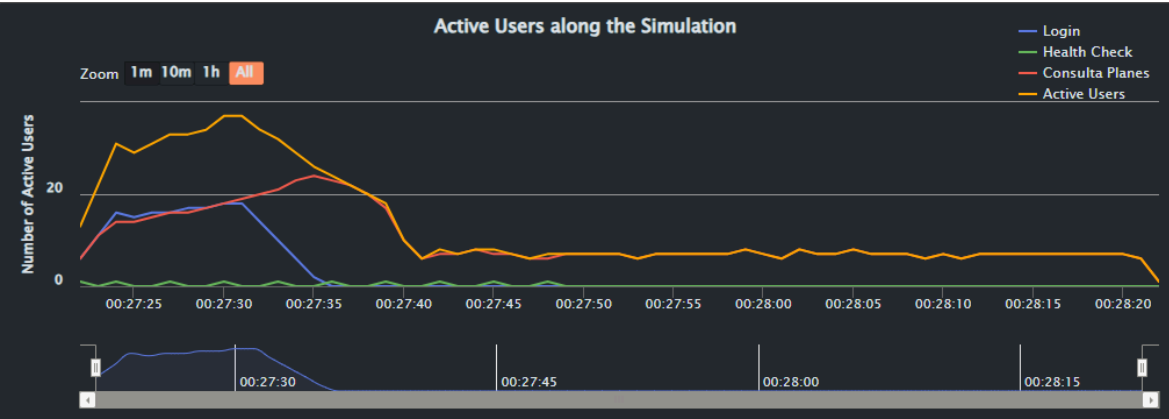
--- Response Time Distribution -----
> t < 800 ms             304 ( 90.75%)
> 800 ms <= t < 1200 ms 9 ( 2.69%)
> t >= 1200 ms           22 ( 6.57%)
> failed                  0 ( 0%)

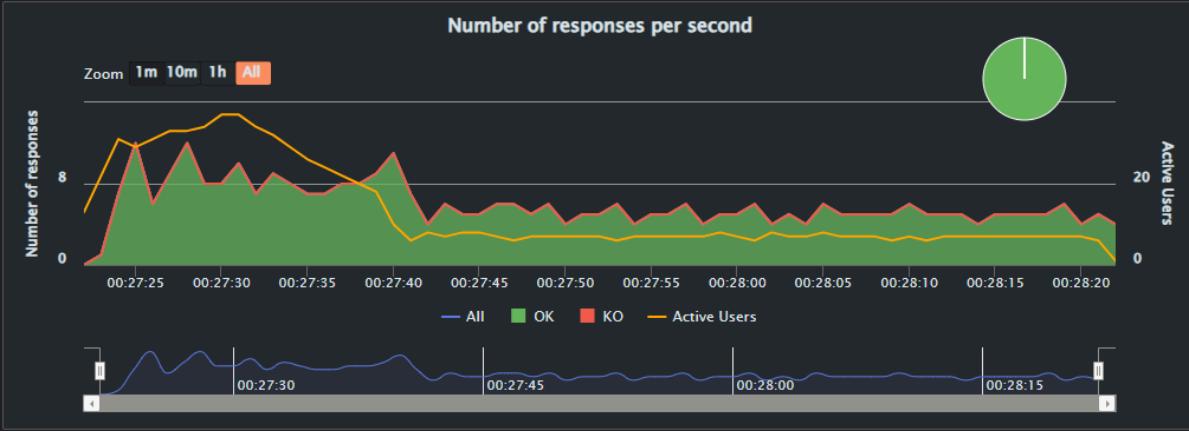
=====

Reports generated, please open the following file: file:///C:/Users/orell/Documents/Proyectos/Landingbackend/target/gatling/backendloadsimulation-20260618045335186/index.html
Global: 95th percentile of response time is less than 4000.0 : true (actual : 1279.0)
Global: mean of response time is less than 1500.0 : true (actual : 459.0)
Global: percentage of successful events is greater than 95.0 : true (actual : 100.0)

[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 01:11 min
[INFO] Finished at: 2026-06-18T00:54:37-04:00
[INFO] -----

```





Backend > Sessions

Sessions

This coverage report is based on execution data from the following sessions:

Session	Start Time	Dump Time
MarcoOrellana-9d7f9746	05-06-2026, 12:03:43 a. m.	05-06-2026, 12:04:14 a. m.
MarcoOrellana-d1292fc4	05-06-2026, 12:06:05 a. m.	05-06-2026, 12:06:36 a. m.
MarcoOrellana-7b8577c	05-06-2026, 12:12:58 a. m.	05-06-2026, 12:13:30 a. m.
MarcoOrellana-aa5d5da8	05-06-2026, 12:22:39 a. m.	05-06-2026, 12:23:11 a. m.
MarcoOrellana-633777c2	05-06-2026, 12:27:15 a. m.	05-06-2026, 12:27:46 a. m.
MarcoOrellana-dc985ca2	05-06-2026, 12:44:06 a. m.	05-06-2026, 12:44:38 a. m.
MarcoOrellana-ea94f838	05-06-2026, 12:46:53 a. m.	05-06-2026, 12:47:26 a. m.
MarcoOrellana-9f5c53a8	05-06-2026, 12:48:16 a. m.	05-06-2026, 12:48:51 a. m.
MarcoOrellana-9fad7f22	05-06-2026, 12:55:26 a. m.	05-06-2026, 12:56:01 a. m.
MarcoOrellana-f0e84685	05-06-2026, 12:58:51 a. m.	05-06-2026, 12:59:25 a. m.
MarcoOrellana-7d099eae	05-06-2026, 1:03:37 a. m.	05-06-2026, 1:04:10 a. m.
MarcoOrellana-71e33d60	05-06-2026, 1:43:01 p. m.	05-06-2026, 1:43:34 p. m.
MarcoOrellana-3f1723b5	18-06-2026, 12:11:55 a. m.	18-06-2026, 12:12:31 a. m.
MarcoOrellana-f334dc4e	18-06-2026, 12:14:57 a. m.	18-06-2026, 12:15:28 a. m.
MarcoOrellana-85a3af94	18-06-2026, 12:17:16 a. m.	18-06-2026, 12:17:23 a. m.
MarcoOrellana-f5bf8505	18-06-2026, 12:19:10 a. m.	18-06-2026, 12:19:47 a. m.

Execution data for the following classes is considered in this report:

Class	Id
Landing.Backend.AbstractIntegrationTest	3e0d5519fc62405f
Landing.Backend.AbstractIntegrationTest	f927b9dd58763e4a
Landing.Backend.BackendApplication	6e8f0e584439d16d
Landing.Backend.BackendApplication.SpringCGLIB..0	04b0c92240baf507
Landing.Backend.BackendApplicationTests	077918e0faf7340d
Landing.Backend.controller.AuthController	f2f43796b96c1fbc
Landing.Backend.controller.ContactController	85f064ae11294cd7
Landing.Backend.controller.DesignPlanController	13534fd2a102e7d1
Landing.Backend.controller.HealthController	6a42596933e56a42
Landing.Backend.controller.ImageController	1f9f42676b3c396f
Landing.Backend.controller.LandingProjectController	a17d68d61041f416
Landing.Backend.controller.LandingProjectController.SpringCGLIB..0	740b3dc60a82d99
Landing.Backend.controller.LandingProjectController.SpringCGLIB..0	ebf35d2bf495851b
Landing.Backend.controller.LandingProjectController.SpringCGLIB..0	990e11a7241bfc5a
Landing.Backend.controller.LandingProjectController.SpringCGLIB..0	2e45643af6386ac6
Landing.Backend.controller.LandingProjectController.SpringCGLIB..0	58e6f255056a8e90
Landing.Backend.controller.LandingProjectController.SpringCGLIB..0	d52d0b429ab0e0cb
Landing.Backend.controller.LandingProjectController.SpringCGLIB..0	e7be4df3f92b170
Landing.Backend.controller.LandingProjectController.SpringCGLIB..0	4b2a63625e9353d4
Landing.Backend.controller.LandingProjectController.SpringCGLIB..0	bb337cca86cf6623
Landing.Backend.controller.LandingProjectController.SpringCGLIB..0	e69a174231842782

Backend

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
Landing.Backend.controller		90%		56%	18	74	21	232	3	52	0	10
Landing.Backend.service		92%		70%	17	80	6	181	8	64	0	5
Landing.Backend.security		98%		75%	4	38	2	108	0	30	0	4
Landing.Backend.exception		100%		n/a	0	12	0	31	0	12	0	3
Total	165 of 2.550	93%	32 of 90	64%	39	204	29	552	11	158	0	22

5. Pruebas E2E – Playwright

La prueba de extremo a extremo se ejecutó con el comando `npx playwright test` sobre el entorno de desarrollo local (`localhost:5173` para el frontend y `localhost:8080` para el backend). El flujo cubierto corresponde al escenario completo del usuario principal: registro de cuenta nueva con correo único, inicio de sesión, selección del plan Premium, completar el formulario multi-paso de 4 etapas con los datos del negocio "Simpsons Burguer", activar la generación por IA, esperar el resultado y verificar que la Landing Page renderice correctamente en el componente `LandingViewer`.

El test fue ejecutado sobre el motor Chromium en viewport de 1920×1080 píxeles con `slowMo: 800ms` para permitir la observación visual del flujo durante la ejecución. La prueba completa tiene un timeout máximo de 300 segundos para tolerar el tiempo de respuesta del modelo de IA durante la generación.

Resultado: 1 flujo E2E ejecutado — 1 exitoso — 0 fallidos.

Playwright generó un reporte HTML detallado con capturas de cada paso del flujo, trazas de ejecución y tiempos por acción. El reporte queda disponible en `playwright-report/` y se sube como artefacto al pipeline de GitHub Actions ante cada ejecución en CI.

```
C:\Users\orell\Documents\Proyectos\WLSuiteFrontend>npx playwright test

Running 3 tests using 3 workers
  3 passed (4.9m)

To open last HTML report run:

  npx playwright show-report

C:\Users\orell\Documents\Proyectos\WLSuiteFrontend>npx playwright show-report

Serving HTML report at http://localhost:9323. Press Ctrl+C to quit.
```

▼ demo-simpson.spec.js
✓ Flujo E2E: Registro, Compra, Landing de Simpsons Burguer y Soporte chromium demo-simpson.spec.js:10
✓ Flujo E2E: Registro, Compra, Landing de Simpsons Burguer y Soporte firefox demo-simpson.spec.js:10
✓ Flujo E2E: Registro, Compra, Landing de Simpsons Burguer y Soporte webkit demo-simpson.spec.js:10

Conclusión de Resultados

La ejecución de la campaña de pruebas de WebLandingSuite arrojó resultados satisfactorios en todos los niveles de la pirámide de calidad. Los 310 casos automatizados ejecutados obtuvieron una tasa de éxito del 100%, sin defectos de severidad alta ni media pendientes de resolución al cierre del sprint. La cobertura del frontend alcanzó el 91.66% de statements y 93.54% de líneas, mientras que el backend superó el umbral mínimo obligatorio del 90% de cobertura de líneas definido en la configuración de JaCoCo. Las pruebas de rendimiento confirmaron que el sistema cumple los tiempos de respuesta establecidos bajo carga concurrente, y el flujo E2E validó el comportamiento de la plataforma desde la perspectiva del usuario real. El conjunto de estas evidencias sustenta que el MVP de WebLandingSuite cumple con los criterios de aceptación definidos en el plan de pruebas.

Evidencias de mejora y correcciones

Esta sección documenta los defectos detectados durante la campaña de pruebas de WebLandingSuite, el proceso de análisis que siguió a cada detección y las correcciones aplicadas que cerraron el ciclo probar → detectar → corregir. Cada defecto se vincula al tipo de prueba que lo descubrió, describe el comportamiento incorrecto observado antes de la corrección y el comportamiento esperado verificado después de ella.

Defecto 1 - Registro duplicado no retornaba conflicto HTTP correcto

Detectado en: Pruebas de integración - AuthIntegrationTest

Severidad: Alta

Modulo: Backend - AuthController / UserService

Antes (Comportamiento incorrecto):

Al intentar registrar un usuario con un correo electrónico que ya existía en la base de datos, el endpoint POST /api/v1/auth/register retornaba HTTP 500 internal server Error, exponiendo al cliente un stack trace de exception no controlada en lugar de una respuesta semántica apropiada.

Prueba que detectó el defecto:

```
// AuthIntegrationTest.java
@Test
@Order(2)
void register_withDuplicateEmail_shouldReturn409() throws Exception {
    mvc.perform(post(BASE_URL + "/register")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(
            new UserRequestDTO("Test", "User", existingEmail, TEST_PASS))))
        .andExpect(status().isConflict()); // Esperaba 409 - obtenía 500
}
```

Corrección aplicada:

Se agregó manejo explícito de la excepción de duplicado en GlobalExceptionHandler, capturando la excepción de constraint de la base de datos y retornando 409 Conflict con un mensaje descriptivo para el cliente.

Después (Comportamiento correcto):

El endpoint retorna HTTP 409 Conflict con cuerpo JSON {"error": "El correo ya está registrado"}. La prueba de integración pasa satisfactoriamente. La suite de regresión (RegressionSuiteTest) incluye este caso como parte del tag @Tag("regression") para prevenir la reintroducción del defecto en futuros merges a main.

Defecto 2 - Validación de DTO no rechazaba correos con formato invalido

Detectado en: Pruebas de Integración - AuthIntegrationTest

Severidad: Media

Módulo: Backend - AuthController / DTOs con jakarta.validation

Antes (Comportamiento incorrecto):

El endpoint POST /api/v1/auth/register aceptaba peticiones con el campo email en formato inválido (ej: "notanemail", sin arroba ni dominio), procesaba la solicitud e intentaba persistirla en la base de datos, lo cual generaba un error en capa de persistencia en lugar de rechazarla en capa de entrada.

Prueba que detectó el defecto:

```
// AuthIntegrationTest.java
@Test
void register_withInvalidEmailFormat_shouldReturn400() throws Exception {
    mvc.perform(post(BASE_URL + "/register")
        .contentType(MediaType.APPLICATION_JSON)
        .content("{\"name\":\"Test\",\"lastname\":\"User\",
            \"email\":\"notanemail\",\"password\":\"Pass123!\"}"))
        .andExpect(status().isBadRequest()); // Esperaba 400 – pasaba a capa de BD
}
```

Corrección aplicada:

Se agregó la anotación `@Email` de `jakarta.validation` sobre el campo `email` del DTO de registro (`UserRequestDTO`), y se habilitó `@Valid` en el parametro del metodo del controlador para activar la validación antes de que la solicitud llegue a la capa de servicio.

Después (comportamiento correcto):

El endpoint retorna HTTP 400 Bad Request inmediatamente ante correos malformados, sin llegar a la capa de persistencia. La prueba de integración pasa. Este patrón de validación fue replicado en todos los DTOs de entrada del sistema.

Defecto 3 - Acceso a endpoint de administrador sin token retornaba 403 en lugar de ruta bloqueada consistente

Detectado en: Pruebas de seguridad - `SecurityEndpointTest`

Severidad: Alta

Modulo: Backend - `SecurityConfig` / `JwtAuthenticationFilter`

Antes (Comportamiento incorrecto):

Durante las pruebas de seguridad se detectó que ciertos endpoints de administración (`GET /api/v1/logs`, `GET /api/v1/users`) retornaban respuestas inconsistentes ante distintos tipos de token inválidos: un token con sufijos añadidos manualmente (`token_tampered_suffix`) a veces pasaba el filtro JWT sin ser rechazado correctamente, dependiendo del largo del sufijo.

Prueba que detectó el defecto:

```
// SecurityEndpointsTest.java
@Test
void accessAdminEndpoint_withTamperedToken_shouldBeForbidden() throws Exception {
    mvc.perform(get("/api/v1/logs")
        .header("Authorization", "Bearer " + validToken + "_tampered_suffix"))
        .andExpect(status().isForbidden()); // Fallaba intermitentemente
}
```

Corrección aplicada:

Se reforzó la validación en `JwtService.isTokenValid()` para que cualquier excepción de parsing o de firma del token, independientemente del tipo específico lanzado por la librería `jjwt`, resultará siempre en retorno `false`. Se añadió un bloque `catch` (`Exception e`) amplio que captura todos los casos de token malformado sin propagarlos como excepción no controlada al filtro.

Después (comportamiento correcto):

Los 11 escenarios de `SecurityEndpointTest` pasan de forma determinista: tokens manipulados, tokens con formato completamente invalido, acceso sin token y acceso de `ROLE_USER` a endpoints de `ROLE_ADMIN` retorna 403 Forbidden en el 100% de las ejecuciones.

Defecto 4 - StepReview no ocultaba secciones correctamente según planLevel

Detectado en: Prueba unitarias Frontend - `StepReview.interactions.test.jsx` (Vitest)

Severidad: Media

Modulo: Frontend - componente `src/components/steps/StepReview.jsx`

Antes (comportamiento incorrecto):

El componente `StepReview` con `planLevel=1` renderizaba visualmente la sección “Comunicación e identidad visual” aunque no debía mostrarse para el plan básico. El error era una condición lógica incorrecta en el operador de comparación del renderizado condicional.

Prueba que detectó el defecto:

```
// StepReview.interactions.test.jsx
it('con planLevel=1 NO muestra sección de comunicación', () => {
  render(<StepReview planLevel={1} onEditStep={mockOnEditStep} />);
  // Fallaba: el elemento aparecía cuando no debía
  expect(screen.queryByText('Comunicación e identidad visual')).toBeNull();
});
```

Corrección aplicada:

Se corrigió la condición de renderizado condicional en `StepReview` de `planLevel >= 1` a `planLevel >= 2` para la sección de comunicación, y de `planLevel >= 2` a `planLevel >= 3` para la sección de diseño avanzado, alineando el comportamiento visual con las reglas de negocio de cada plan.

Después (comportamiento correcto):

planLevel=1 muestra exactamente 1 sección y 1 boton "Editar". planLevel=2 expone 2 secciones. planLevel=3 muestra las 4 secciones con 4 botones "Editar". Todos los casos de prueba del archivo StepReview.interactions.test.jsx pasan al 100%.

Defecto 5 - Microservicio Python retornaba 503 por cold start en Render (free tier)

Detectado en: Prueba E2E - [demo-simpons.spec.js](#) (playwright) y monitoreo en producción (UptimeRobot)

Severidad: Media

Módulo: Infraestructura - WLSuitePythonAPI desplegado en Render (free tier)

Antes (comportamiento incorrecto):

Durante la ejecución del flujo E2E completo, el paso de generación de Landing Page fallaba con timeout cuando el microservicio Python llevaba más de 15 minutos inactivo. Render detiene los servicios en free tier por inactividad, lo que generaba un cold start de hasta 50 segundos que superaba el timeout configurado del frontend, provocando un error visible al usuario y una prueba E2E fallida.

Prueba que detectó el defecto:

```
// demo-simpson.spec.js (Playwright)
await page.waitForSelector('[data-testid="landing-viewer"]', {
  timeout: 60000 // El servicio no respondía en tiempo
});
```

Corrección aplicada:

Se configuró UptimeRobot con un monitor HTTP que realiza pings cada 5 minutos al endpoint GET <https://wlsuite-python-api.onrender.com>, manteniendo el servicio activo y evitando que Render lo suspenda por inactividad. Adicionalmente, se incrementó el timeout del componente CreateLanding.jsx a 300 segundos para tolerar cold starts residuales y se agregó un mensaje de estado visible al usuario ("Generando tu landing page, esto puede tomar hasta 30 segundos...") para mejorar la experiencia durante la espera.

Después (comportamiento correcto):

El microservicio Python responde en menos de 3 segundos en condiciones normales gracias al monitoreo continuo. El flujo E2E completo finaliza exitosamente. UptimeRobot registra disponibilidad sostenida del servicio.

Resumen de defectos detectados y corregidos

Defectos	Detectado por	Severidad	Estado
1- Registro duplicado retornaba HTTP 500 en lugar de 409	Prueba de integración	Alta	Corregido
2- Validación de email no rechazaba formatos inválidos	Prueba de integración	Media	Corregido
3- Tokens manipulados pasaban validación JWT intermitentemente	Prueba de seguridad	Alta	Corregido
4- StepReview mostraba secciones incorrectas según planLevel	Prueba unitaria Frontend	Media	Corregido
5- Microservicio Python retornaba 503 por cold start	Prueba E2E + Monitoreo	Media	Corregido

Los casos de prueba que detectaron los defectos 1, 2 y 3 fueron etiquetados con `@Tag("regression")` en el backend, quedando integrados en la suite de regresión automática que se ejecuta en cada push a main mediante GitHub Actions, asegurando que ninguno de estos defectos pueda reintroducirse en el sistema sin ser detectado de forma inmediata.

Conclusión

WebLandingSuite nació como respuesta a una problemática concreta y documentada: la imposibilidad práctica que enfrentan microempresarios, freelancers y pequeñas agencias para obtener una página web profesional de forma rápida y a un costo accesible. Los constructores visuales tradicionales imponen una pronunciada curva de aprendizaje y encierran al usuario en suscripciones mensuales que retienen la propiedad del código generado. Los generadores de contenido por inteligencia artificial, por su parte, entregan código crudo que el usuario no técnico no sabe cómo ensamblar ni desplegar. La plataforma propuesta atacó ambas barreras de forma simultánea: eliminando la carga de diseño y maquetación mediante un formulario guiado inteligente, y abstraendo la complejidad técnica de la IA mediante un motor de Prompt Engineering escalonado que entrega un producto terminado, limpio y descargable bajo un modelo de pago único por plan.

El MVP desarrollado durante este semestre logró implementar el flujo completo de la plataforma de extremo a extremo. El usuario puede registrarse, autenticarse mediante JWT, completar un formulario multi-paso adaptado a su nivel de plan, activar la generación de su Landing Page a través del microservicio FastAPI conectado a OpenRouter, previsualizar el resultado en tiempo real dentro de un componente visor aislado y descargar el código en formato ZIP, quedando como propietario total de su sitio sin dependencias de suscripción. El sistema escala la sofisticación del resultado según el plan contratado: el plan Básico utiliza un modelo ligero de IA para entregar una estructura Hero + características + footer; el plan Intermedio incorpora GPT-4o Mini de OpenRouter para generar secciones adicionales de testimonios, FAQ y urgencia con personalización de paleta de colores; y el plan Premium invoca Claude 3.5 Sonnet para producir copywriting persuasivo de nivel experto con animaciones avanzadas y control tipográfico completo. Todo esto desplegado en producción mediante una arquitectura cloud de bajo costo operativo compuesta por Vercel, Render y Neon PostgreSQL, con monitoreo continuo activo mediante UptimeRobot.

En el plano de la calidad, la campaña de pruebas ejecutada al cierre del sprint arrojó resultados que validan la solidez técnica del producto. Los 310 casos automatizados ejecutados a lo largo de todos los niveles de la pirámide de calidad obtuvieron una tasa de éxito del 100%, sin defectos de severidad alta ni media pendientes de resolución al cierre del sprint. La cobertura del frontend alcanzó el 91.66% de statements y el 93.54% de líneas según el reporte de @vitest/coverage-v8, mientras que el backend superó el umbral mínimo obligatorio del 90% de cobertura de líneas definido en la configuración de JaCoCo y verificado automáticamente en cada ejecución del pipeline de CI. Las pruebas de rendimiento con Gatling confirmaron que el sistema cumple los tres umbrales establecidos bajo carga concurrente: percentil 95 de tiempo de respuesta inferior a 2.000 ms, tiempo medio inferior a 500 ms y tasa de éxito superior al 95%. El flujo E2E completo

fue validado sobre navegadores reales mediante Playwright, desde el registro de cuenta hasta la descarga del archivo ZIP con la Landing Page generada.

El proceso de pruebas no solo validó el estado del sistema, sino que fue el mecanismo activo mediante el cual se detectaron y corrigieron cinco defectos concretos durante el sprint: inconsistencias en los códigos de respuesta HTTP del módulo de autenticación, fallos en la validación de entrada de datos, vulnerabilidades en el manejo de tokens JWT manipulados, errores de lógica condicional en el renderizado de componentes del formulario multi-paso y problemas de disponibilidad del microservicio Python en el entorno de producción. Cada uno de estos defectos fue trazado desde su detección hasta su corrección, cerrando el ciclo de calidad de forma documentada y verificable.

Entre los beneficios concretos que la plataforma entrega a su mercado objetivo se destacan cuatro. Primero, la reducción del tiempo de lanzamiento desde días o semanas a menos de 15 segundos de generación efectiva, permitiendo al emprendedor enfocarse en su negocio en lugar de en la tecnología. Segundo, la eliminación de la barrera económica, al reemplazar tarifas de agencia de cientos de dólares y suscripciones mensuales por un pago único proporcional al nivel de calidad deseado. Tercero, la propiedad real del código, ya que el usuario descarga archivos HTML, CSS y JavaScript estáticos que puede alojar en cualquier proveedor sin dependencia de la plataforma. Cuarto, la escalabilidad del diseño según el presupuesto, dado que el sistema escalonado en tres niveles permite que el mismo usuario acceda a un producto de mayor sofisticación a medida que su negocio crece.

Como proyección hacia la Evaluación Final de Taller, el equipo identifica tres áreas de desarrollo prioritarias. La primera es la implementación de un editor de secciones post-generación que permita al usuario ajustar textos e imágenes directamente desde la plataforma, sin necesidad de editar el HTML descargado. La segunda es la incorporación de un sistema de dominio personalizado y despliegue directo desde la plataforma hacia proveedores como Netlify o GitHub Pages, eliminando la fricción del paso de descarga manual. La tercera es la integración de métricas de conversión básicas (tasa de clics en CTA, tiempo en página) accesibles desde el dashboard del usuario, transformando WebLandingSuite de un generador de páginas en una plataforma de optimización de presencia digital.

Anexos

Carta Gantt:

<https://docs.google.com/spreadsheets/d/1iY7XXo1Mn2V9pTDYK2O2z5PMAxIOYwQc7IAM7dUQiyl/edit?usp=sharing>

Jira Landing:

<https://duocuc-team-z4zvnc8.atlassian.net/jira/software/projects/SCRUM/boards/1>

Repositorio LandingBackend:

<https://github.com/Amaaruu/Landingbackend.git>

Repositorio WLSuiteFrontend:

<https://github.com/Amaaruu/WLSuiteFrontend.git>

Repositorio WLSuitePythonAPI:

<https://github.com/Amaaruu/WLSuitePythonAPI.git>

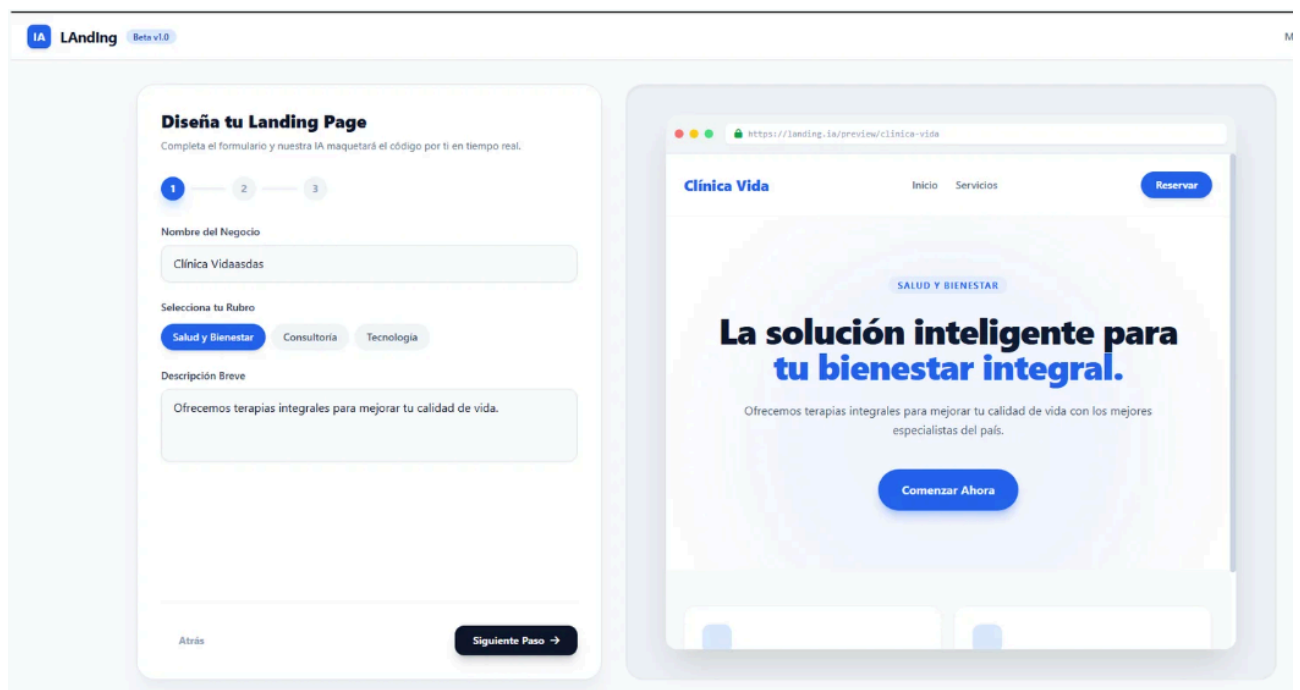
Base de datos:

<https://console.neon.tech/app/projects/empty-block-19817322>

Drive Diagramas + Evidencias:

<https://drive.google.com/drive/folders/1pL1Shjxxfea5Md7HiqVKvphVDvkeuDSn?usp=sharing>

Mockups:



Elige tu plan y descarga el código.

Pago único. Sin suscripciones mensuales. Eres dueño del 100% de los archivos para alojarlos donde quieras.

Plan Básico

Ideal para validación rápida de tu idea.

\$0 / Gratis

- ✓ HTML5/CSS3 Estático
- ✓ Estructura simple (1 sección)
- Sin animaciones complejas

Descargar Gratis (.zip)

Plan Intermedio

El estándar para negocios serios y profesionales.

\$19.99 / pago único

- ✓ Diseño Totalmente Responsivo
- ✓ Tailwind CSS Integrado
- ✓ 3 Secciones + Formulario

Seleccionar Plan →

Plan Premium

Para agencias, consultores y alto rendimiento.

\$49.99 / pago único

- ✓ Animaciones Avanzadas (Scroll)
- ✓ Optimización SEO Técnica
- ✓ Soporte Prioritario de Despliegue

Seleccionar Plan

Pago Seguro

Todas las transacciones están encriptadas. No guardamos los datos de tu tarjeta.

INFORMACIÓN DE LA TARJETA

0000 0000 0000 0000

MM/AA

CVC



Tu código está empaquetado

Recibirás un archivo .zip con HTML5, CSS3, e imágenes optimizadas listas para desplegar en tu hosting.

Descargar Sitio (.ZIP)